

《机电控制系统实践》课程——

实验指导书



清华大学机械工程系实验室

目 录

实验指导书.....	1
目 录.....	I
0 实验要求（非常重要！）	1
1 STM32 单片机教学系统	3
1.1 硬件环境介绍	3
1.2 开发环境使用	5
1.2.1 开发环境搭建	5
1.2.2 使用 STM32CubeIDE 建立一个自己的程序	10
1.2.3 将编译的程序文件写入单片机中	19
2 单片机实验一：RGB 灯闪烁和蜂鸣器报警	24
2.1 实验目的	24
2.2 实验原理	24
2.2.1 RGB 灯	24
2.2.2 蜂鸣器	25
2.2.3 按键	26
2.3 实验内容	27
2.3.1 RGB 灯闪烁	27
2.3.2 控制蜂鸣器发声	28
3 单片机实验二：运动控制（基础）	30
3.1 实验目的	30
3.2 实验原理	30
3.2.1 电机	30
3.2.2 电机驱动模块	30
3.3 实验内容	32
3.3.1 直流电机驱动	32
3.3.2 PWM 波调速	34
4 单片机实验三：运动控制（提高）	37

4.1 实验目的	37
4.2 实验原理	37
4.2.1 转弯调速原理	37
4.2.2 变速运动原理	38
4.3 实验内容	38
4.3.1 轨迹调控	38
4.3.2 变速运动	38
5 单片机实验四：黑线循迹	39
5.1 实验目的	39
5.2 实验原理	39
5.2.1 红外对管传感器原理	40
5.2.2 黑线循迹原理	40
5.2.3 黑线循迹动作逻辑原理	41
5.3 实验内容	42
6 单片机实验五：红外避障	45
6.1 实验目的	45
6.2 实验原理	45
6.2.1 红外线发射/接收二极管原理	45
6.2.2 红外避障电路原理	46
6.2.3 避障动作逻辑原理	47
6.3 实验内容	48
6.3.1 红外传感	48
6.3.2 避障动作	48
7 单片机实验六：视觉识别（基础）	49
7.1 实验目的	49
7.2 实验原理	49
7.2.1 开发环境使用	49
7.2.2 颜色识别	51
7.2.3 数字识别	51
7.2.4 自主学习分类（选做）	51

7.3 实验内容	52
7.3.1 颜色识别	52
7.3.2 数字识别	53
7.3.3 自主学习分类（选做）	55
8 单片机实验七：循迹、避障、视觉综合	58
8.1 实验目的	58
8.2 实验原理	58
8.3 实验内容	59
8.3.1 按键切换	59
8.3.2 循迹避障	59
8.3.3 综合实验	59
附录 A STM32 寄存器分布表.....	62
A.1 系统架构	62
A.2 时钟控制寄存器（RCC_CR）	63
A.3 时钟配置寄存器	64
A.4 时钟中断寄存器	66
A.5 APB2 外设复位寄存器.....	68
A.6 APB1 外设复位寄存器.....	70
A.7 AHB 外设时钟使能寄存器	72
A.8 APB2 外设时钟使能寄存器.....	73
A.9 APB1 外设时钟使能寄存器.....	74
A.10 备份域控制寄存器	77
A.11 控制/状态寄存器	78
附录 B 单片机系统原理图	79
B.1 开发板实物引脚分配图	79
B.2 开发板原理图	80

0 实验要求（非常重要!）

以下 9 个实验课要求，每个要求都会纳入实验成绩考核环节，请认真阅读并严格遵守：

- 1) 每次实验课开始前要先找助教签到，会如实记录迟到时间；
- 2) 实验中应以组内同学交流为主，允许前后左右桌简单交流，不允许课堂内自由走动；
- 3) 实验中允许提问，但请谨慎使用提问权限，控制提问次数。实验应认真预习，实验过程中学会自己解决问题，最好的永恒的答案是——Google 永远是你的好朋友；
- 4) 实验中严禁未经同意而私自使用其他组的任何设备完成实验，有关仪器问题请向助教寻求帮助；
- 5) 关于实验验收：实验完成后，请认真对照检查，确保无误后请求验收；请谨慎考虑重复验收的可能；
- 6) 实验室是公用的，下一节课的同学也可能马上进行实验，故请推己及人，要求每次实验完成后，收拾实验台恢复原貌，并经助教检查批准后方可离开；
- 7) 每次实验应按时完成。最多允许本次实验拖延到下次实验时间验收(最后一次实验无法推迟)，延迟验收情况会如实记录。允许并鼓励通过自学等方式提前完成实验；
- 8) 关于实验请假：每次请假应至少课前两个小时给助教发邮件说明请假缘由并提供有效证明文件；
- 9) 每次实验后要整理实验内容作为最终课程报告的一部分。最终提交文件包括 **C 文件** 程序源代码及 **PDF 格式** 的课程报告。所有文件压缩打包后提交。要求**每组独立完成报告、独立提交**。

PDF 格式的课程报告（参考报告范例）内容要求如下：

- 1) 基本信息：包括姓名、学号等；
- 2) 实验内容与程序设计：
 - ① 设计背景、目标、指标与内容；
 - ② 需求分析报告：开发环境、功能需求、接口需求等；
 - ③ 功能设计说明：硬件配置、程序设计、工作区（赛道）设计；
 - ④ 程序流程图与源代码：结合流程图（推荐使用 Visio 软件绘图）说

明你的程序设计方案，并配以简明扼要的文字说明；

⑤ 测试报告：简要描述各功能验收指标、测试原理、实验步骤（硬件连线、调试步骤）、实验现象并对现象做出解释，设计是否完成了预期目标；

⑥ 参考文献；

3) 实验收获：可以是成功经验，也可以是失败教训；

4) 实验建议：对实验改进的建议，没有可以不写。

表 0.1 单片机实验安排（根据课程实际情况灵活调整）

序号	实验内容	实验日期
1	LED 闪烁、按键启动和蜂鸣器报警	9 月 4 日上午
2	运动控制（基础）	9 月 4 日下午
3	运动控制（提高）	9 月 5 日上午
4	黑线循迹	9 月 5 日下午
5	红外避障	9 月 8 日上午
6	视觉识别	9 月 8 日下午
7	循迹、避障、视觉综合	9 月 9 日上午

1 STM32 单片机教学系统

学好单片机，需要做好两部分工作。一是掌握扎实的单片机基础理论，这部分包括单片机的工作原理、中断的工作原理、IO 模块以及定时器工作原理等。二是在理论的指导下进行编程实践，在实践中不断强化对单片机的认识。单片机学习的最终目的在于使用，因此实践是以上两部分学习的重点。

1.1 硬件环境介绍

为了保证实验的顺利完成，请在实验前对照以下实验硬件清单检查硬件是否完整。

表 1.1 实验硬件清单

序号	元件名称	数量
1	STM32 单片机开发板	1 块
2	锂电池	2 块
3	OLED 显示屏	1 块
4	轮胎	4 个
5	310 编码电机	4 个
6	电机固定支架	2 个
7	电池底板 (附带红外避障、RGB 灯)	1 块
8	上层扩展平台	1 个
9	螺丝刀+螺丝铜柱包	1 套
10	红外遥控器	1 个
11	超声波模块	1 个
12	四路巡线模块	1 个
13	XH2.54 排线 (6pin 双头)	1 根
14	K210 视觉识别模块	1 套
15	K210 加高支架	1 个
16	充电器	1 根
17	存储卡	1 个
18	读卡器	1 个
19	数据线	1 根

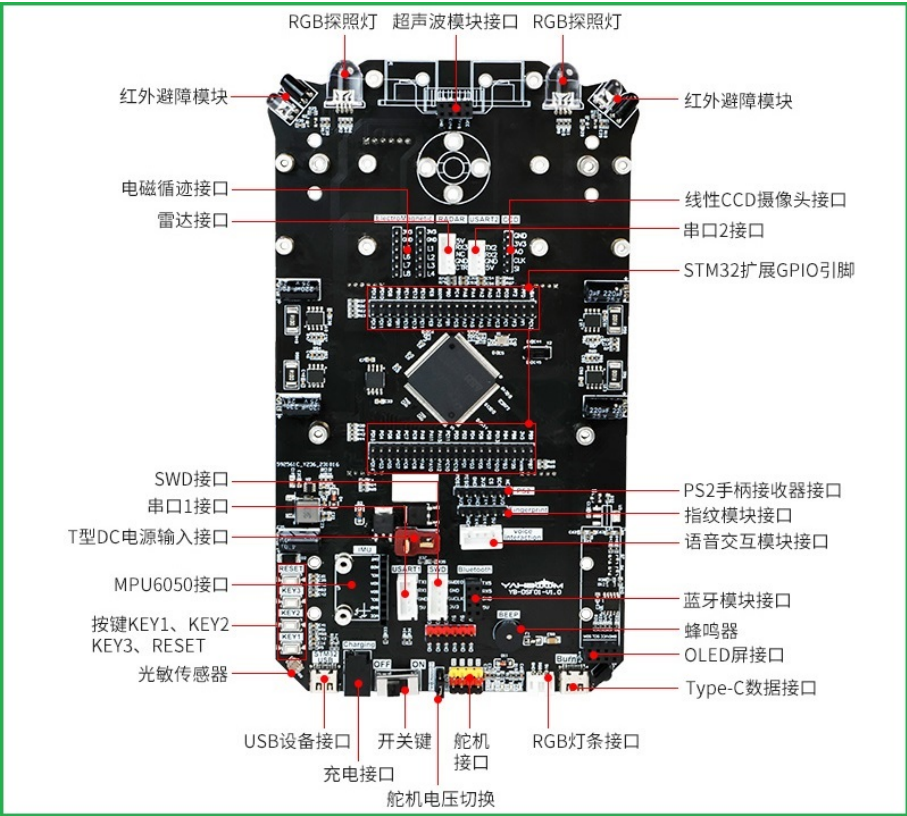


图 1.1 单片机开发板实物图



图 1.2 各配件模块

1.2 开发环境使用

1.2.1 开发环境搭建

单片机的开发平台有很多，这里选用了 STM32CubeIDE 开发平台，下面就对此开发平台作一个介绍。

IDE 是集成开发环境（Integrated Development Environment）的缩写，用于集成各种开发工具和功能，提供统一的开发环境。STM32CubeIDE 是基于 Eclipse 的集成开发环境，专门用于开发 STM32 应用程序。主要特点包括：整合了 CubeMX 配置工具；提供了丰富的插件和工具链支持；开发人员可直接进行代码编写、调试和测试。

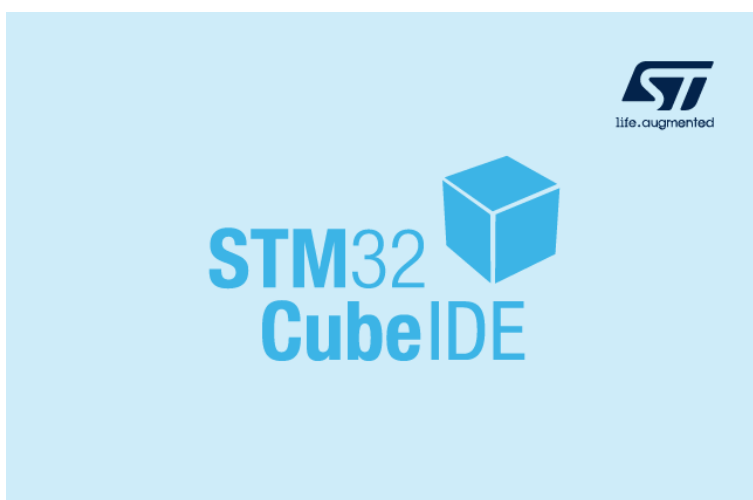
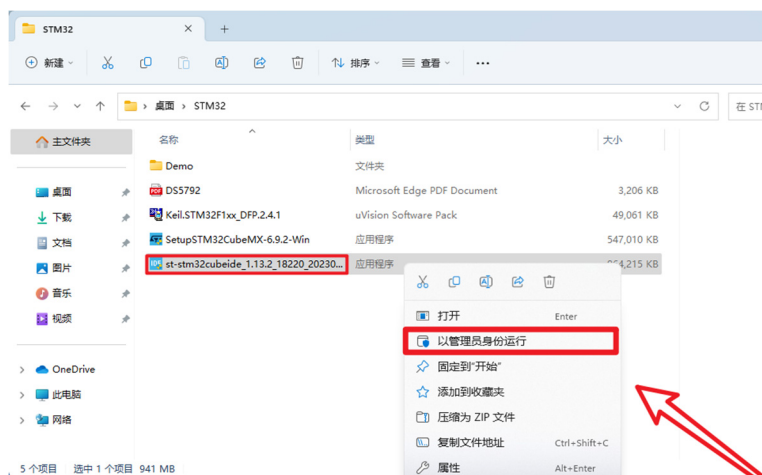


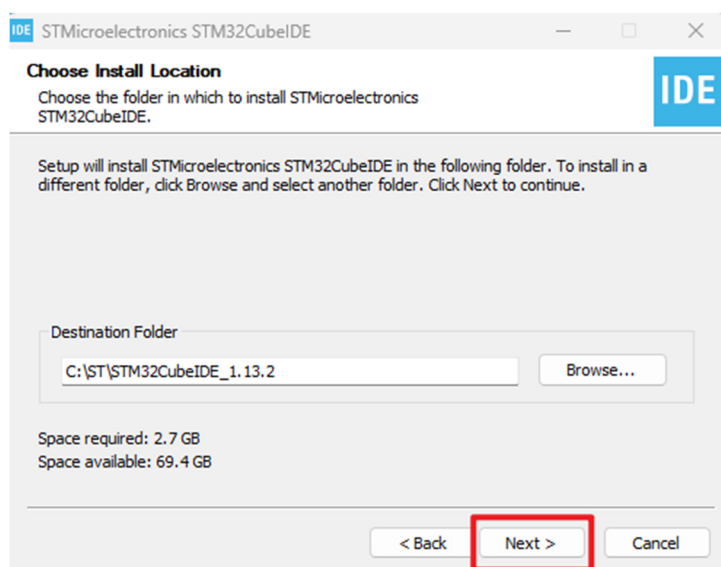
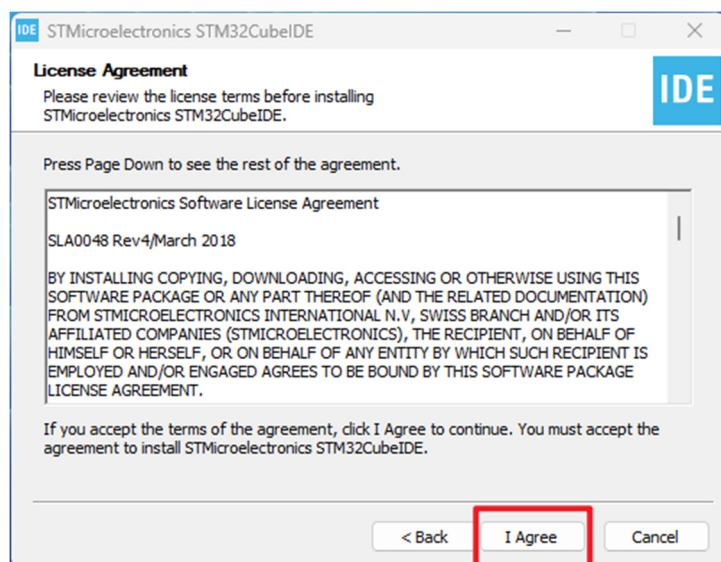
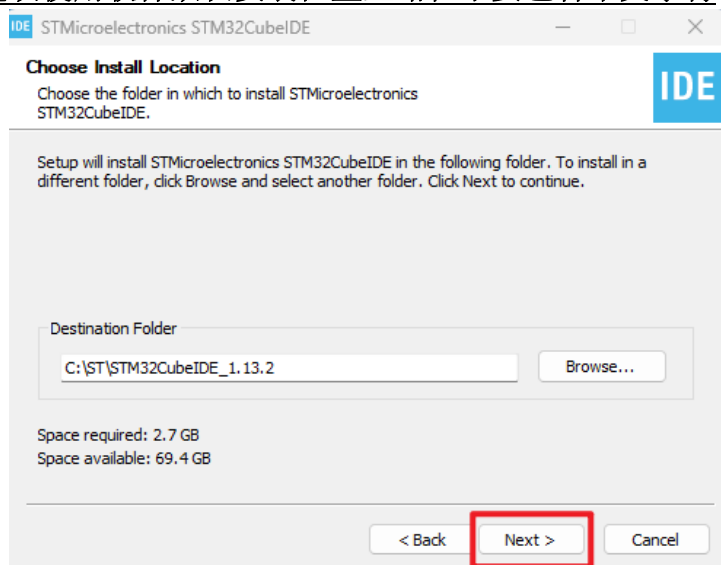
图 1.3 STM32 CubeIDE

软件安装时，以管理员身份打开安装包，若打开软件安装包出现“NSIS ERROR: error launching installer”错误提示：请将安装包放入不含中文字符的路径下，建议命名为“STM32”。

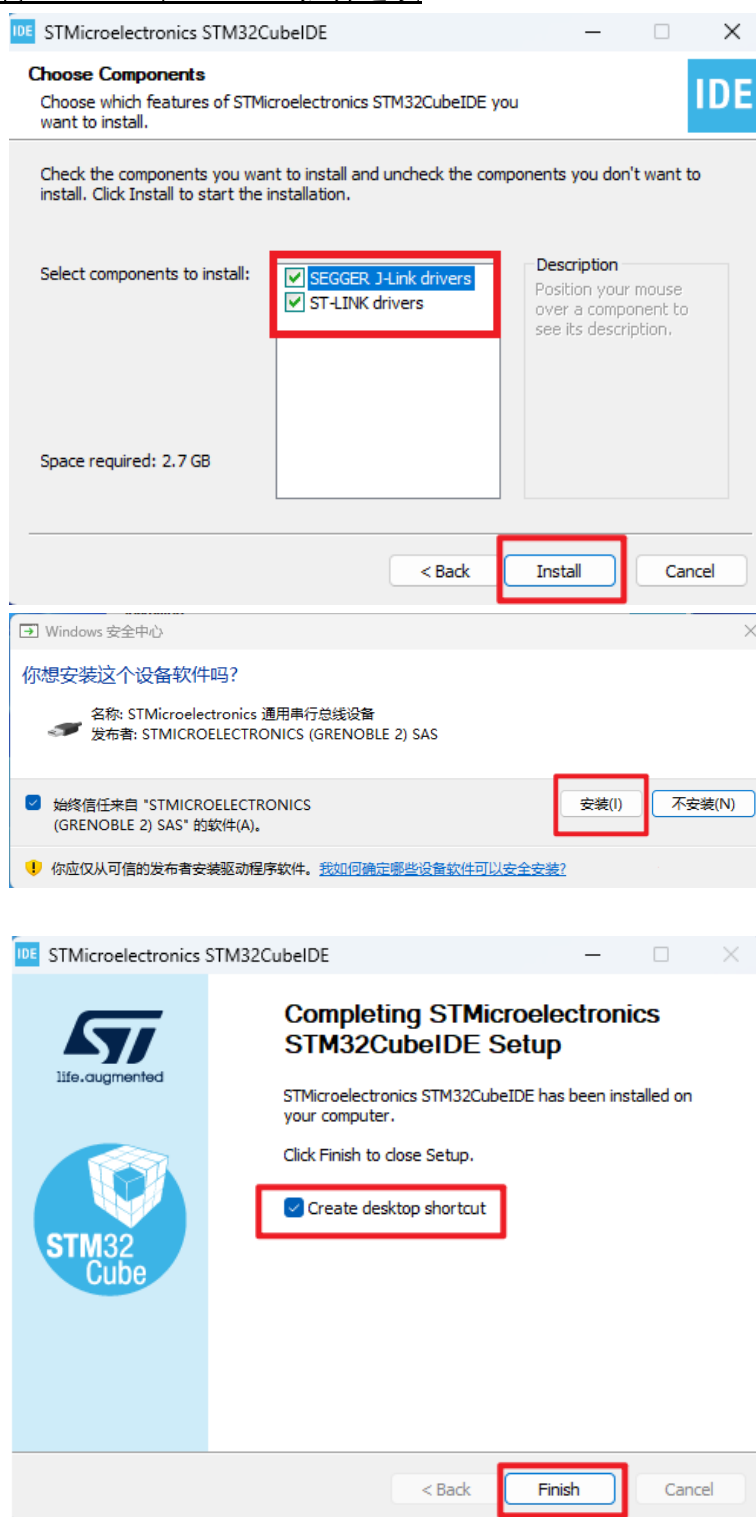
。



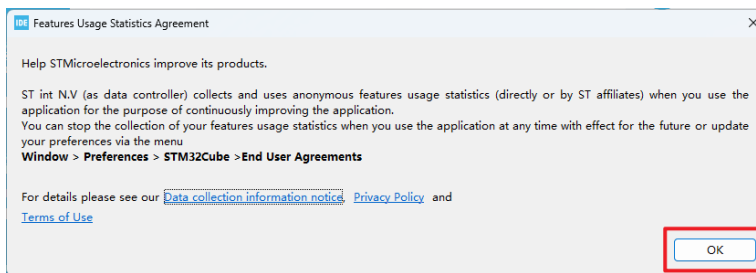
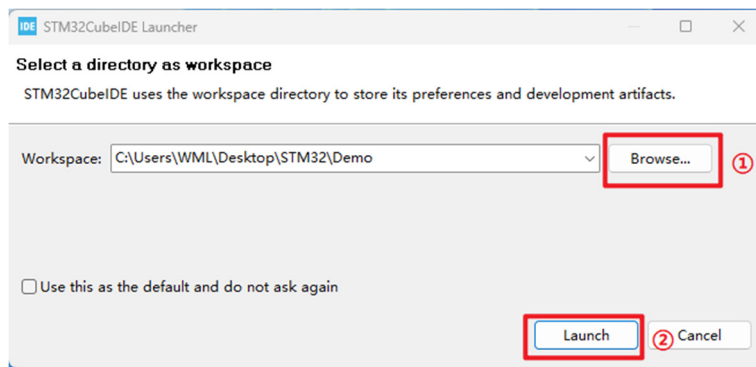
以下都建议使用软件默认安装位置且路径不要包含中文字符



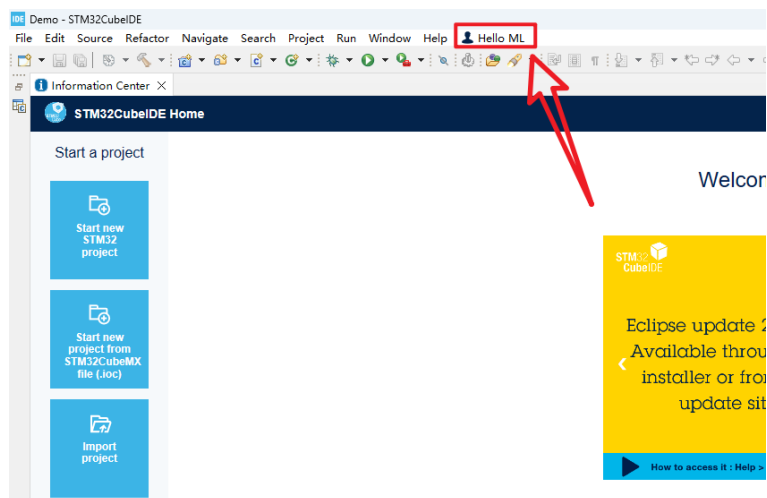
勾选安装 ST-Link 和 J-Link 驱动选项



安装完成后，进行软件配置。首先，双击桌面应用，选择工作空间（自己可以新建一个文件夹，文件夹路径不能含有中文字符，建议为上述建立的“STM32”文件夹）。



其次，注册-登录账号（需要先注册，验证邮件注意查收，可能位于【垃圾邮件】位置）：



User Authentication Dialog

Already registered?

Enter your e-mail address and password to login your myST user.

E-mail address

Password

☐ Remember me on this computer.

Login

[Forgot password?](#)

New user?

myST brings you a set of personalized features:

- Participate to ST Events
- Stay informed with ST eNewsletters
- Get help with ST Online Support
- Discuss on the ST Community
- Benefit from our Online Design Tools
- Download Software
- Order free samples
- Manage your weekly product updates
- Buy ST Products & Tools

Create Account

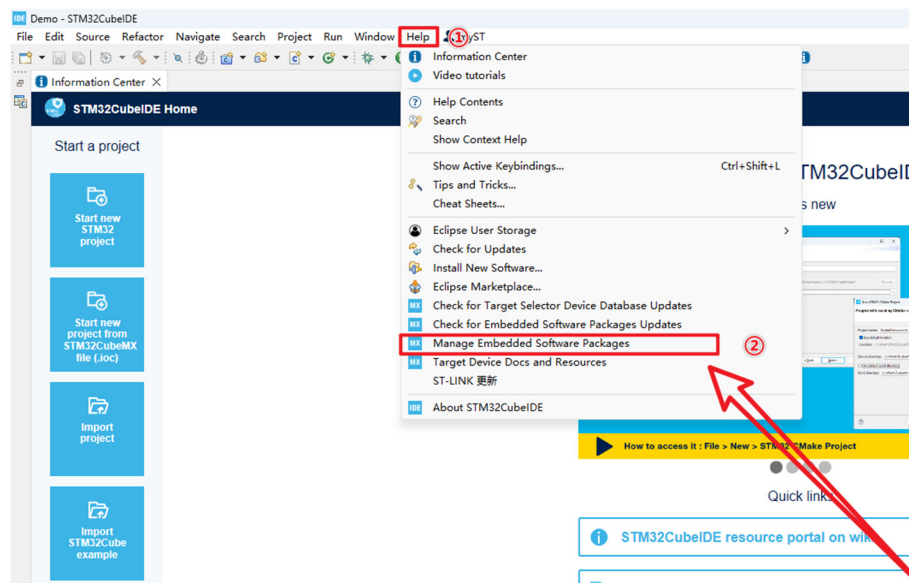
Feedback

安装固件包：

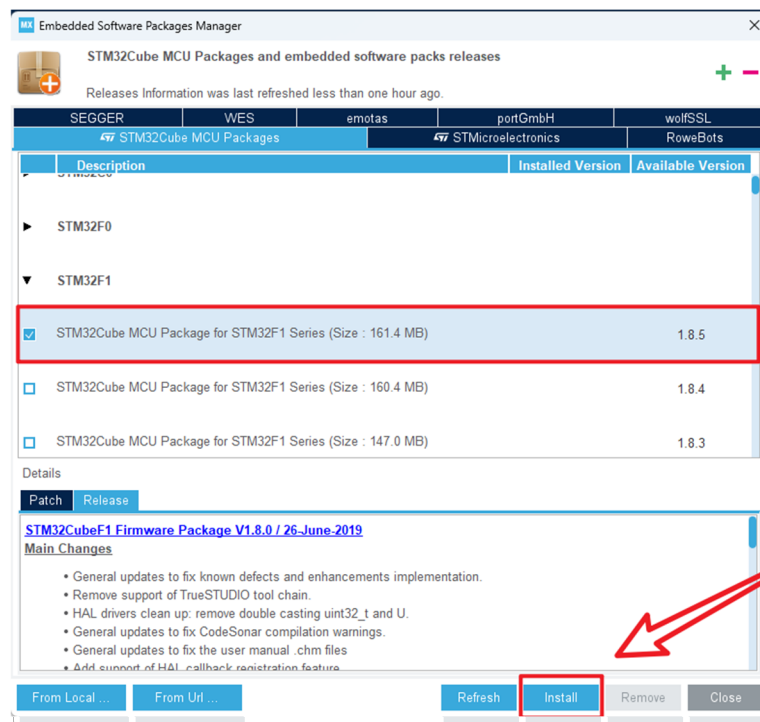
Help → Manage embedded software packages, 安装 STM32F1 系列最新固件包。

请注意：

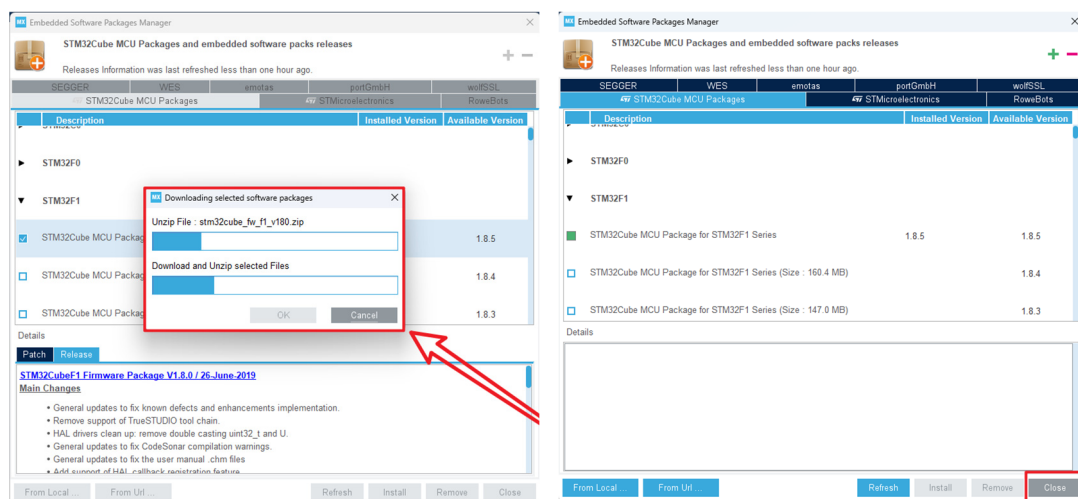
1. 未登录账号是无法安装芯片固件包的。
2. 安装或卸载都是选中芯片固件包前面的方框，单击该选项是不会有安装和卸载选项的。



选择最新的固件包（1.8.6）



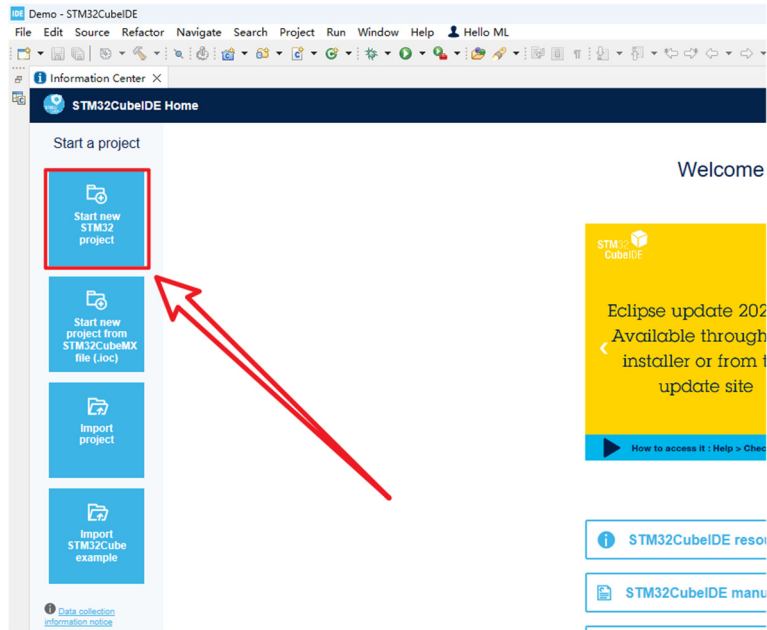
等待下载完毕后，关闭界面。



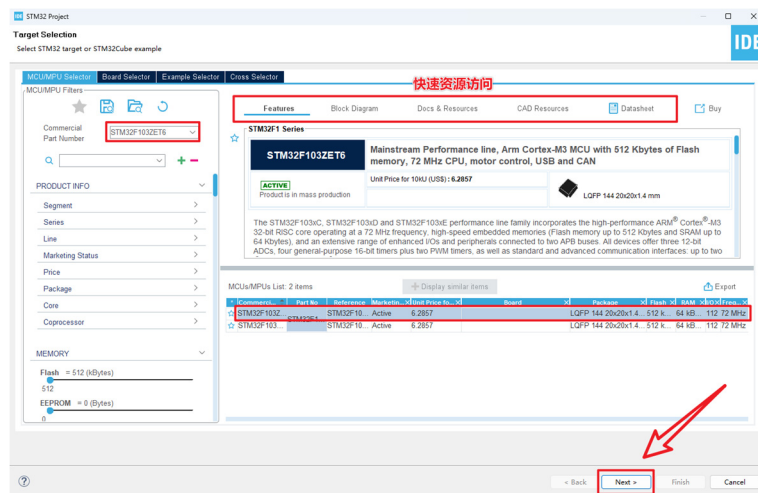
1.2.2 使用 STM32CubeIDE 建立一个自己的程序

STM32CubeIDE 既可进行引脚配置，又可以直接进行代码编写和调试。下面介绍一下建立一个工程项目的步骤。

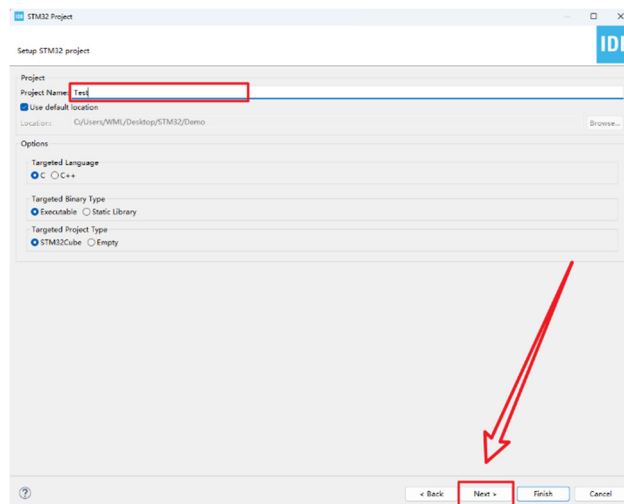
第一步，新建工程。一般首次使用该软件才通过这个选项新建工程。

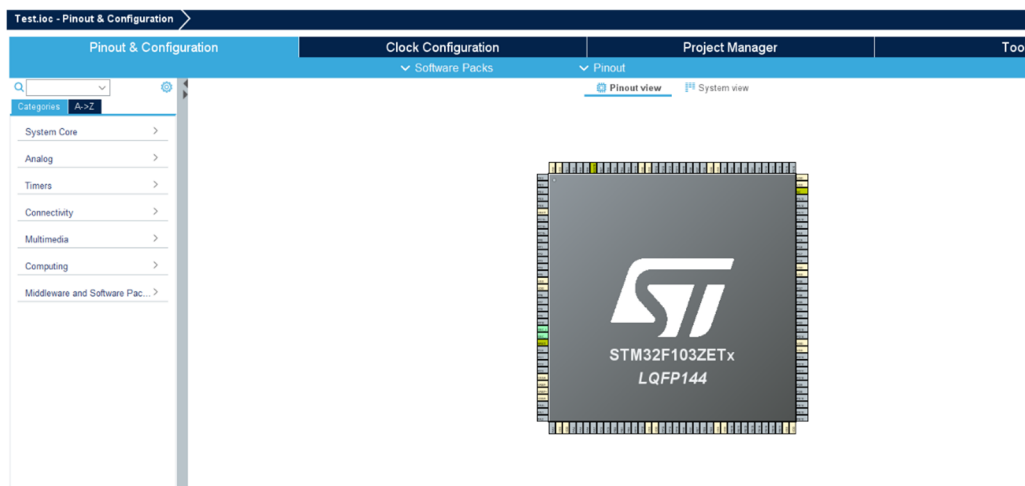
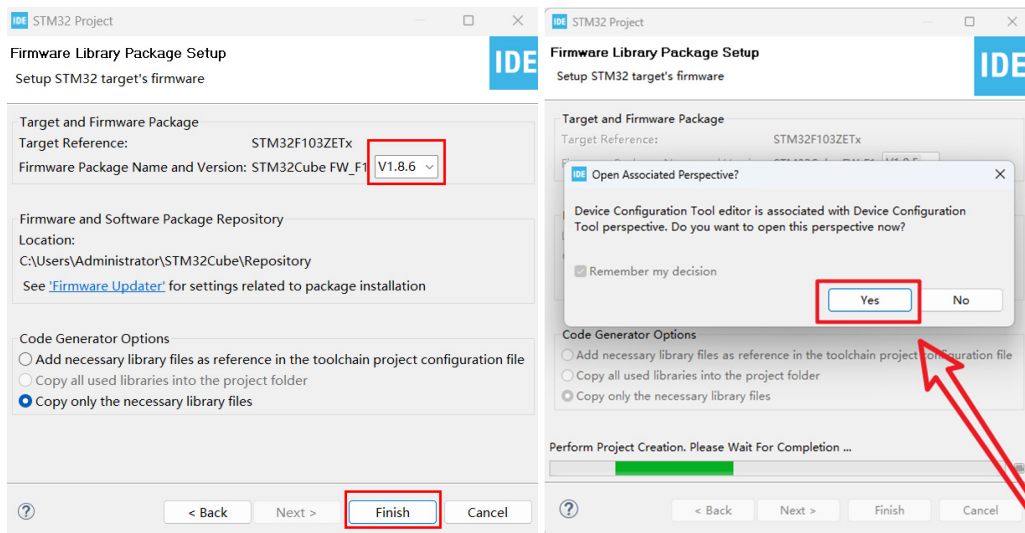


第二步，芯片选型（选择本小车的 STM32 主板型号 STM32F103ZET6）。

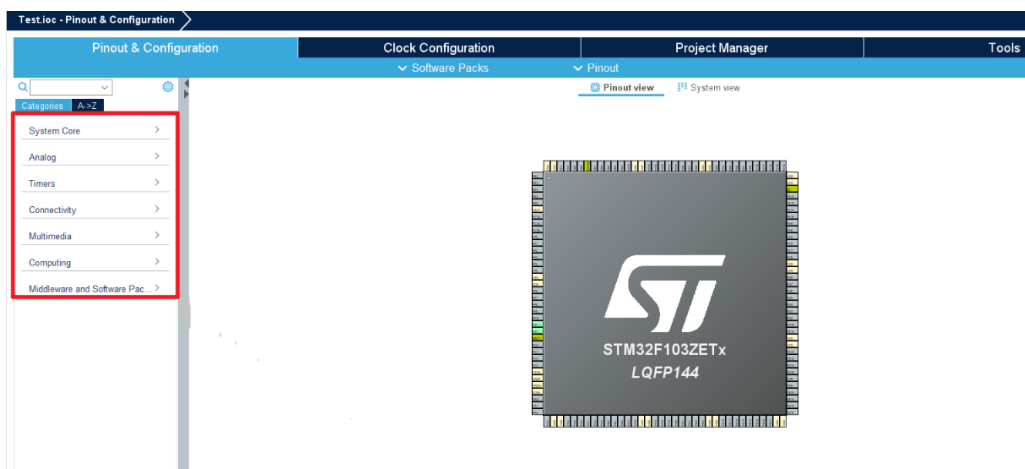


第三步，工程设置。默认配置即可，工程文件名称可以自定义。

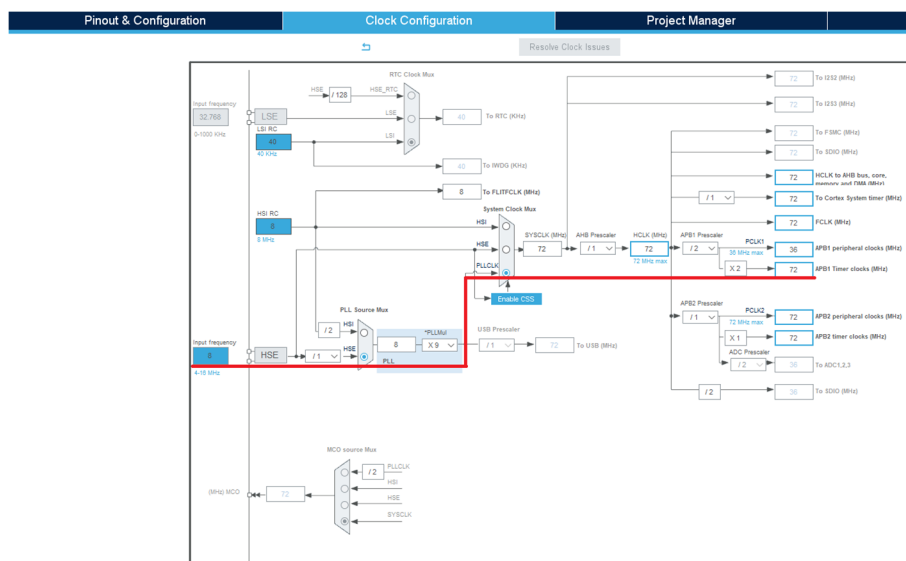




第四步，引脚布局和配置。需配置 SYS、RCC。



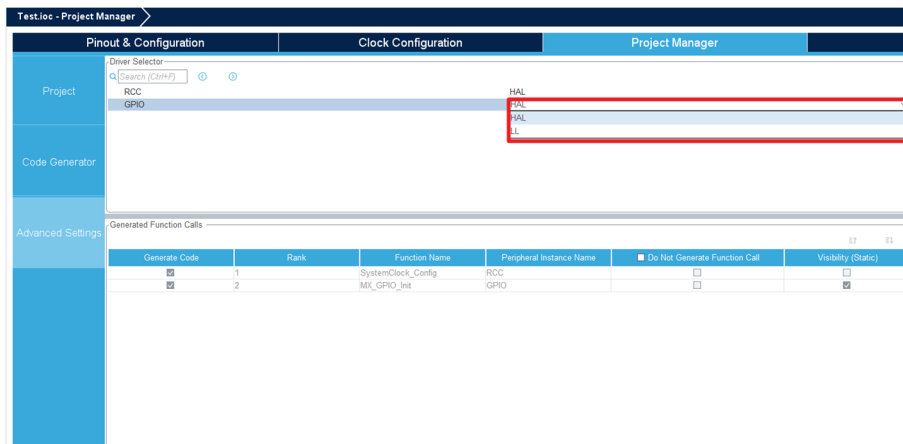
第五步，时钟配置。参考红线涉及的选项。



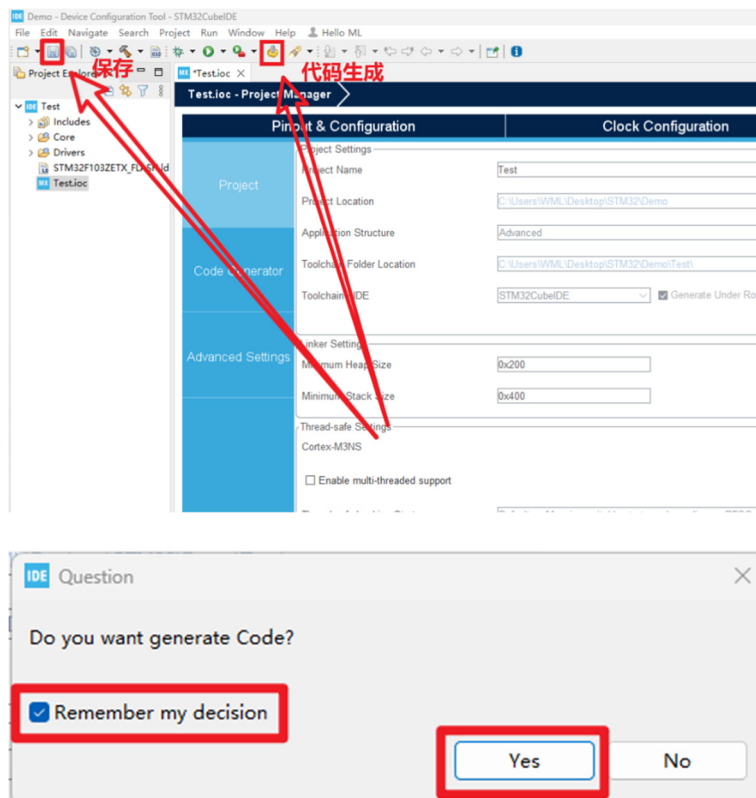
第六步，工程管理。Project: 工程，Code Generator: 代码生成，Advance Settings: 高级设置，选择初始化代码所用的库：此处演示 HAL 库。

Test.io - Project Manager	
Pinout & Configuration	Clock Configuration
Project	Project Settings Project Name: Test Project Location: C:\Users\WML\Desktop\STM32\Demo Browse Application Structure: Advanced ☐ Do not generate the main()
Code Generator	Toolchain Folder Location: C:\Users\WML\Desktop\STM32\Demo\Test\ Toolchain / IDE: STM32CubeIDE ☒ Generate Under Root
Advanced Settings	Linker Settings Minimum Heap Size: 0x200 Minimum Stack Size: 0x400 Thread-safe Settings Cortex-M3NS ☐ Enable multi-threaded support Thread-safe Locking Strategy: Default - Mapping suitable strategy depending on RTOS selection Mcu and Firmware Package Mcu Reference: STM32F103ZETx Firmware Package Name and Version: STM32Cube_FW_F1_V1.8.5 ☒ Use latest available version

Test.io - Project Manager	
Pinout & Configuration	Clock Configuration
Project	STM32Cube MCU packages and embedded software packs ☐ Copy all used libraries into the project folder ☒ Copy only the necessary library files ☐ Add necessary library files as reference in the toolchain project configuration file
Code Generator	Generated files ☒ Generate peripheral initialization as a pair of '.c/.h' files per peripheral ☐ Backup previously generated files when re-generating ☒ Keep User Code when re-generating ☒ Delete previously generated files when not re-generated
Advanced Settings	HAL Settings ☐ Set all free pins as analog (to optimize the power consumption) ☐ Enable Full Assert User Actions Before Code Generation: Browse After Code Generation: Browse Template Settings Select a template to generate customized code: Settings...



第七步，生成初始化代码。 点击代码生成或者保存按钮都可以生成初始化代码。使用 **Ctrl+S** 快捷键进行保存也会生出初始化代码。



第八步，编写 main.c 主代码。 用户代码必须位于 USER CODE BEGIN 和 USER CODE END 中间，否则下次使用 STM32CubeMX 外设配置功能会删除该位置之外的代码。

```

62  * @retval int
63  */
64  int main(void)
65  {
66      /* USER CODE BEGIN 1 */
67
68      /* USER CODE END 1 */
69
70      /* MCU Configuration-----*/
71
72      /* Reset of all peripherals, Initializes the Flash interface and the SysTick. */
73      HAL_Init();
74
75      /* USER CODE BEGIN Init */
76
77      /* USER CODE END Init */
78
79      /* Configure the system clock */
80      SystemClock_Config();
81
82      /* USER CODE BEGIN SysInit */
83
84      /* USER CODE END SysInit */
85
86      /* Initialize all configured peripherals */
87      MX_GPIO_Init();
88      /* USER CODE BEGIN 2 */
89
90      /* USER CODE END 2 */
91
92      /* Infinite loop */
93      /* USER CODE BEGIN WHILE */

```

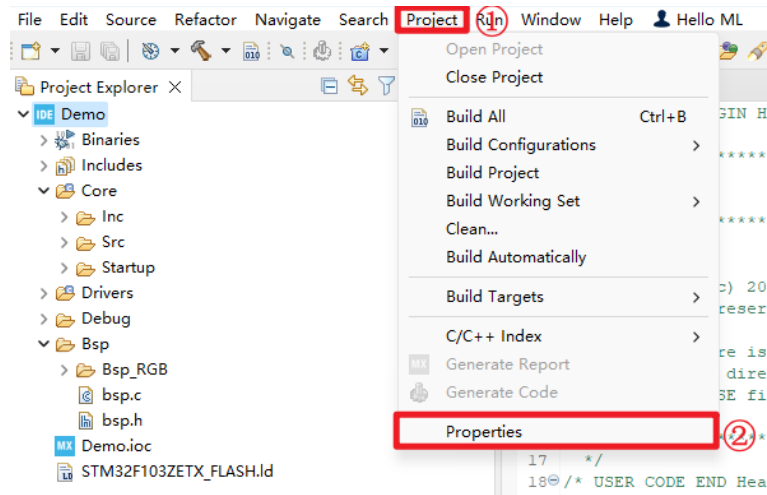
第九步，建立 **bsp** 文件夹，编写 **bsp.c** (功能函数代码)和 **bsp.h** (函数声明代码)。

- 1、按照工程编程逻辑及规范，为代码清晰明了，建议在 **bsp.c** 功能函数代码中撰写要实现的具体功能的代码（即分别撰写可实现不同功能的 **bsp.c** 函数用于实现不同功能，**main.c** 主函数中仅调用功能函数即可）。

桌面 > STM32 > Test1 >				
名称	修改日期	类型	大小	
.settings	2024/8/20 14:50	文件夹		
Bsp	2024/8/20 16:03	文件夹		
Core	2024/8/20 14:50	文件夹		
Debug	2024/8/20 16:16	文件夹		
Drivers	2024/8/20 14:50	文件夹		
.cproject	2024/8/20 16:15	CPROJECT 文件	25 KB	
.mxproject	2024/8/20 15:59	MXPROJECT 文件	7 KB	
.project	2024/8/20 14:50	PROJECT 文件	2 KB	
STM32F103ZETX_FLASH.ld	2024/8/20 15:59	LD 文件	6 KB	
Test1.ioc	2024/8/20 15:59	IOC 文件	5 KB	

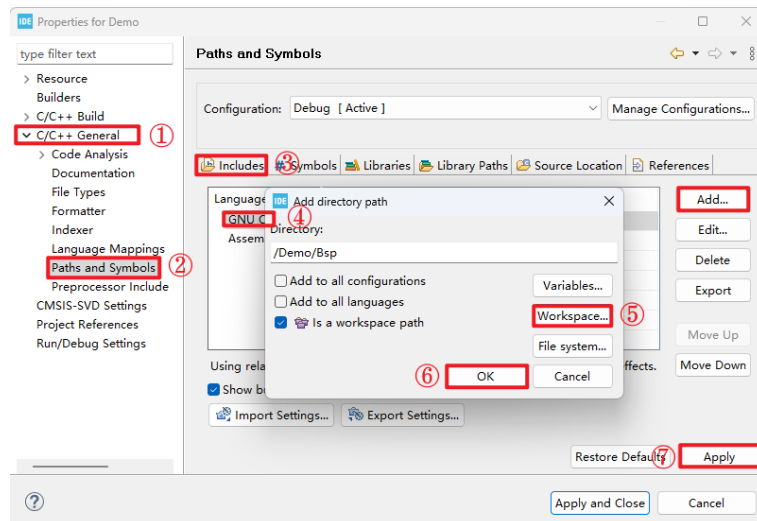
桌面 > STM32 > Test1 > Bsp >				
名称	修改日期	类型	大小	
bsp.c	2023/12/4 14:25	C 文件	1 KB	
bsp.h	2023/12/4 14:25	H 文件	1 KB	

- 2、编程软件中导入 **bsp.c** 和 **bsp.h** 文件步骤如下：



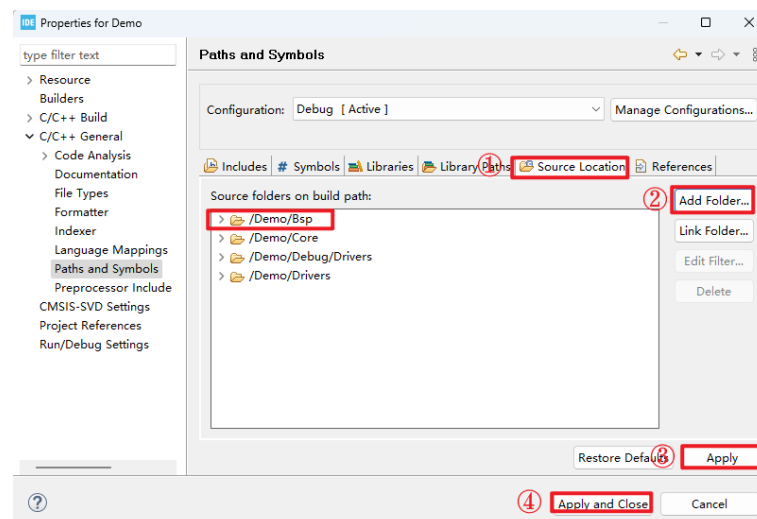
① Includes

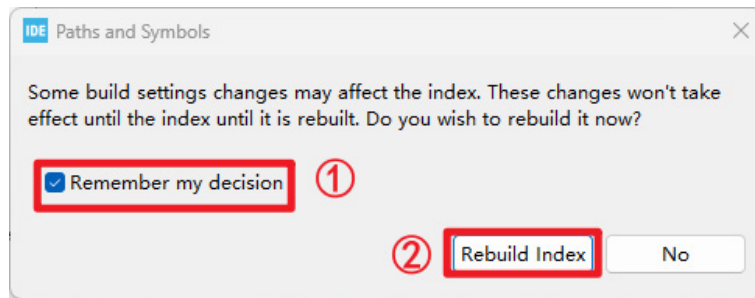
添加.h 文件路径: bsp 文件夹



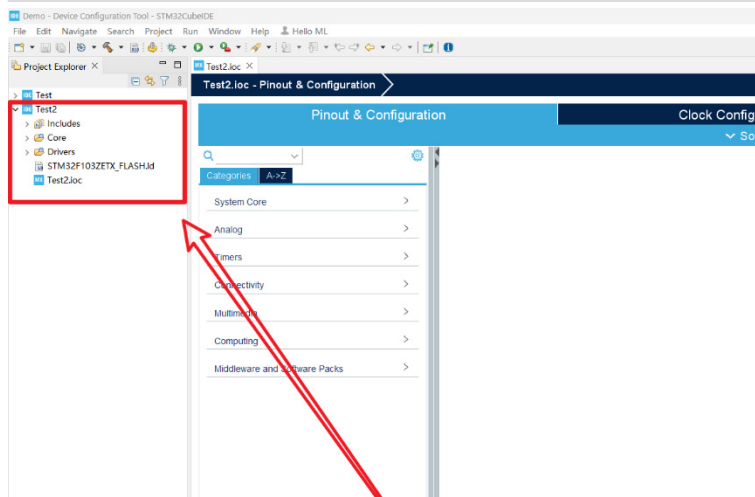
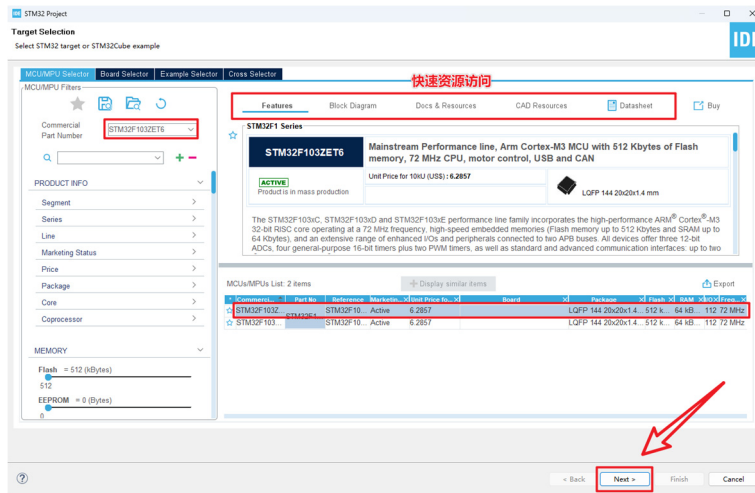
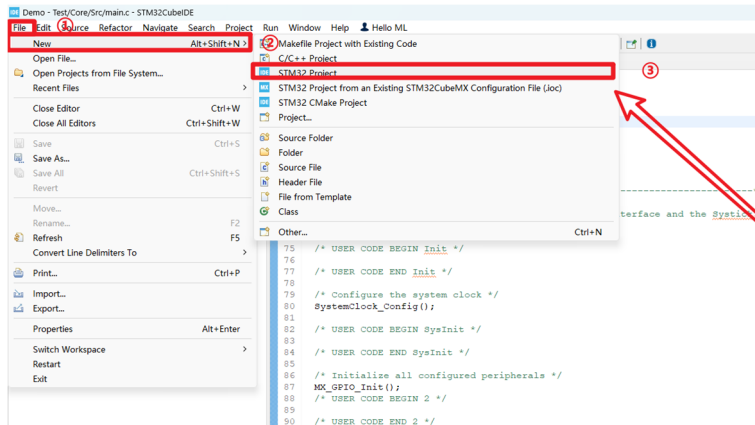
② Source Location

添加.c 文件夹路径: bsp 文件夹路径





第十步，再次新建工程（一般无需使用），新建工程会出现工程文件管理区。



1.2.3 将编译的程序文件写入单片机中

将最小系统安装在扩展板上，并用串行线将电脑和多功能板相连。

在确认连接正确之后即可通过串行线为开发板供电，此时还不能通过专用的程序下载软件下载程序，要进行程序下载。

双击 CH341SER.EXE 安装包进行驱动安装，运行后的界面如图 1.4。点击安装，即可弹出【驱动安装成功】窗口。



图 1.4 驱动安装界面

下一步即可进行程序烧录。双击串口下载软件 FLYMCU 图标，弹出界面如图 1.6。（此时若一直打不开 FLYMCU 软件，则选中软件图标右键“兼容性疑难解答”进行修复即可解决。如下图 1.5 所示。）

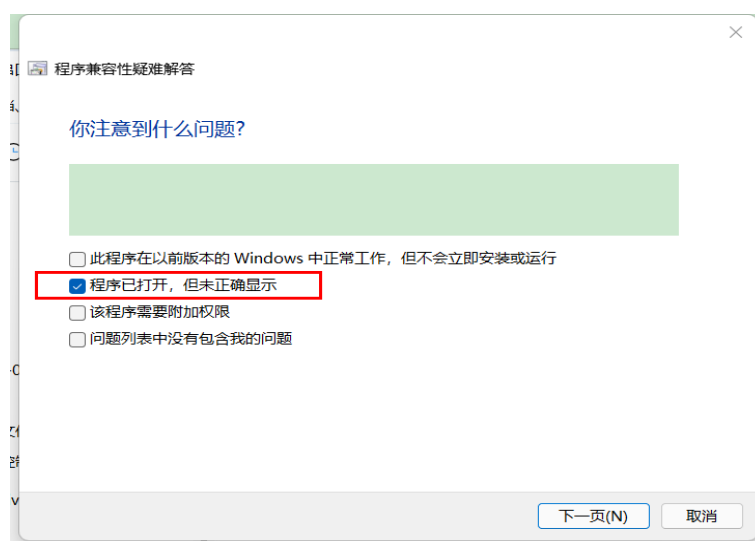


图 1.5 驱动安装界面

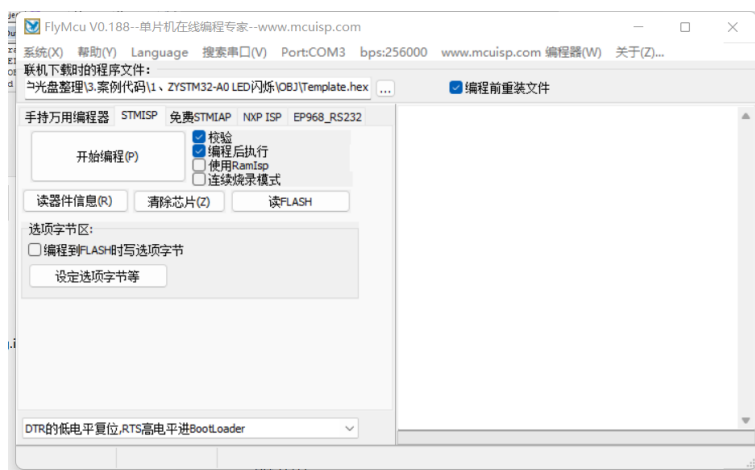


图 1.6 串口下载软件 FLYMCU 界面

具体程序下载步骤如下：

第一步，下载参数配置。主要设置有三个，分别在图 1.7 中用红色圈圈住。

Port 是串口的编号，这项的选择与单片机的串口和电脑串口的连接关系有关，打开设备管理器可以查看端口，如图 1.8；**bps** 为串口波特率，要注意和程序内的一致性，一般常用 9600 或 **115200**；下面一个红圈选项是程序写入时的状态，这里选择 **“DTR 的低电平复位，RTS 高电平进 BootLoader”**。

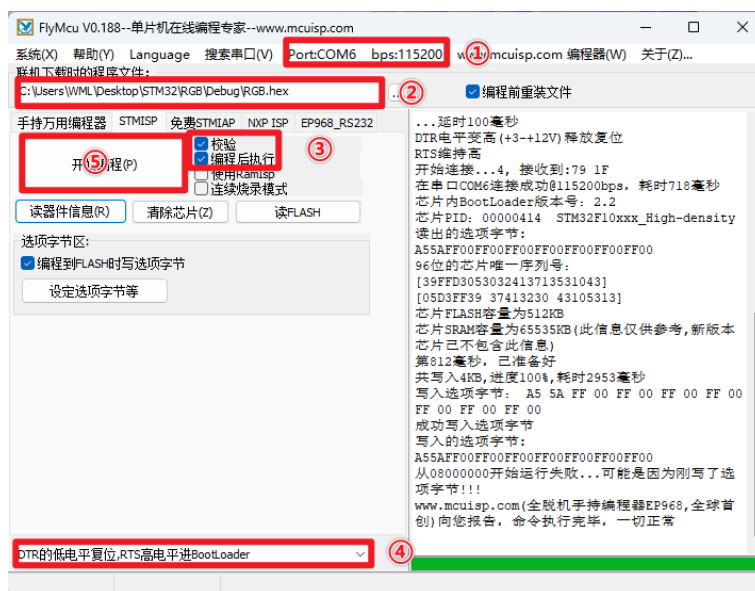


图 1.7 STM32 下载器配置

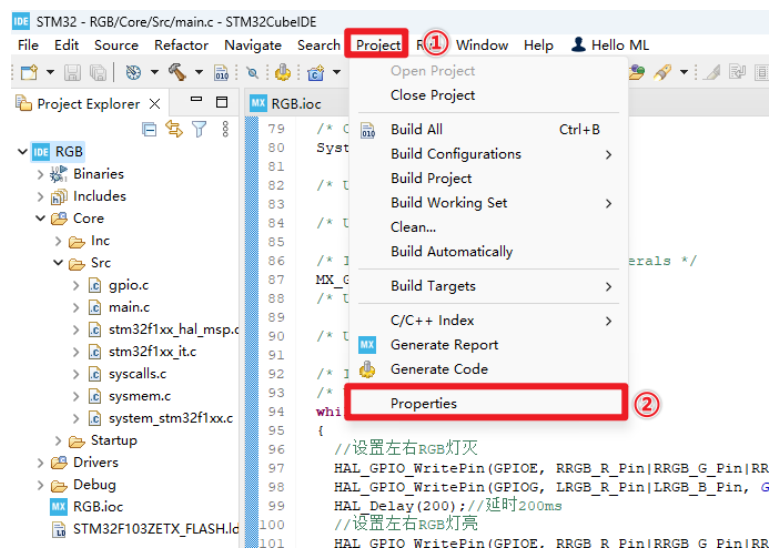


图 1.8 打开设备管理器查看端口

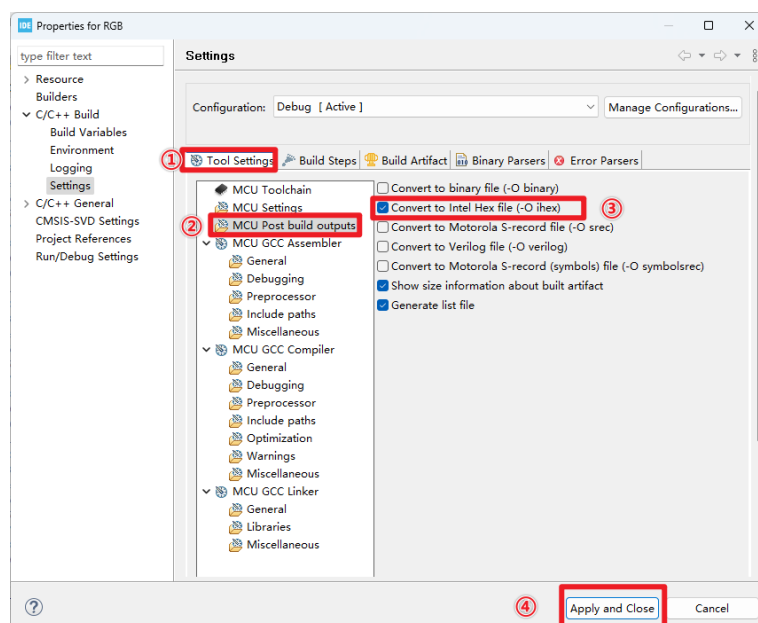
第二步，程序烧写

1、生成.hex 文件

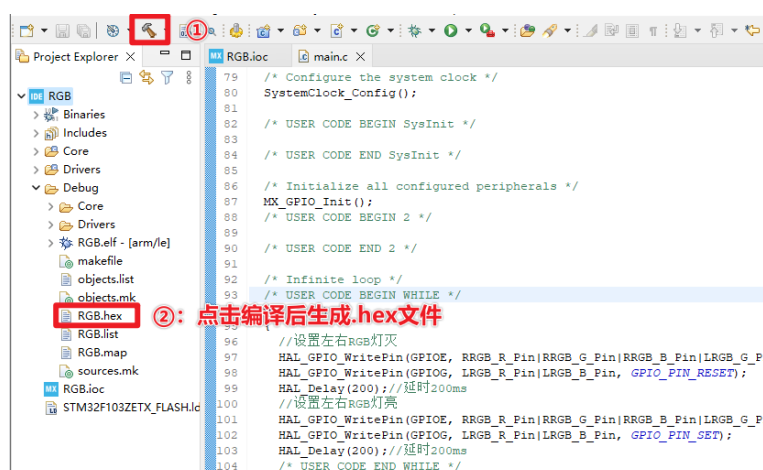
使用 STM32CubeIDE 默认是不会生成.hex 文件，需要勾选相应选项才可以。
(STM32CubeIDE 勾选生成.hex 文件只会对当前操作的项目有效；若自己新建或添加新的项目，则需要重新勾选生成.hex 文件，每个项目首次勾选一次即可)



勾选以下对应的选项



.hex 文件位置：该工程文件的 Debug 文件夹下



在正确设置后，在【...】处选择刚刚编译后的.hex 文件，然后点击开始编程就可以将程序下载到单片机中。

如果无法正确下载，则请按如下情况进行检查：

1. 检查串口线两端是否正常连接到电脑和试验板。
2. 下载软件的设置是否正确（串口波特率，电脑串口号）。
3. 关闭稳压电源，重新插拔单片机接口电源线后重新下载。

一般经过以上情况就可以排除故障，如果故障依旧，请通过助教更换实验仪器。

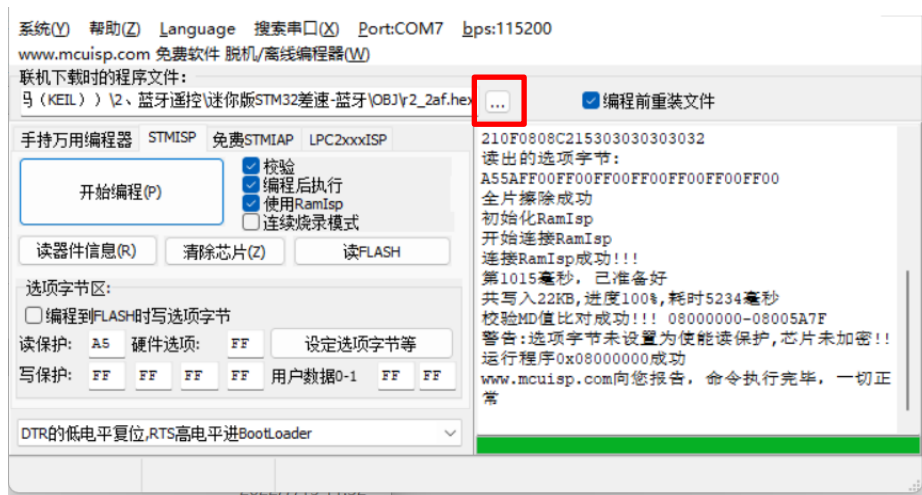


图 1.9 烧写成果界面

2 单片机实验一：RGB 灯闪烁和蜂鸣器报警

2.1 实验目的

- 1) 熟悉基于 STM32CubeIDE 的单片机整体编程环境以及程序开发流程；
- 2) 了解 RGB 灯、蜂鸣器和按键的应用背景
- 3) 理解 RGB 灯、蜂鸣器和按键的工作原理
- 4) 掌握 RGB 灯、蜂鸣器和按键的硬件和软件设计

2.2 实验原理

2.2.1 RGB 灯

LED 发光二极管（light emitting diode）是一种能够将电能转化为光能的固态半导体元器件。和普通二极管一样，发光二极管只能够往一个方向导电。当电流从 p 极（阳极）流向 n 极（阴极）时，电子与电子空穴相遇、复合，电子会跌落到较低的能阶，同时会以光子的形式释放出能量。这就是 LED 灯的发光原理。光的波长（即光的颜色）由组成 p-n 结材料的禁带能量决定。

每个 RGB 灯在一个外壳内包含三个独立的 LED 芯片，每个芯片发出特定的颜色：红色、绿色或蓝色。这些都有其强度级别，并且以各种方式组合时可以创建多样化的调色板。

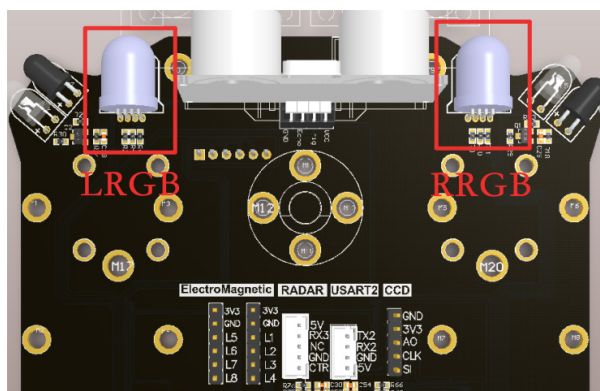


图 2.1 位于小车车头位置的 LED 发光二极管

在本实验中，我们采用的 RGB 灯位于小车车头（图 2.1）。其硬件设计如图 2.2 所示：以 RRGB 为例，其一端口接地，另外三个端口分别与 PE2、PE3 和 PE4 引脚相连。当端口处于高电平时，对应 LED 发光，而当端口处于低电平时，对应 LED 熄灭。理论上，可以通过改变不同端口电平获得不同的颜色。

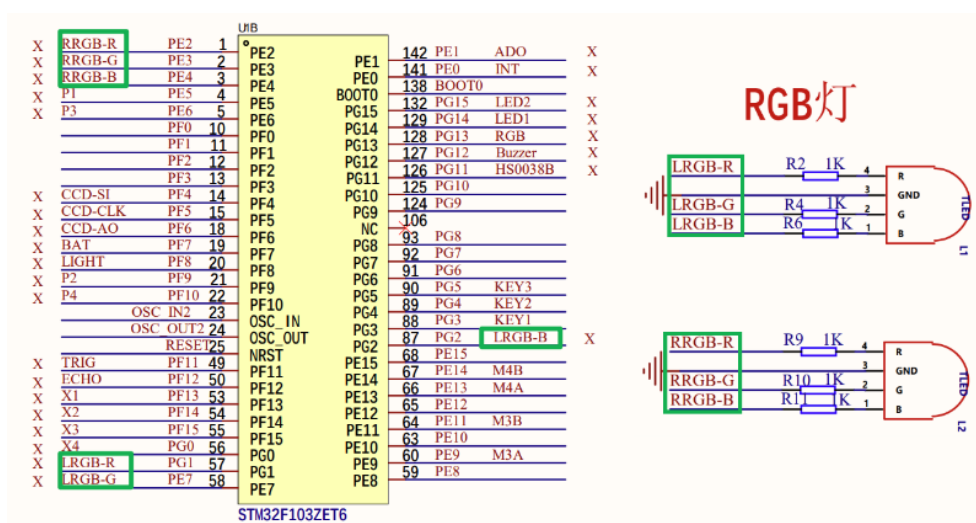


图 2.2 LED 发光二极管硬件设计

2.2.2 蜂鸣器

蜂鸣器是一种能发出声音的元器件，广泛应用于报警器、洗衣机、打印机、电话机等场景中。根据内部是否有振荡源，蜂鸣器分为有源蜂鸣器和无源蜂鸣器。前者内部有振荡源，给高低电平即可控制发声，发声频率固定。后者内部没有振荡源，提供不同的振荡脉冲频率可发出不同频率声音。

在本实验中，我们采用有源蜂鸣器。如图 2.3 所示，它位于小车上方的控制板上。蜂鸣器的硬件设计如图 2.4 所示，其端口与 PG12 引脚相连。当引脚 PG12 处于高电平时，蜂鸣器发声，当引脚 PG12 处于低电平时，蜂鸣器静音。

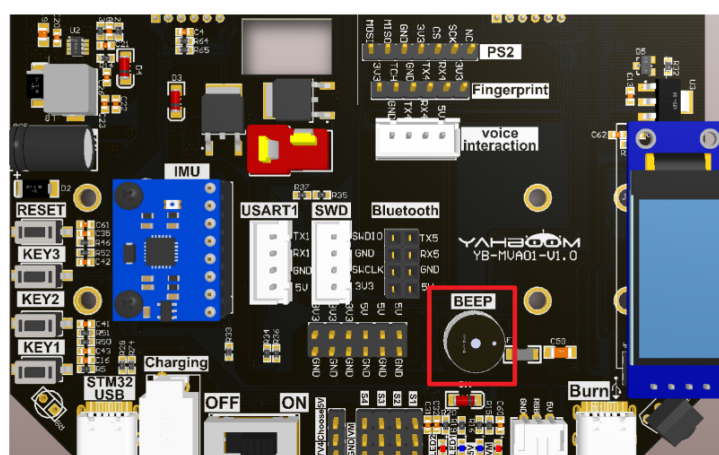


图 2.3 位于小车上方的控制板上的蜂鸣器

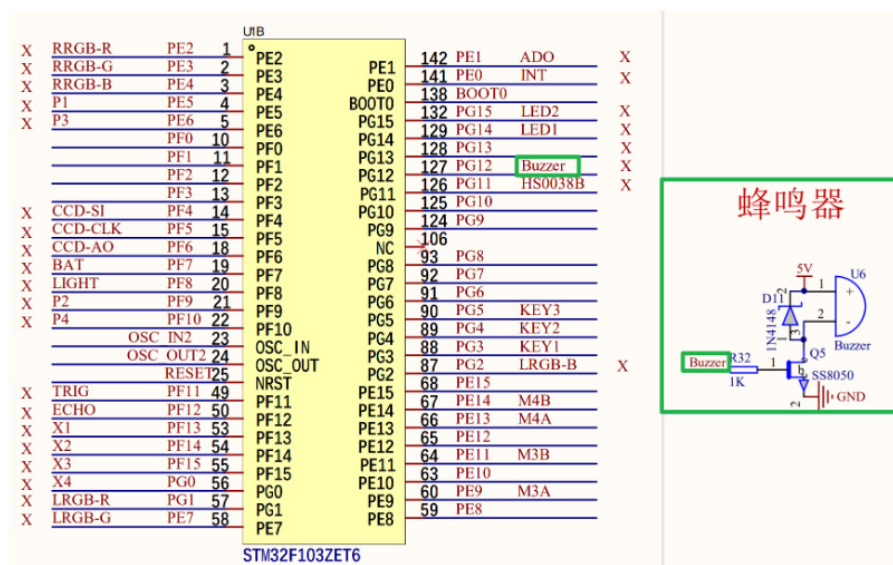


图 2.4 蜂鸣器硬件设计

2.2.3 按键

在单片机中，按键是常见的输入设备，属于开关量元件，通过按下或释放来控制电路的通断。电路断开时，按键的状态值为 0；电路接通时，按键的状态值为 1。一般来说，按键分为轻触式按键和自锁式按键两类。

在本实验中，我们采用轻触式按键来控制蜂鸣器的发声。如图 2.5 所示，按键位于小车上方的控制板上。图 2.6 呈现了按键的硬件设计：一端接在左侧电路中的电阻与电容之间，另一端悬空（按键的一个引脚与数字端口 PGx 连接）。按下按钮时，按键引脚由高电平转为低电平，松开按钮后，按键引脚由低电平又转为高电平。可设置检测到上升沿或下降沿或双边沿即触发中断。中断触发后执行 `void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)` 函数，该函数用于执行具体的中断任务，可用于控制 LED 和蜂鸣器。除了用于触发中断的按键，在小车的控制板上还有一个用于复位的按键。

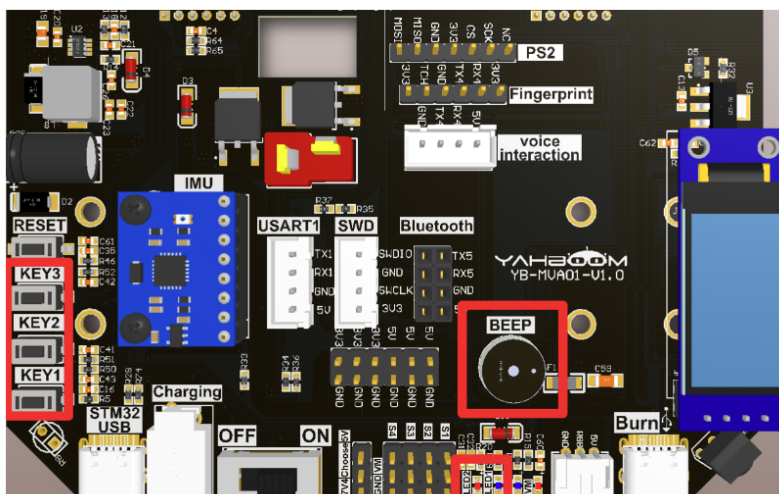


图 2.5 位于小车上方的控制板上的按键

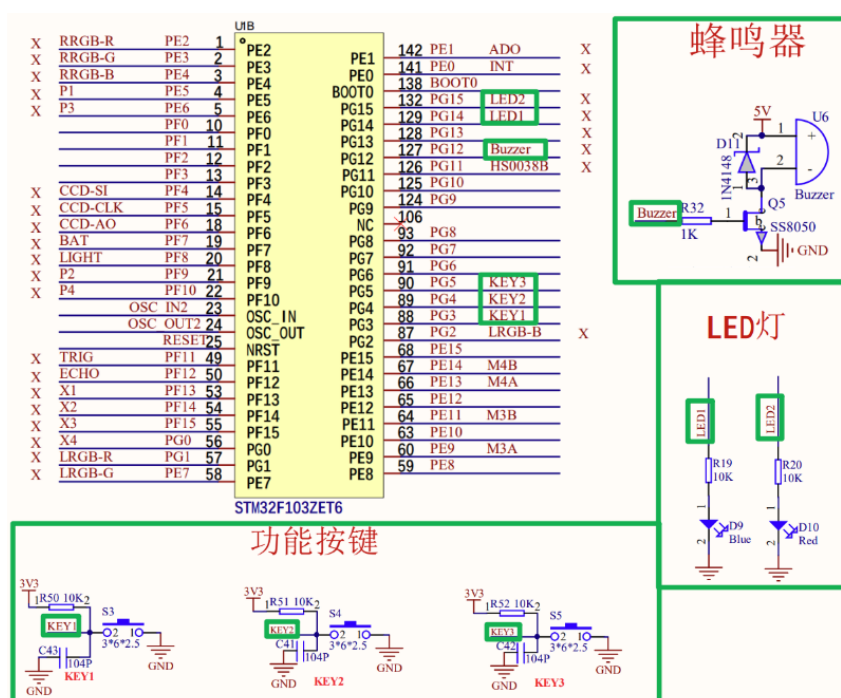


图 2.6 按键硬件设计

2.3 实验内容

2.3.1 RGB 灯闪烁

通过引脚控制 RGB 灯闪烁，每次发光持续 0.2 s，间隔 0.2 s。

部分例程代码如下：

```

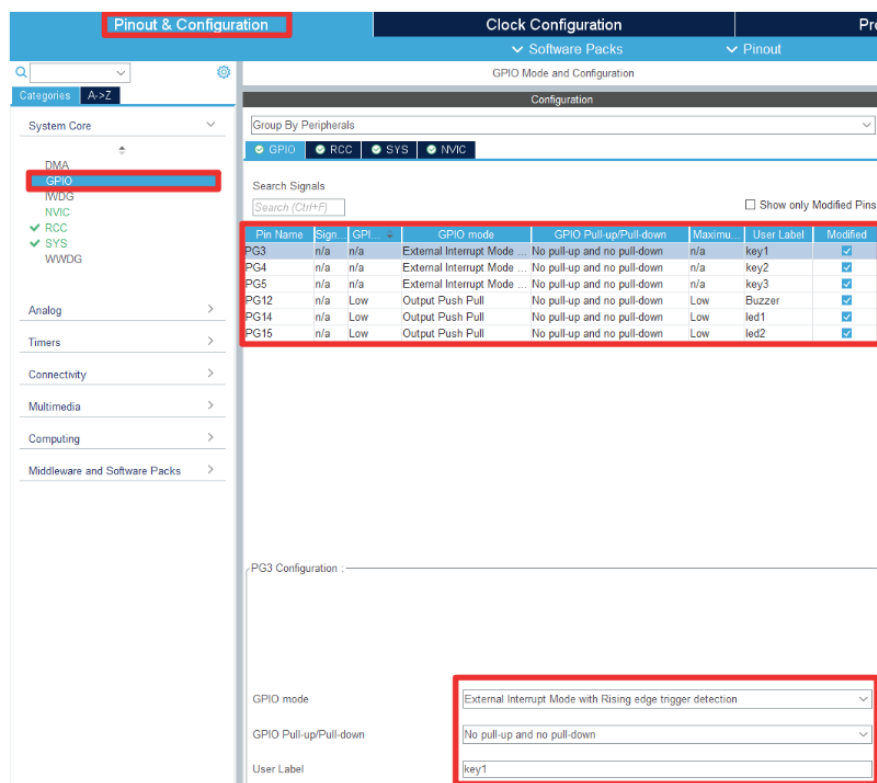
while (1)
{
    //设置左右RGB灯灭
    HAL_GPIO_WritePin(GPIOE, RRGB_R_Pin|RRGB_G_Pin|RRGB_B_Pin|LRGB_G_Pin, GPIO_PIN_RESET);
    HAL_GPIO_WritePin(GPIOG, LRGB_R_Pin|LRGB_B_Pin, GPIO_PIN_RESET);
    HAL_Delay(200); //延时200ms
    //设置左右RGB灯亮
    HAL_GPIO_WritePin(GPIOE, RRGB_R_Pin|RRGB_G_Pin|RRGB_B_Pin|LRGB_G_Pin, GPIO_PIN_SET);
    HAL_GPIO_WritePin(GPIOG, LRGB_R_Pin|LRGB_B_Pin, GPIO_PIN_SET);
    HAL_Delay(200); //延时200ms
    /* USER CODE END WHILE */
    /* USER CODE BEGIN 3 */
}

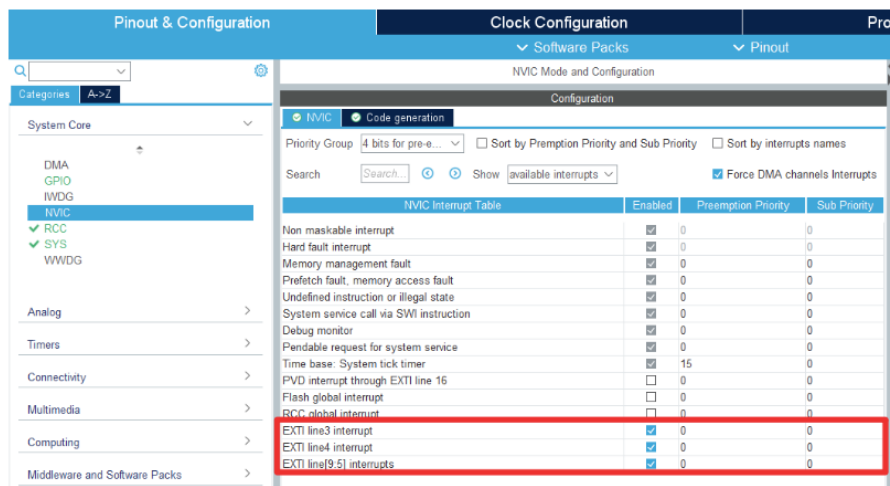
```

2.3.2 控制蜂鸣器发声

通过按键控制蜂鸣器发声，按下 key1 或 key3 键蜂鸣器发声，按下 key2 键蜂鸣器静音。

部分引脚配置步骤如下：





例程代码如下：

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    switch (GPIO_Pin)
    {
        case GPIO_PIN_3:
            HAL_GPIO_WritePin(Buzzer_GPIO_Port, Buzzer_Pin, GPIO_PIN_SET);
            break;
        case GPIO_PIN_4:

            HAL_GPIO_WritePin(Buzzer_GPIO_Port, Buzzer_Pin, GPIO_PIN_RESET);
            break;
        case GPIO_PIN_5:
            HAL_GPIO_WritePin(Buzzer_GPIO_Port, Buzzer_Pin, GPIO_PIN_SET);
            break;
    }
}
```

3 单片机实验二：运动控制（基础）

3.1 实验目的

- 1) 了解常用电机类型和差速转向的基本原理；
- 2) 掌握 STM32 的 GPIO 配置及直流电机的控制方法；
- 3) 掌握 STM32 输出 PWM 波的方法；
- 4) 掌握模块化编程的思想并进行应用。

3.2 实验原理

3.2.1 电机

电机是一种将电能转换为机械能的设备，在工业生产中有着广泛的应用。如图 3.1 所示是一些常用的电机类型。

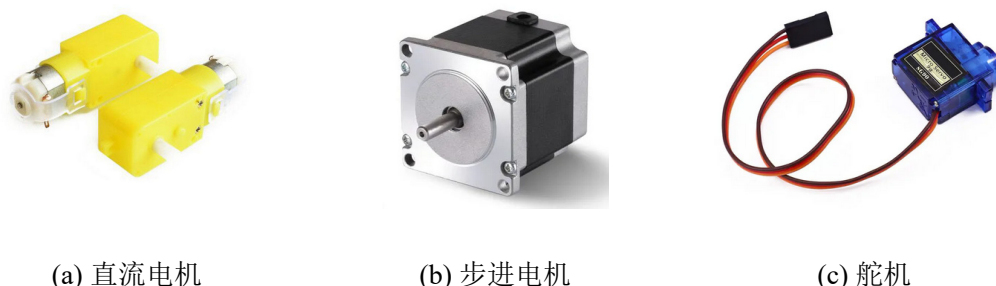


图 3.1 一些常用电机类型

直流电机是将直流电能转换为机械能的电动机，因其良好的调速性能而在电力拖动中得到广泛应用。本课程使用的便是双向直流电机，并带有 AB 相增量式霍尔编码器。

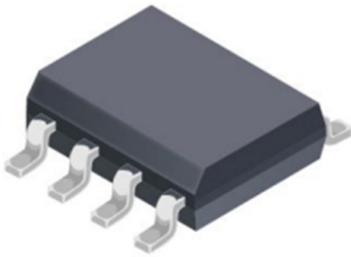
步进电机是一种将电脉冲信号转换成相应角位移或线位移的电动机。每输入一个脉冲信号，转子就转动一个角度或前进一步，其输出的角位移或线位移与输入的脉冲数成正比，转速与脉冲频率成正比，因此步进电动机又称脉冲电动机。步进电机适用于有定位要求的控制系统。

舵机是一种位置（角度）伺服的驱动器。输出轴的位置（角度）随输入的控制信号的占空比而变化，且与控制信号的占空比一一对应。舵机适用于需要角度不断变化并可以保持的控制系统。

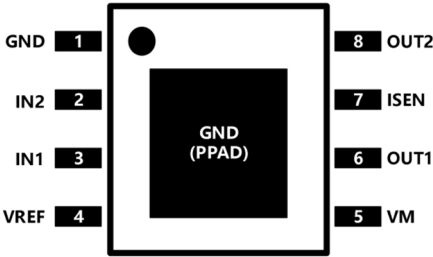
3.2.2 电机驱动模块

直流电机属于大功率器件，单片机的 GPIO 口无法直接驱动，需要配合电机驱动电路来进行控制。

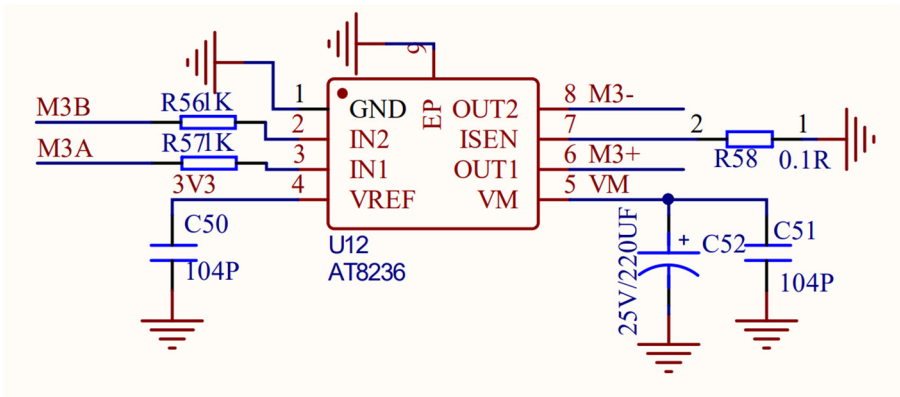
本课程使用的电机驱动模块采用的元器件为 AT8236，是单通道 H 桥电流控制电机驱动器。该芯片设计用于在 5.5 V 至 30 V 的电压下提供高达 6 A 的双向驱动电流，设计用于驱动电感性负载，以及在正电源应用中的其他大电流/高压负载。如图 3.2 所示是 AT8236 芯片的外观图、引脚图和接线图。课程使用的电机为 310 电机，带有 AB 相增量式霍尔编码器。



(a) AT8236 芯片



(b) AT8236 芯片引脚图



(c) AT8236 接线图

图 3.2 电机驱动模块 AT8236 原理图

STM32F103ZET6 集成了 4 个 AT8236 单通道刷式直流电机驱动芯片。输入管脚 IN1、IN2 控制 H 桥的输出状态，下表是输入输出间的逻辑关系：

表 3.1 电机驱动原理

IN1	IN2	OUT1	OUT2	功能
0	0	Z	Z	滑行，休眠
1	0	H	L	正向
0	1	L	H	反向
1	1	L	L	刹车

【非 PWM 调速模式】

将使能引脚直接连接单片机高电平，通过控制 IN1 和 IN2 的电位高低，以控制电机运动状态，便可实现电机的正转与反转，进而实现智能小车的运动。此种情况的控制方法较为简单，但电机只能以固定的速度正转或反转，无法调速。因为智能小车的左右两个电机不可能完全相同，所以智能小车在前进或后退时可能会向一侧偏移。

【PWM 调速模式】

向电机输入 PWM 波，通过调控 PWM 波的占空比以调控 310 电机的转速，进而调控智能小车的运动速度。此种情况下的控制方法稍微复杂，但可以通过调控电机转速，调控小车的运动方向、速度及转弯半径等。PWM 波可以使用 STM32 的定时器和输出比较功能输出。

表 3.2 电机 PWM 驱动原理

IN1	IN2	功能
PWM	0	正转 PWM，快衰减
1	PWM	正转 PWM，慢衰减
0	PWM	反转 PWM，快衰减
PWM	1	反转 PWM，慢衰减

PWM 周期：T = 50us → f = 20KHz:

$$T(s) = \frac{(ARR + 1) * (PSC + 1)}{TIM_CLK(Hz)} = \frac{(3599 + 1) * (0 + 1)}{72000000(Hz)} = 0.00005s = 50us$$

3.3 实验内容

3.3.1 直流电机驱动

【实验目标】

编写控制智能小车前进、后退、左转、右转、左旋转、右旋转、制动的函数，并在主函数中进行调用，实现智能小车前进、后退、左转、右转、左旋转、右旋转、制动的运动方式。

【实验思路】

- 1) 在 IOC 工程文件中配置 RCC、SYS、GPIO 模块，配置系统时钟；
- 2) 通过控制 IO 口输出电位的高低，分别控制四个电机正转、反转及制动，进行调试，可以考虑写成函数，以便之后调用；

- 3) 分别编写智能小车前进、后退、左转、右转、左旋转、右旋转、制动的函数，使用 HAL_Delay(int16_t time)函数控制小车前进、后退、左转、右转、左旋转、右旋转、制动的时长；
- 4) 在主函数调用初始化函数及智能小车前进、后退、左转、右转、左旋转、右旋转、制动的函数，实现智能小车运动控制（当前暂不需要控制小车的运动速度）。

【结果要求】

本实验预期需要得到的结果可以参考表 3.3。

表 3.3 直流电机驱动实验结果输出

文件名	函数名	备注
motor.c motor.h	void motor1_forward(void)	
	void motor1_backward(void)	
	void motor2_forward(void)	
	void motor2_backward(void)	
	void motor3_forward(void)	
	void motor3_backward(void)	
	void motor4_forward(void)	
	void motor5_backward(void)	
	void motor_stop(void)	

【例程代码】

部分例程代码如下：

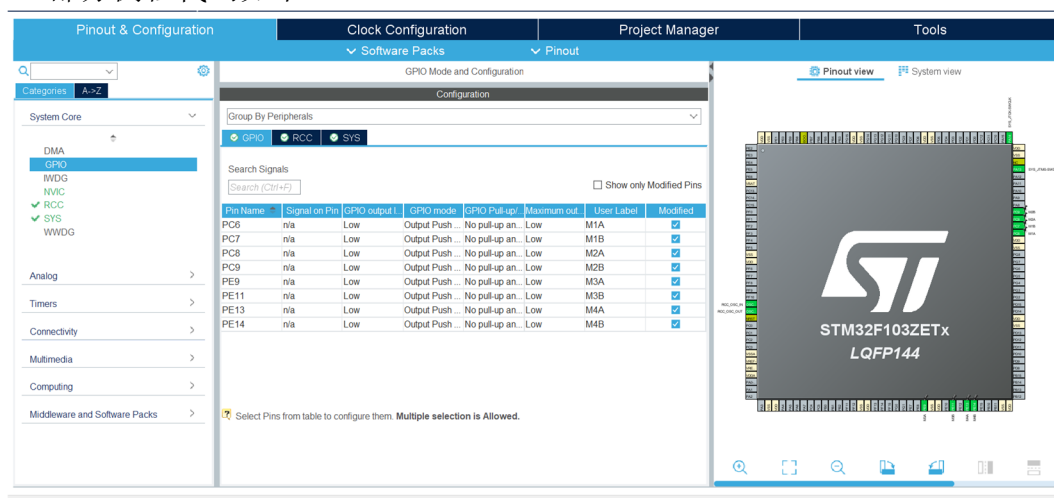


图 3.3 ioc 工程文件配置

```

void motor_stop(void)
{
    HAL_GPIO_WritePin(GPIOC,GPIO_PIN_6|GPIO_PIN_7|GPIO_PIN_8|GPIO_PIN_9,GPIO_PIN_RESET);
    HAL_GPIO_WritePin(GPIOE,GPIO_PIN_9|GPIO_PIN_11|GPIO_PIN_13|GPIO_PIN_14,GPIO_PIN_RESET);
}

void motor1_forward(void)
{
    HAL_GPIO_WritePin(GPIOC,GPIO_PIN_6,GPIO_PIN_SET);
    HAL_GPIO_WritePin(GPIOC,GPIO_PIN_7,GPIO_PIN_RESET);
}
void motor1_backward(void)
{
    HAL_GPIO_WritePin(GPIOC,GPIO_PIN_7,GPIO_PIN_SET);
    HAL_GPIO_WritePin(GPIOC,GPIO_PIN_6,GPIO_PIN_RESET);
}

```

3.3.2 PWM 波调速

【实验目标】

在直流电机驱动实验的基础上，通过 PWM 波调控电机的转动速度，使得智能小车可以沿直线前进和后退，通过函数调用控制智能小车的运动速度。

【实验思路】

在直流电机驱动实验的基础上进行修改（另存文件再修改）：

- 1) 在 IOC 工程界面配置 RCC 时钟使能（TIM1 和 TIM8 外设的时钟），将四个通道均设置为 PWM 输出模式；
- 2) 配置时基单元（选择时钟源，选择计数模式、配置 PSC 预分频器、配置 ARR 自动重装器）；
- 3) 配置输出比较单元（配置 CCR 捕获/比较器、选择输出比较模式、选择极性，配置输出使能）；
- 4) 运行控制，启动计时器；
- 5) 通过调控 PWM 波的占空比（注意占空比较小时电机可能无法转动），分别控制四个电机以指定速度正转、反转及制动，进行调试，可以考虑写成函数，以便之后调用；
- 6) 新建.c 文件和.h 文件并分别编写智能小车前进、后退、左转、右转、左旋转、右旋转、制动的函数，实现智能小车以指定的速度运动指定的时间；
- 7) 在主函数调用初始化函数及智能小车前进、后退、左转、右转、左旋转、右旋转、制动的函数，实现智能小车运动控制（需要控制小车的运动速度）。

【结果要求】

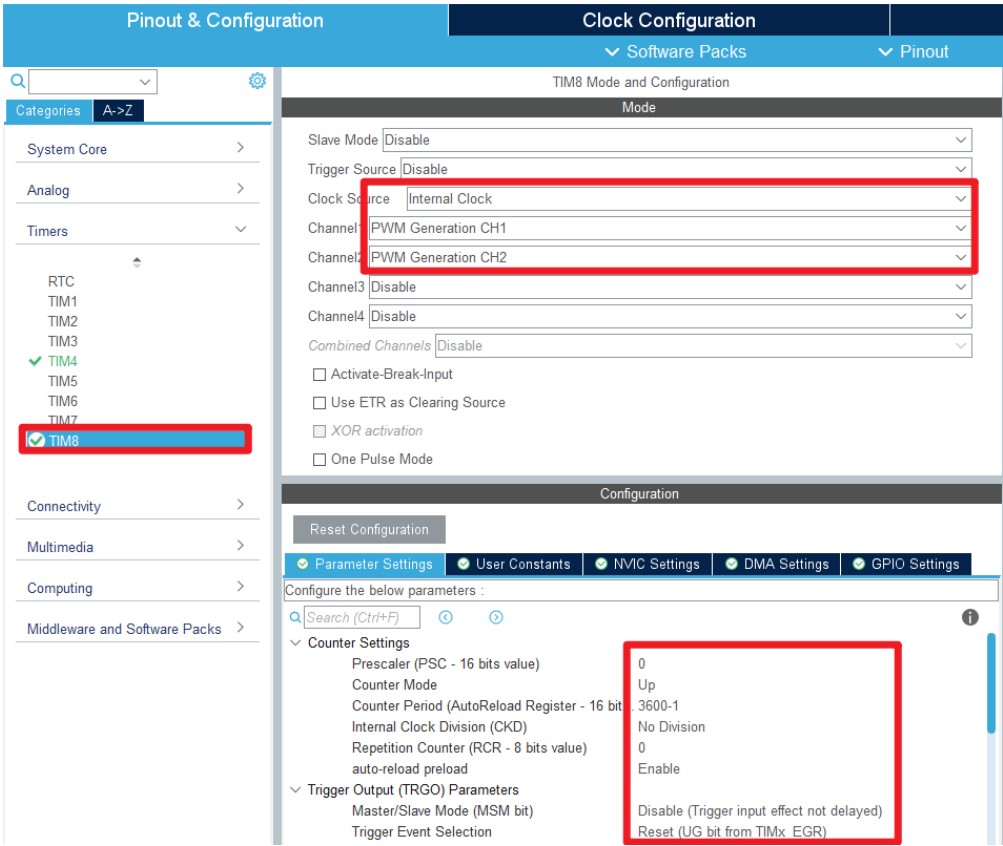
本实验预期需要得到的结果可以参考表 3.4。

表 3.4 PWM 波调速实验结果输出

文件名	函数名	备注
motorPWM.c motorPWM.h	void pwm_motor_stop(void)	
	void pwm_motor1_forward(int16_t speed)	
	void pwm_motor1_backward(int16_t speed)	
	void pwm_motor2_forward(int16_t speed)	
	void pwm_motor2_backward(int16_t speed)	
	void pwm_motor3_forward(int16_t speed)	
	void pwm_motor3_backward(int16_t speed)	
	void pwm_motor4_forward(int16_t speed)	
	void pwm_motor4_backward(int16_t speed)	

【例程代码】

TIM1 和 TIM8 配置相同：这里演示 TIM8 CH1 和 CH2 的配置：



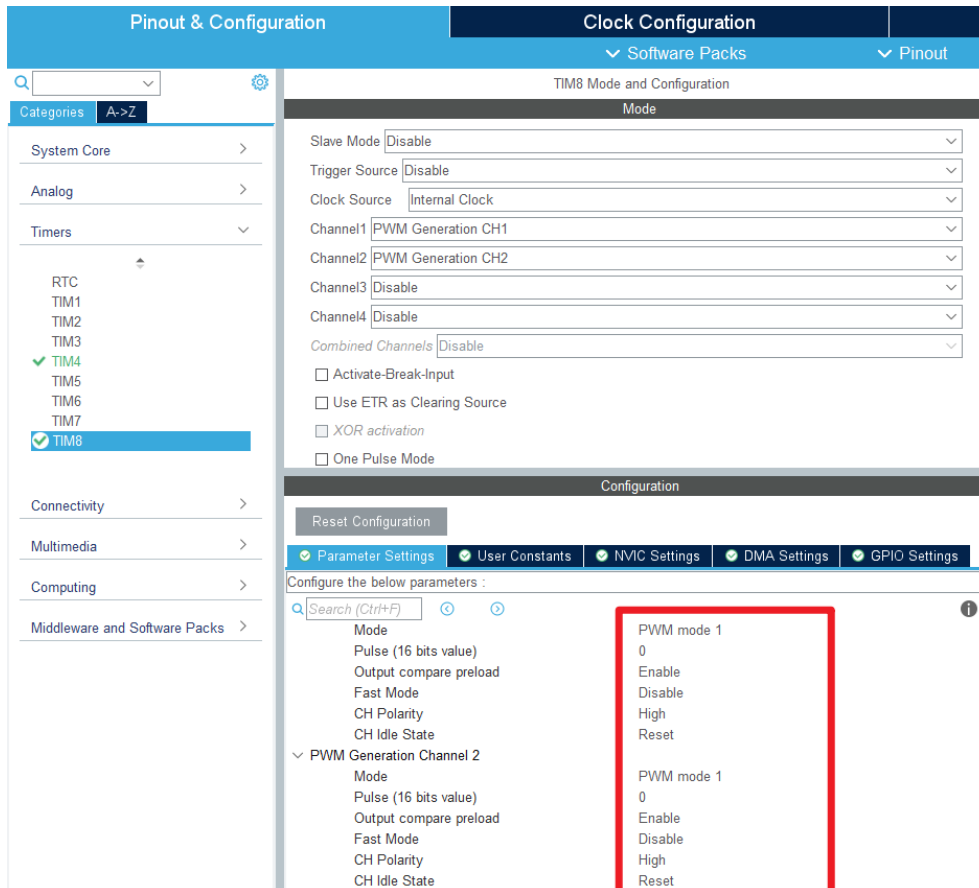


图 3.4 TIM8 端口配置

部分例程代码如下：

```
// 启动tim1的pwm输出
HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_1);
HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_2);
HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_3);
HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_4);
// 启动tim8的pwm输出
HAL_TIM_PWM_Start(&htim8, TIM_CHANNEL_1);
HAL_TIM_PWM_Start(&htim8, TIM_CHANNEL_2);
HAL_TIM_PWM_Start(&htim8, TIM_CHANNEL_3);
HAL_TIM_PWM_Start(&htim8, TIM_CHANNEL_4);
void pwm_motor_stop(void) //四个电机停止
{
    TIM8->CCR1 = 0;
    TIM8->CCR2 = 0;
    TIM8->CCR3 = 0;
    TIM8->CCR4 = 0;
    TIM1->CCR1 = 0;
    TIM1->CCR2 = 0;
    TIM1->CCR3 = 0;
    TIM1->CCR4 = 0;
}
void pwm_motor1_forward(int16_t speed)//电机1前进
{
    TIM8->CCR1 = speed;
    TIM8->CCR2 = 0;
}
void pwm_motor1_backward(int16_t speed)//电机1后退
{
    TIM8->CCR1 = 0;
    TIM8->CCR2 = speed;
}
```

4 单片机实验三：运动控制（提高）

4.1 实验目的

- 1) 巩固并熟练掌握智能小车前进后退左转右转的相关操作；
- 2) 在上述基础上，熟练掌握小车运动状态的切换，能够控制小车实现特定轨迹运动及变速运动的功能。

4.2 实验原理

本实验作为实验二智能小车运动控制（基础）的延伸，实验的基本原理与实验二相同。在本实验中，灵活应用延时功能并不断调节小车所处的运动状态，即可控制智能小车按照既定轨迹与所需求的速度进行运动。本实验与后续红外循迹不同，红外循迹是闭环控制，小车在行进过程中通过不断自我修正行走路线，从而实现循迹；本实验属于开环控制，根据预设路径给小车赋予一定的程序，使小车按照既定轨迹前进。

4.2.1 转弯调速原理

弯道是复杂运动轨迹中比较重要的一环，面对不同的运动轨迹，小车转弯的曲率半径可能不同。调解曲率半径的方法是调整小车两侧车轮的差速比，具体通过控制两侧驱动轮电机的占空比实现。图 4.1 所示为差速小车的运动模型。

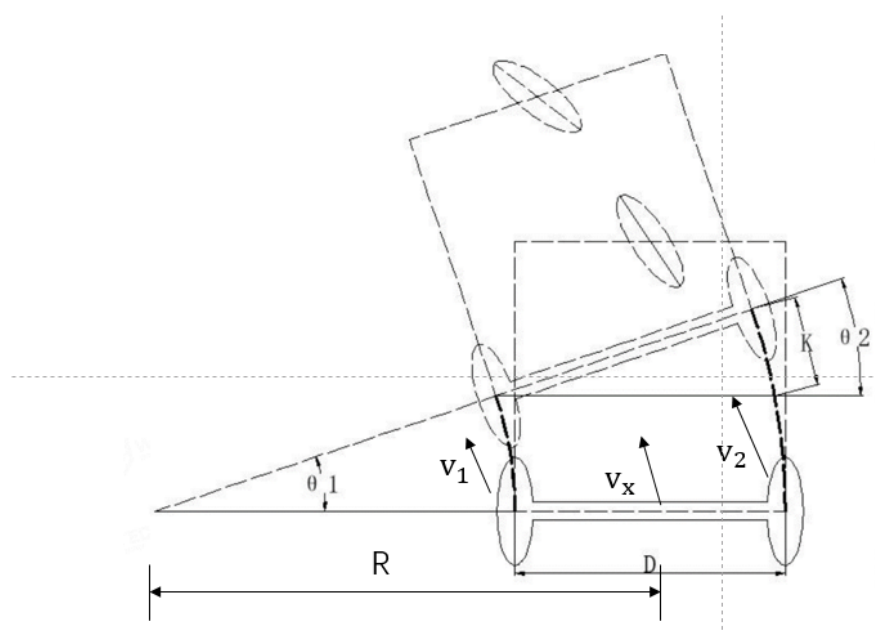


图 4.1 差速小车运动模型

其中，车轮中点位移：

$$R \cdot \theta_1 = \frac{v_1 + v_2}{2} \cdot \Delta t \quad (4-1)$$

公式（4-1）中 R 为小车转弯的曲率半径。两侧车轮位移之差：

$$K = (v_2 - v_1) \cdot \Delta t = D \cdot \theta_2 \quad (4-2)$$

近似有：

$$\theta_1 = \theta_2 \quad (4-3)$$

综合以上三式可得曲率半径与速度的关系：

$$R = \frac{(v_1 + v_2) \cdot D}{2(v_2 - v_1)} \quad (4-4)$$

依据该关系式，我们可以控制小车按合适的曲率进行转弯。

4.2.2 变速运动原理

小车车轮转速由占空比决定。在多个循环步中逐渐提高小车电机的占空比，小车就能实现近似匀加速运动；在多个循环步中逐渐降低小车电机的占空比，小车就能实现近似的匀减速运动。

4.3 实验内容

4.3.1 轨迹调控

在实际工程应用中，智能小车需要按照规定的轨迹运动，才能实现货物的定点运输等功能。本实验要求大家对实验二中小车的前进、后退、左转、右转等功能进行组合，使小车完成一个特定的轨迹运动流程。

可供选择的运动轨迹：“8”字型、三叶草形、心形等（可自行设计想要实现的轨迹）。

实验流程：确定拟实现的运动轨迹并在纸上绘出，通过编程不断修正小车运行曲线，直至达到预期的运动轨迹。

4.3.2 变速运动

实现小车从静止不动开始，进入到加速运动阶段，到达最大速度后，保持匀速行驶一段时间，随后经过减速阶段回到静止状态。

5 单片机实验四：黑线循迹

5.1 实验目的

- 1) 熟练掌握 STM32CubeIDE 整体编程环境以及程序开发流程；
- 2) 了解四路循迹模块的工作原理以及编程结构；
- 3) 熟练掌握循迹模块的具体程序及编译调试；
- 4) 结合电机运动控制实现黑线循迹的功能。

5.2 实验原理

通过传感器实时采集轨迹数据，进而判断小车主体与轨迹线的相对位置关系。当检测到小车偏离轨迹线时，控制器依据偏离程度选择相应的校正策略，从而下达转向或速度调整指令至驱动电机，确保轨迹追踪的准确性。

在众多循迹类型中，黑白反差式追踪是较为常见和简单的循迹类型。该技术主要依赖在白色背景上铺设的黑色引导线来定义小车的行驶路径。此背景下，小车在标有黑色线条的平面上行进，利用黑色和白色表面反射率差异显著的特性，通过接收端的光强度变化来识别和跟踪路径中的黑色标记。以此为原理的红外探测法是目前应用较为普遍、原理简单的检测方法。

红外探测法的工作机制是利用红外线在不同颜色表面的物理表面具有不同的反射性质的特点，在小车行驶过程中持续向地面发射红外光，当红外光照射浅色地面时发生漫反射，部分反射光被装在小车上的接收管接收；而照射到黑色迹线时则红外光被吸收，导致小车上的接收管接收不到反射信号。

本次实验以小车现有的硬件系统为基础，利用红外对管传感器实现智能小车循迹功能的简单模拟，以此为例讲解循迹的基本原理和思路。

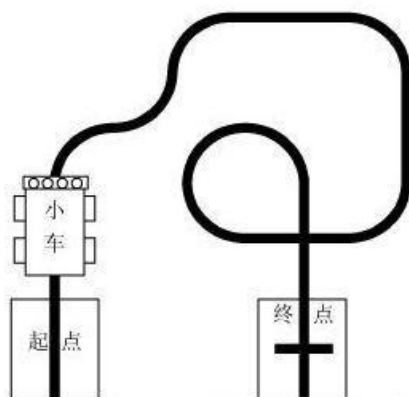


图 5.1 循迹示例

5.2.1 红外对管传感器原理

示例实验采用的是红外对管传感器，该模块对环境光线适应能力强，其由发射管接收管和配套电路组成，一般浅色的是发射管，黑色的是接收管。工作时发射管发射一定频率红外线，当检测方向遇到反射面时，红外线的反射光被接收管接收。经过比较器电路处理之后，信号输出接口输出数字信号：当检测到黑线时，循迹模块指示灯亮、输出端为低电平，否则灯灭、并输出高电平。

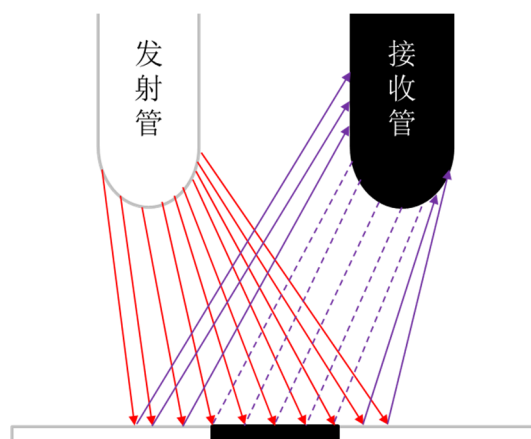


图 5.2 红外对管循迹原理图

5.2.2 黑线循迹原理

本实验中采用位于小车底板前端下面的红外循迹电路实现四路红外对管传感器的信号采集与状态判断，实物部分如图 5.3 所示，其中浅蓝色的为发射管、黑色的为接收管。其硬件原理如图 5.4 所示以及实物连接图如图 5.5 所示，其展示了四路循迹模块与单片机使用 XH2.54 的 6Pin 连接线的引脚连接对应关系。

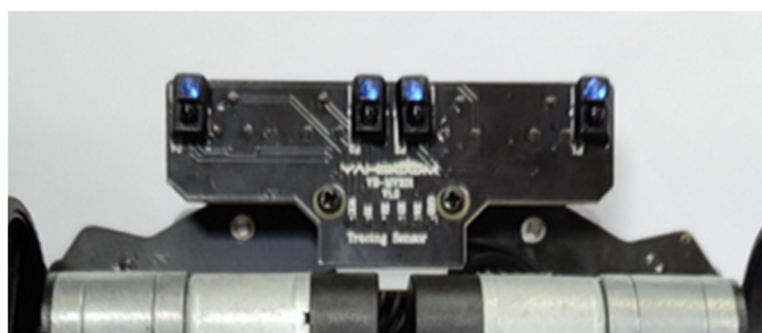


图 5.3 红外对管四路循迹模块实物图

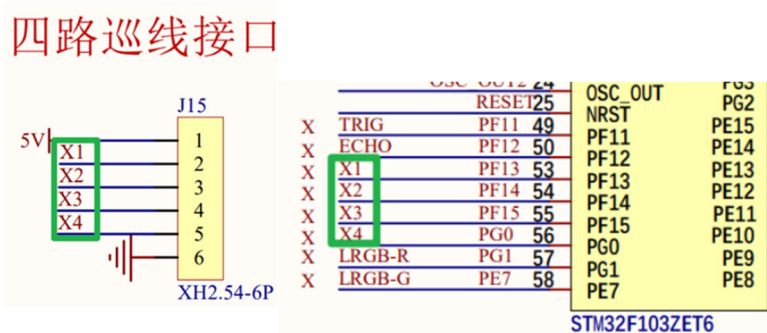


图 5.4 硬件原理图

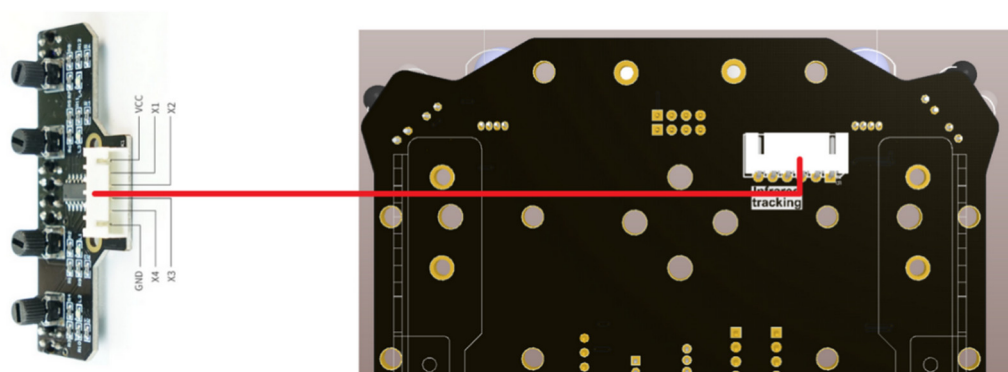


图 5.5 实物连接图

5.2.3 黑线循迹动作逻辑原理

小车循迹时，STM32 单片机根据传感器是否收到反射回的红外光输出的高低电平来确定黑线的位置和小车的行走路线。本实验小车循迹时，使用了四个红外对管模块，对于一些简单情况的处理在图 5.6 中进行了体现。

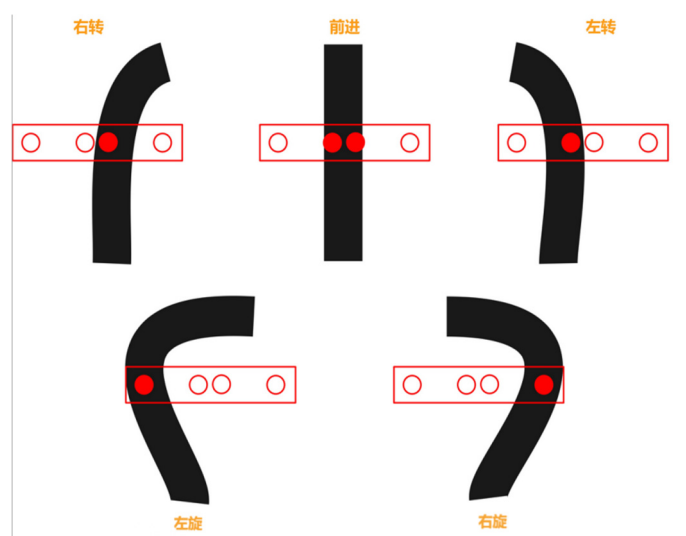


图 5.6 四路循迹模块常见的检测情况

当对管输出端为低电平则表示对管在黑线上，输出端为高电平则表示对管在黑线外。当只有中间两个探测器探测到黑线时，可以认为小车并没有偏离轨迹，而使小车保持前进状态即可；当只有右侧的探测器探测到黑线时，表明小车相对轨迹偏左时，此时程序实现小车快速反应右转，并参考图 5.6 中所示的情况调整小车右转的幅度或速度，已匹配最佳的反馈方式；同理当小车左侧的探测器探测到黑线时，程序实现左侧转动反馈，最终实现小车在白色地板上循黑线行走。

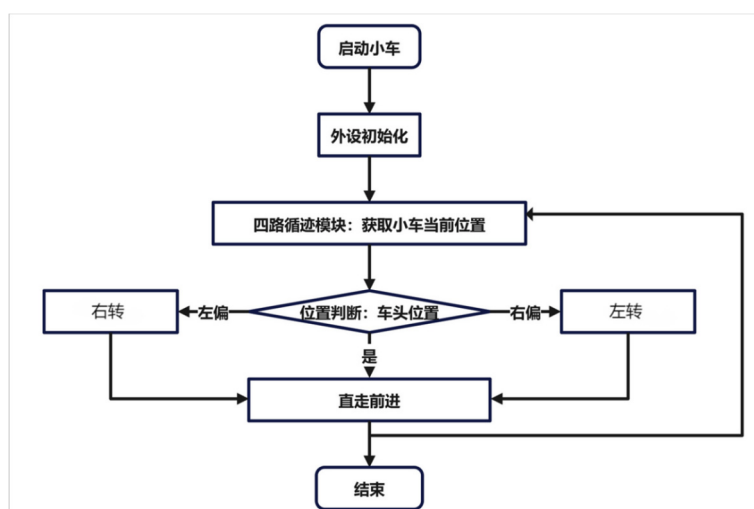


图 5.7 参考的循迹动作程序流程图

实验过程中为防止小车在弯曲黑线上循迹时冲出黑线，需注意测试、调试小车速度实现速度与红外循迹模块频率匹配，以期达到最优的运行速度及轨迹。

5.3 实验内容

实现小车沿着黑色轨迹线从起点自主行驶至终点。注意小车速度及自动行驶的流畅性。

其中本节核心的工程配置部分如下图所示，省略的工程配置部分：新建工程、芯片选择、工程配置、引脚配置的 SYS、RCC 配置、时钟配置以及工程配置内容可见实验指导书之前的内容。

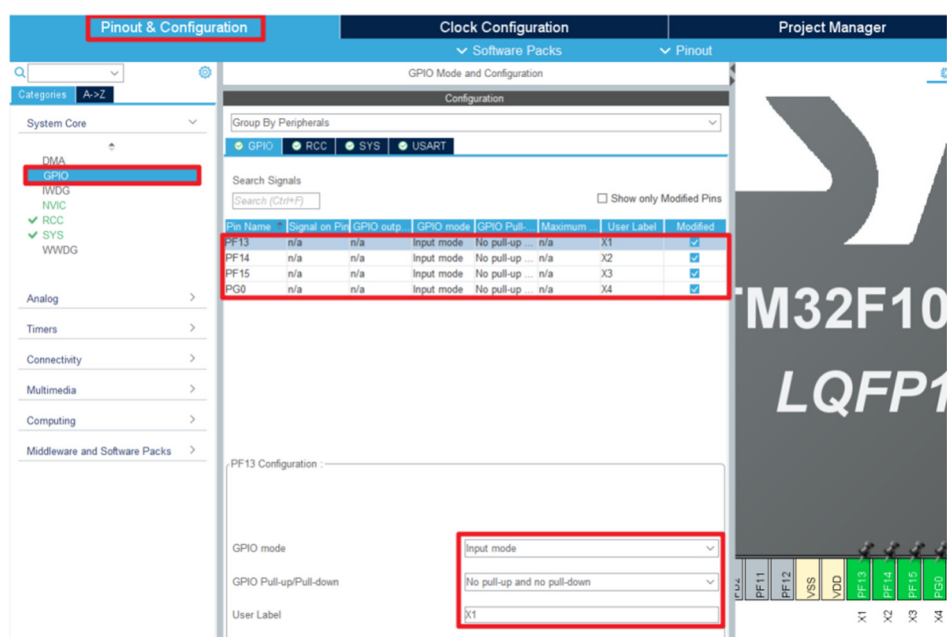


图 5.8 GPIO 引脚配置

并给出如下可能用到的函数：

函数原型	void Motion_Set_Speed (int16_t speed_m1, int16_t speed_m2, int16_t speed_m3, int16_t speed_m4)
功能描述	控制小车四个电机转向和速度
输入参数 1	speed_m1: Motor1 转向和速度
输入参数 2	speed_m2: Motor2 转向和速度
输入参数 3	speed_m3: Motor3 转向和速度
输入参数 4	speed_m4: Motor4 转向和速度
返回值	无

函数原型	uint8_t Read_irtracking(Irtracking_Pin irtracking)
功能描述	读取四路循迹模块接口电平
输入参数	irtracking: 四路循迹模块引脚, X1、X2、X3、X4
返回值	0 或 1: 0 代表低电平, 1 代表高电平

部分例程代码如下：

```
void car_irtrack(void)
{
    if((IN_X1 == 0 && IN_X3 == 0) && IN_X2 == 1 && IN_X4 == 1) //直走 go straight
    {
        Motion_Set_Speed(300,300,300,300);
    }

    if(IN_X1 == 0 && IN_X3 == 1 && IN_X4 == 1 && IN_X2 == 1)//小幅度调整 small adjustment
    {
        Motion_Set_Speed(0,0,500,500);
    }
    else if(IN_X1 == 1 && IN_X3 == 0 && IN_X4 == 1 && IN_X2 == 1)
    {
        Motion_Set_Speed(500,500,0,0);
    }

    if(IN_X2 == 0 && IN_X3 == 1 ) //大幅度左右转 Turn left and right sharply
    {
        Motion_Set_Speed(-500,-500,500,500);
    }
    else if(IN_X4 == 0 && IN_X1 == 1 )
    {
        Motion_Set_Speed(500,500,-500,-500);
    }

    //其它情况保持不变 Other things remain unchanged
}
```

6 单片机实验五：红外避障

6.1 实验目的

- 1) 熟悉 STM32CubeIDE 整体编程环境以及 MCU 程序开发流程；
- 2) 了解红外发射、红外接收模块的工作原理以及编程结构；
- 3) 学习红外发射、红外接收模块的编程技术。

6.2 实验原理

实时检测道路障碍并作出有效的规避是无人驾驶汽车在行驶中需要解决的关键问题之一。为避免发生车辆碰撞事故，要求车辆能够对周围环境中存在的障碍物进行实时、精确的检测与判断，并且针对不同类型的障碍物，进行合理的行驶轨迹规划，依照规划的避障路线完成行驶过程。智能车辆的避障过程示意图如图 6.1 所示。

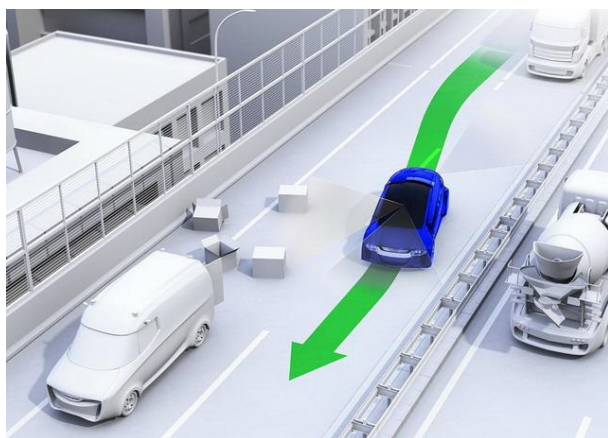


图 6.1 智能车辆避障过程示意图

本实验将在现有的智能小车所具备的软硬件基础上，通过对障碍检测、避障动作执行过程作一定的简化，实现上述智能车辆避障功能的模拟与还原，加深大家对机械、电子电路方面相关知识的理解与认识。

6.2.1 红外线发射/接收二极管原理

根据所基于的物理原理不同，目前适用于车辆障碍检测的传感器包括：摄像头、超声波雷达、毫米波雷达、红外线探头等，其中红外线探头是利用红外线在障碍物表面反射的过程进行检测，原理简单易于实现，在无外界干扰的情况下具有较高的检测精度与带宽。

实验中小车使用到一对红外线发射/接收二极管外观如图 6.2 所示，其基本工作原理如图 6.3 所示。红外发射管在通电后正向导通，向外发射出红外线。接收管反向施加电压，通常处于截止状态。当有障碍物时，发射管射出的红外线经过障碍物表面反射至接收管，由于 PN 结受到光子轰击将产生反向电流，接收管反向导通。设计电路判断红外接收管的导通/截止状态即可实现障碍物检测。

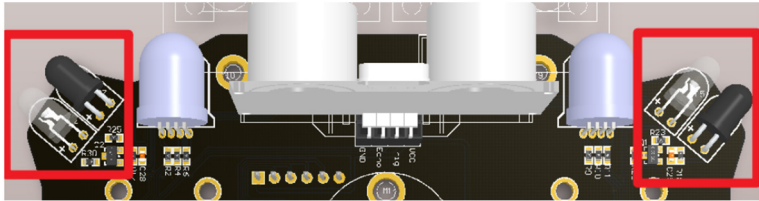


图 6.2 红外发射/接收二极管外观

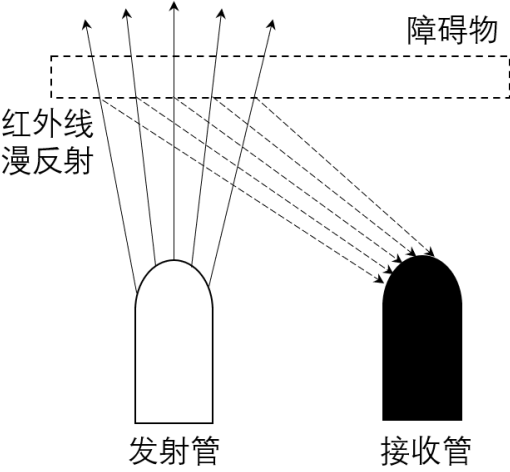


图 6.3 红外发射/接收二极管工作原理

6.2.2 红外避障电路原理

本实验中采用位于小车底板上的红外避障电路实现红外发射/接收二极管的驱动与状态判断，该电路如图 6.4 所示。在图 6.4 中：L4，L5 为红外发射管，D7，D8 为红外接收管。通过两路 ADC 转换，可以将红外避障模块输出的模拟电压转换为数字值；数值范围是 0 至 $2^{12}-1$ （即 0 至 4095）。

表 6.1 红外避障电路端口配置

红外避障（开发板集成）	对应引脚
P1	PE5(控制左边避障模块开关)
P2	PF9(ADC3_IN7)
P3	PE6(控制右边避障模块开关)
P4	PF10(ADC3_IN8)

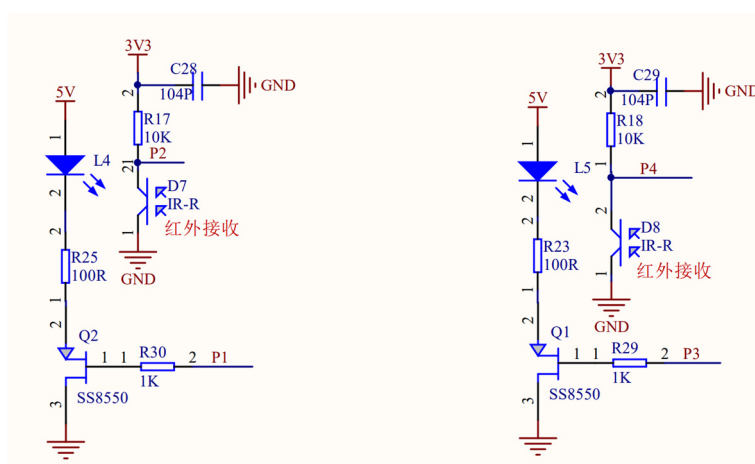


图 6.4 红外避障电路

综上，通过编写程序读取 MCU 的 I/O 端口的输入电平的低（或高）即可实现判断障碍物的有（或无）。以上红外避障电路工作原理可以简要概括为图 6.5 所示。

6.2.3 避障动作逻辑原理

在已实现障碍物的检测基础上，结合之前实验中已掌握的车辆前进/后退/左转/右转命令，需要设计程序令电机驱动车辆作出相应的避障动作避免发生碰撞。对此一个参考的避障动作逻辑是：车辆不断向前行驶，检测到前方两侧障碍物后，停止行驶，然后后退，然后左转，重复该过程直至前方无障碍物，如图 6.6 所示。

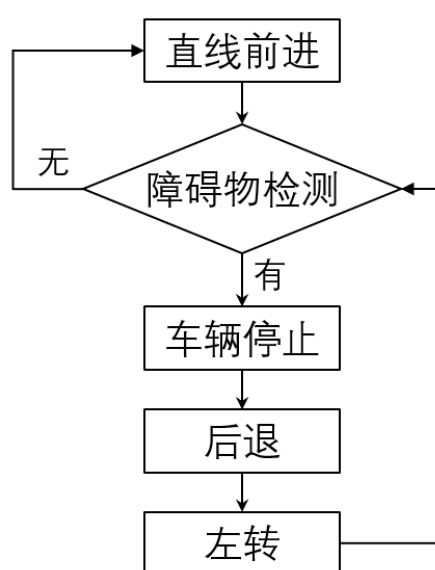


图 6.5 参考的避障动作逻辑原理图

6.3 实验内容

6.3.1 红外传感

实现正确判断车辆前方有无障碍物，并通过设置蜂鸣器或指示灯表示障碍物的有/无状态。

6.3.2 避障动作

实现障碍物的规避动作，要求车辆在行驶过程中遇到障碍物后采取相应动作，避免发生碰撞。部分例程代码如下：

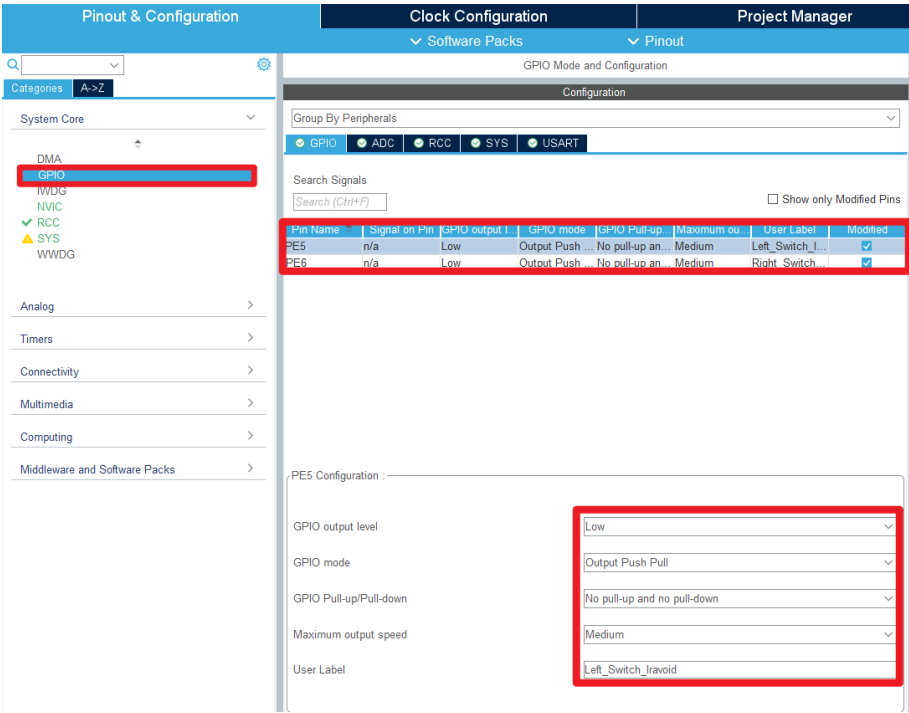


图 6.6 配置 GPIO 和 ADC 模块

```
uint16_t Adc_Get_Iravoid(uint32_t ch)
{
    ADC_ChannelConfTypeDef sConfig = {0};
    sConfig.Channel = ch; //ADC通道
    sConfig.Rank = ADC_REGULAR_RANK_1;
    sConfig.SamplingTime = ADC_SAMPLETIME_239CYCLES_5;
    if (HAL_ADC_ConfigChannel(&hadc3, &sConfig) != HAL_OK)
    {
        Error_Handler();
    }

    HAL_ADC_Start(&hadc3); //开始采集
    HAL_ADC_PollForConversion(&hadc3, 500);
    return HAL_ADC_GetValue(&hadc3);
}

left_data = Adc_Get_Iravoid(ADC_CHANNEL_7);
right_data = Adc_Get_Iravoid(ADC_CHANNEL_8);
```

7 单片机实验六：视觉识别（基础）

7.1 实验目的

- 1) 了解 K210 视觉识别模块的开发环境 CanMV IDE;
- 2) 掌握 K210 视觉识别模块的颜色识别方法;
- 3) 掌握 K210 视觉识别模块的手写数字方法;
- 4) 掌握 K210 视觉识别模块的自主学习分类方法。

7.2 实验原理

7.2.1 开发环境使用

K210 视觉识别模块搭载 K210 芯片，其芯片架构包含了一个自研的神经网络硬件加速器 KPU，可以高性能地进行卷积神经网络运算。其开发环境选用 CanMV IDE 软件，该软件可根据助教提供的软件包自行下载。编程语言使用 MicroPython，其为 Python 3 编程语言的一个简洁而高效的实现，包括 Python 标准库的一个小子集，并经过优化，可以在微控制器和受限环境中运行。

CanMV IDE 软件界面如图 7.1 所示。区域①菜单栏用于操作文件，使用工具等。区域②快捷按钮为区域①中文件和编辑中部分功能的快捷方式。区域③按钮为连接设备和程序控制按钮。区域④代码编辑区用于编辑要运行的代码。区域⑤串行终端用于显示串口打印的调试信息。区域⑥图像预览用于显示摄像头的图像预览。区域⑦固件信息用于显示连接的串口号以及固件的相关信息。

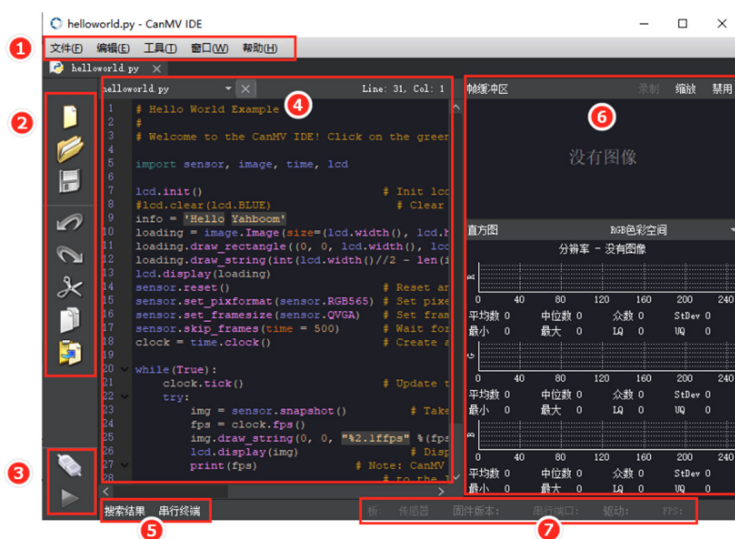


图 7.1 CanMV IDE 软件界面介绍

将 K210 模块通过 microUSB 数据线连接到电脑 USB 口，然后点击 CanMV IDE 左下角连接按钮。弹出选择串口界面，如图 7.2 所示。串口号可通过设备管理器/串口查询，选择 K210 模块对应的串口号。如果有多个串口请先确定对应的串口号再点击 OK 打开。如果连接不上，可以点击高级设置，选择 Mode-2。连接成功会将图标变为已连接状态，同时灰色播放键变为绿色。

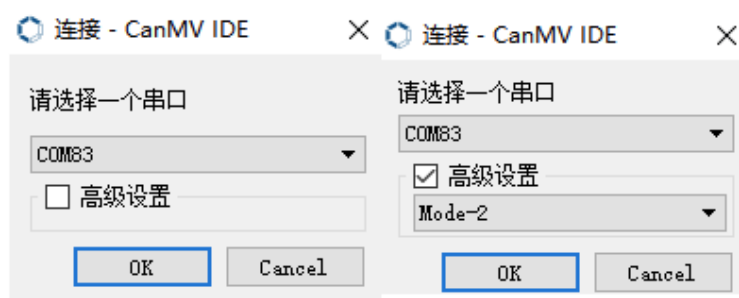


图 7.2 选择串口界面

连接完成后，可以看到左下角的连接图标变成已连接符号。可打开文件->示例->Basics->helloworld.py，如图 7.3 所示。示例中有多项文件可供学习，涉及 K210 模块的多项功能，学有余力的同学可自行学习。

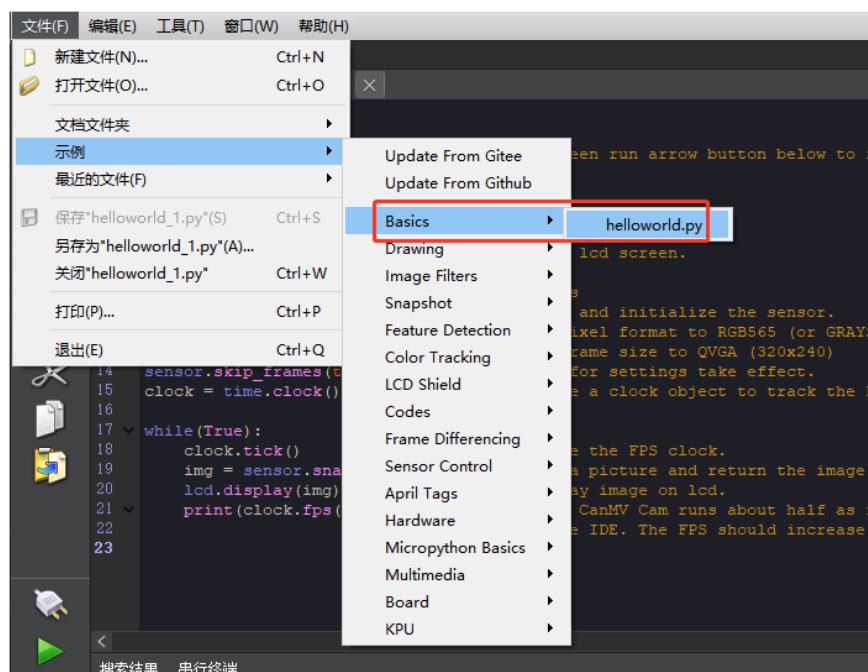


图 7.3 打开示例操作

点击绿色的播放键可运行程序，此时可以看到 LCD 显示摄像头画面，同时 IDE 右上角也有显示摄像头画面。绿色播放键图标变成红色停止键，点击

一下可以停止运行的程序。

上述步骤运行的程序，断开连接、按复位键或者关机后，程序丢失不保存。若想要将程序保存至 K210 模块内，重新上电后即可自动运行，则需要执行以下步骤。

在保持连接的情况，点击工具->保存当前打开的脚本为（main.py）到 CanMV Cam。出现去除注释并将空格转换为制表符提示后，点击 Yes 保存程序，完成后，断开连接或者按一下复位按键，程序会自动运行。如果有插入 TF 卡，程序优先保存到 TF 卡，并且优先运行 TF 卡里的 main.py 文件。

7.2.2 颜色识别

该实验主要学习颜色识别功能，根据颜色的 LAB 值，框出相同颜色的物品。LAB 颜色空间是一种色彩模型，它基于人类视觉对颜色的感知方式。LAB 值由三个部分组成：L（Lightness）：亮度，表示颜色的明暗程度。L 的值范围通常是从 0（黑色）到 100（白色）。A：从绿色到红色的色度分量。A 的值可以是正数或负数。正值表示颜色偏向红色，负值表示颜色偏向绿色。B：从蓝色到黄色的色度分量。同样，B 的值也可以是正数或负数。正值表示颜色偏向黄色，负值表示颜色偏向蓝色。颜色识别的功能主要是分析颜色的 LAB 值，先把要识别的颜色放方框内，然后系统会根据方框内读取到的颜色的 LAB 值，再与摄像头采集到的颜色的 LAB 值作为分析对比，如果符合要求则画出方框，表示识别到该颜色。由于识别颜色存在误差，最好是在识别颜色与背景颜色差距较大情况下识别，如果背景颜色与识别的颜色相近，误识别的几率会增加。

7.2.3 数字识别

该实验主要学习 K210 识别数字的功能。实验中基于已经训练好的机器学习模型，对手写或打印数字进行识别。由于采用卷积神经网络，该模型可对图像进行高效识别与分类。模型训练过程已经提前完成，同学们可直接调用。另外，由于模型训练采用的是白底黑字，所以请使用白底黑字素材，如果背景太乱，或者数字太小，可能会出现误识别的情况。

7.2.4 自主学习分类（选做）

该部分内容主要学习 K210 自主学习分类功能，仅作参考，供学有余力的同学选做。拿出三个不同的物体，拍摄不同角度的图片，K210 进行分类。整个例程分为三种状态，第一种状态（INIT）是初始化状态，第二种状态（TRAIN）是训练状态，第三种状态（CLASSIFY）是分类状态，每种状态对应一个 loop 函数运行程序内容。三种状态的切换通过 BOOT 键控制。

7.3 实验内容

7.3.1 颜色识别

首先导入相关库，并初始化摄像头和 LCD 显示屏。

```
import sensor, image, time, lcd

lcd.init()
sensor.reset()
sensor.set_pixformat(sensor.RGB565)
sensor.set_framesize(sensor.QVGA)
sensor.skip_frames(time = 2000)
sensor.set_auto_gain(False)
sensor.set_auto_whitebal(False)
clock = time.clock()
```

接着摄像头画面中绘制一个 50*50 大小的白色方框，此时的功能为提示用户将要识别的颜色放入方框内。

```
r = [(320//2)-(50//2), (240//2)-(50//2), 50, 50]
for i in range(50):
    img = sensor.snapshot()
    img.draw_rectangle(r)
    lcd.display(img)
```

等到方框又白色变成绿色，则开始学习颜色的 LAB 值，多次读取数值，并取平均值作为学习的 LAB 值结果。

```
print("Learning thresholds...")
threshold = [50, 50, 0, 0, 0, 0] # Middle L, A, B values.
for i in range(50):
    img = sensor.snapshot()
    hist = img.get_histogram(roi=r)
    lo = hist.get_percentile(0.01) # Get the CDF of the histogram at the 1% range (ADJUST AS NECESSARY)!
    hi = hist.get_percentile(0.99) # Get the CDF of the histogram at the 99% range (ADJUST AS NECESSARY)!
    # Average in percentile values.
    threshold[0] = (threshold[0] + lo.l_value()) // 2
    threshold[1] = (threshold[1] + hi.l_value()) // 2
    threshold[2] = (threshold[2] + lo.a_value()) // 2
    threshold[3] = (threshold[3] + hi.a_value()) // 2
    threshold[4] = (threshold[4] + lo.b_value()) // 2
    threshold[5] = (threshold[5] + hi.b_value()) // 2
    for blob in img.find_blobs([threshold], pixels_threshold=100, area_threshold=100, merge=True, margin=10):
        img.draw_rectangle(blob.rect())
        img.draw_cross(blob.cx(), blob.cy())
        img.draw_rectangle(r, color=(0,255,0))
    lcd.display(img)
```

当颜色学习完成，新建一个 while 循环，开始识别摄像头画面中的颜色，分析与上一步学习到的颜色的 LAB 值是否相符合，如果符合则框出来对应的颜

色块。

```
print("Thresholds learned...")
print("Start Color Recognition...")
while(True):
    clock.tick()
    img = sensor.snapshot()
    for blob in img.find_blobs([threshold], pixels_threshold=100, area_threshold=100,
merge=True, margin=10):
        img.draw_rectangle(blob.rect())
        img.draw_cross(blob.cx(), blob.cy())
    lcd.display(img)
    print(clock.fps())
```

下面展示一个简单的实验过程，如图 7.4 所示。

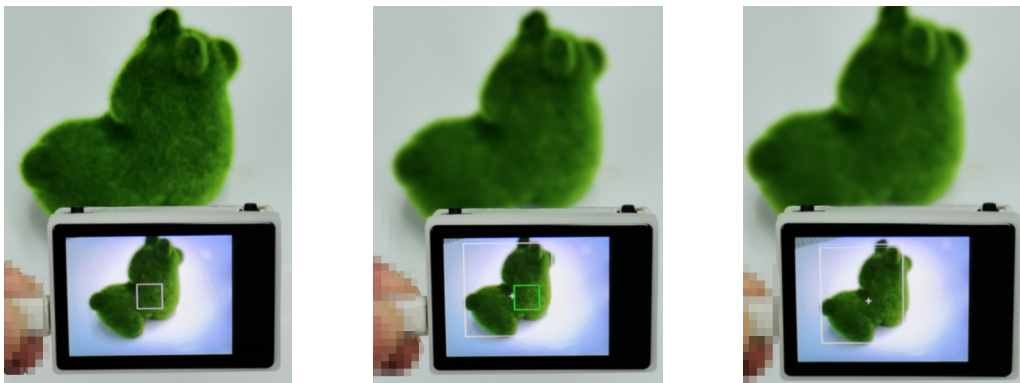


图 7.4 颜色识别实验过程

将 K210 模块通过 microUSB 数据线连接到电脑上，CanMV IDE 点击连接按钮，连接完成后点击运行按钮，运行例程代码。也可以将代码作为 main.py 下载到 K210 模块上运行。等待系统初始化完成后，LCD 显示摄像头画面，并且屏幕中间有一个白色的方框，请将要识别的颜色放到白色方框内，白色方框持续时间大约 3 秒。等到白色方框变为绿色方框时，此时系统开始学习绿色方框内的颜色 LAB 值，还会出现其他白色的方框作为预览效果，大约 5 秒后，绿色方框消失，则表示学习完成。此时将摄像头朝向要识别的颜色，系统会自动框出识别的颜色。

7.3.2 数字识别

实验前请将模型文件导入内存卡上中，具体操作是将 TF 卡插入读卡器，读卡器插入电脑 USB 口，下载资料中的模型文件压缩包并解压，将 KPU 文件夹复制到 H:盘的根目录。随后，将内存卡插入到 K210 模块的内存卡插槽上。

首先，导入相关库，并初始化摄像头和 LCD 显示屏，加载模型文件：

```
/sd/KPU/mnist/uint8_mnist_cnn_model.kmodel。
```

```
kpu = KPU()
kpu.load_kmodel("/sd/KPU/mnist/uint8_mnist_cnn_model.kmodel")
```

接着，新建 while 循环读取摄像头画面，然后复制一个 112*112 大小的画面，对像素进行取反等处理，再将图像传入 KPU 里进行计算，与模型文件做运算，最终得到最优识别结果和识别分数。

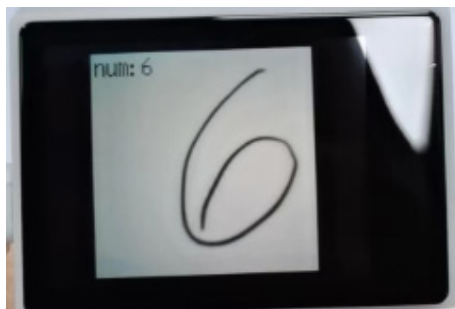
```
while True:
    gc.collect()
    img = sensor.snapshot()
    img_mnist1=img.to_grayscale(1)
    img_mnist2=img_mnist1.resize(112,112)
    img_mnist2.invert()
    img_mnist2.strech_char(1)
    img_mnist2.pix_to_ai()

    out = kpu.run_with_output(img_mnist2, getlist=True)
    max_mnist = max(out)
    index_mnist = out.index(max_mnist)
    score = KPU.sigmoid(max_mnist)
```

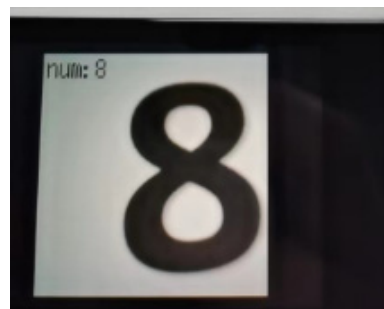
由于识别的数字有可能会出现问题，请尽量在白色背景下识别。接下来进行过滤，根据实际测试出来的结果，背景全白的情况下，会误识别成数字 1，背景全黑的情况下，会误识别成数字 5。所以单独对识别到数字 1 和数字 5 进行额外分析，如果识别分数大于 0.999 才认为是对应的数字，反正就认为没有识别到数字。其中 0.999 可以根据实际识别效果做调整。最终将识别到的数字显示在屏幕上。

```
if index_mnist == 1:
    if score > 0.999:
        display_str = "num: %d" % index_mnist
        print(display_str, score)
        img.draw_string(4,3,display_str,color=(0,0,0),scale=2)
elif index_mnist == 5:
    if score > 0.999:
        display_str = "num: %d" % index_mnist
        print(display_str, score)
        img.draw_string(4,3,display_str,color=(0,0,0),scale=2)
else:
    display_str = "num: %d" % index_mnist
    print(display_str, score)
    img.draw_string(4,3,display_str,color=(0,0,0),scale=2)
lcd.display(img)
```

下面展示一个简单的实验过程，如图 7.5 所示。将 K210 模块通过 microUSB 数据线连接到电脑上，CanMV IDE 点击连接按钮，连接完成后点击运行按钮，运行例程代码。也可以将代码作为 main.py 下载到 K210 模块上运行。等待系统初始化完成后，LCD 显示摄像头画面，将摄像头拍摄自己手写的数字或打印的数字，屏幕左上角会显示当前识别到的数字。



(a) 数字 6



(b) 数字 8

图 7.5 数字识别实验过程

7.3.3 自主学习分类（选做）

该部分内容仅作参考，供学有余力的同学选做。实验前将模型文件导入内存卡中。实验中先导入相关库，并初始化摄像头和 LCD 显示屏，加载模型文件：`/sd/KPU/self_learn_classifier/mb-0.25.kmodel`。整个例程分为三种状态，第一种状态（INIT）是初始化状态，第二种状态（CLASSIFY）是分类状态，第三种状态（TRAIN）是训练状态，每种状态对应一个 loop 函数运行程序内容。

```
if state_machine.current_state == STATE.INIT:
    loop_init()
elif state_machine.current_state == STATE.CLASSIFY:
    loop_classify()
elif state_machine.current_state == STATE.TRAIN_CLASS_1 \
    or state_machine.current_state == STATE.TRAIN_CLASS_2 \
    or state_machine.current_state == STATE.TRAIN_CLASS_3:
    loop_capture()
```

初始化状态的 loop 函数主要功能是显示提示信息，短按 BOOT 进行下一步，长按 BOOT 键重新开始。

```
def loop_init():
    if state_machine.current_state != STATE.INIT:
        return

    img_init.draw_rectangle(0, 0, lcd.width(), lcd.height(), color=(0, 0, 255),
        fill=True, thickness=2)
    img_init.draw_string(65, 90, "Self Learning Demo", color=(255, 255, 255), scale=2)
    img_init.draw_string(5, 210, "Short press: next", color=(255, 255, 255), scale=1)
    img_init.draw_string(5, 225, "Long press: restart", color=(255, 255, 255),
        scale=1)
    lcd.display(img_init)
```

训练状态的 loop 函数主要功能是显示训练的类别和操作的进度。在训练状态中，需要多次按下 BOOT 按键进行下一步。默认情况下，总共训练三个类别，每个类别需要拍摄 5 张图片。

```
def loop_capture():
    global central_msg, bottom_msg
    img = sensor.snapshot()
    if central_msg:
        img.draw_rectangle(0, 90, lcd.width(), 22, color=(0, 0, 255), fill=True,
thickness=2)
        img.draw_string(55, 90, central_msg, color=(255, 255, 255), scale=2)
    if bottom_msg:
        img.draw_string(5, 208, bottom_msg, color=(0, 0, 255), scale=1)
    lcd.display(img)
```

分类状态的 `loop` 函数主要功能是进行分析对比，分类出当前摄像头画面是属于哪一种类别，显示对应的类别和识别分数。

```
def loop_classify():
    global central_msg, bottom_msg
    img = sensor.snapshot()

    scores = []
    feature = kpu.run_with_output(img, get_feature=True)
    high = 0
    index = 0
    for j in range(len(features)):
        for f in features[j]:
            score = kpu.feature_compare(f, feature)
            if score > high:
                high = score
                index = j
    if high > THRESHOLD:
        bottom_msg = "class:{},score: {:.2.1f}".format(index + 1, high)
    else:
        bottom_msg = None

    # display info
    if central_msg:
        print("central_msg:{}".format(central_msg))
        img.draw_rectangle(0, 90, lcd.width(), 22, color=(0, 255, 0), fill=True,
thickness=2)
        img.draw_string(55, 90, central_msg, color=(255, 255, 255), scale=2)
    if bottom_msg:
        print("bottom_msg:{}".format(bottom_msg))
        img.draw_string(5, 208, bottom_msg, color=(0, 255, 0), scale=1)
    lcd.display(img)
```

该实验提供具体代码，同学们请认真学习相关代码，根据实际任务需求做出适当修改。这里简要介绍一下实验过程（图 7.6），需要特别注意的是，由于需要用到 BOOT 按键，不要在 CanMV IDE 里直接运行，CanMV IDE 目前无法检测到 BOOT 按键，请将代码作为 `main.py` 下载到 K210 模块上运行。

等待开机完成，LCD 屏幕显示 Self Learning Demo（图 7.6(a)），提示按一下进入下一个状态，长按重新开始。此时按一下 BOOT 按键进入下一个状态。在训练第一个物体时，按一下 BOOT 按键进入下一步，开始采集物体图片素材。

将第一个要识别的物体放到摄像头可视范围内，可以看到屏幕左下角显示 Class 1: #P1（图 7.6(b)），按一下 BOOT 按键，则记录类别 1 的第 1 个素材图片。其中 Class 表示类别序号，P 表示图片序号。稍微转变一下拍摄角度，再按一下 BOOT 键，采集第二张素材图片。当采集了五张图片后，类别 1 采集结束。接下来进入类别 2 的采集过程，按一下 BOOT 键开始采集类别 2 的素材图片，步骤与采集类别 1 的相同。类别 2 采集完五张素材图片后，进入类别 3 采集五张素材图片。结束采集后自动进入分类状态。屏幕中间显示 Classificatiion 表示已经进入分类状态，再按一次 BOOT 按键自动消失提示。此时将摄像头拍摄刚刚训练的三个类别的其中一个，屏幕左下角会显示对应类别序号和分数（图 7.6(c)）。如果识别效果不好，或者图像采集出错，可以按一下复位，或者长按 BOOT 按键重新开始。

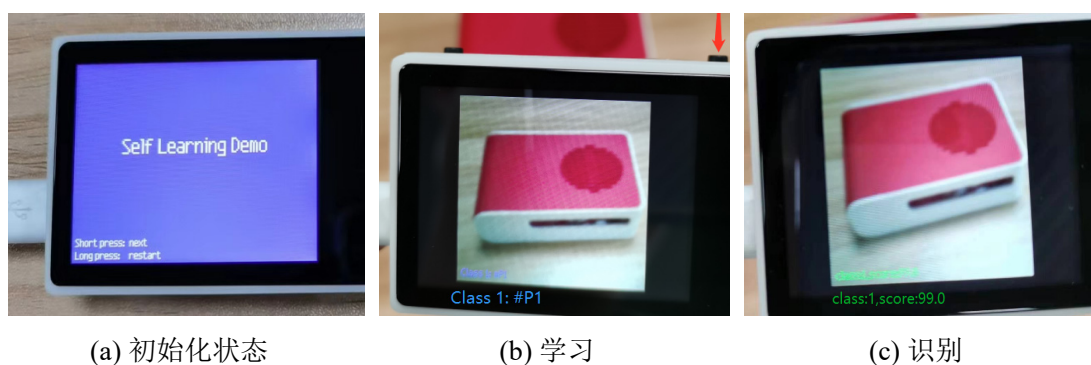


图 7.6 自主学习分类实验过程

8 单片机实验七：循迹、避障、视觉综合

8.1 实验目的

- 1) 熟悉 STM32CubeIDE 整体编程环境以及 MCU 程序开发流程；
- 2) 学习掌握 STM32 单片机的综合编程技术；
- 3) 进行研究型实验，提高综合能力。

8.2 实验原理

STM32 开发板的硬件资源是十分强大的，拥有 18 大功能，本实验将黑线循迹、红外避障和视觉识别三大模块集成到一起，应用到上述几个实验的全部原理，同时实现小车沿着地面黑线循迹、避障与视觉识别三大功能进行综合测试，以此让大家了解 STM32 单片机开发功能的强大之处。



图 8.1 智能小车黑线循迹、红外避障、视觉识别示意图

智能小车系统以处理器为核心，其系统结构如图 8.2 所示。为了使其能够沿所需轨迹快速行驶并精确避开障碍物，处理器必须把路径轨迹及视觉处理信号、障碍物的迅速判断信号、相应的转向电机控制以及直流驱动电机的控制精密地结合在一起。

如果传感器部分的数据没有正确地采集和识别，转向电机控制的失当，都会造成模型车严重抖动甚至偏离赛道；如果直流电机的驱动控制效果不好，也会造成直线路段的速度过低，弯曲路段速度过快等问题。因此，本实验需要综合考虑各模块部分的原理及匹配，以实现综合实验的设计功能。

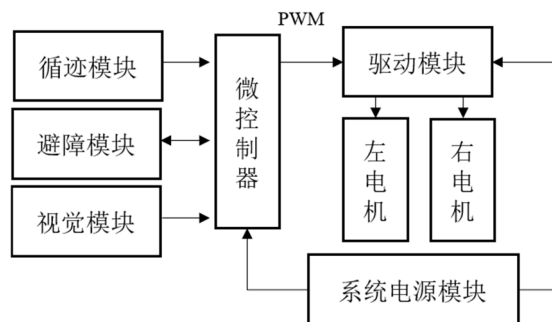


图 8.2 基于 STM32 智能小车总体结构框图

8.3 实验内容

8.3.1 按键切换

按键切换设计：将不同的功能结合到一起，需进行模式的切换。通过检测是否有键按下对模式进行选择。当没有按下按键时选择为模式一：循迹模式，按下按键时选择为模式二：避障模式。KEY：默认高电平，按下低电平。

部分例程代码如下：

```
#include <key/bsp_key.h>

/*重写中断回调处理函数*/
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    switch (GPIO_Pin)
    {
        case GPIO_PIN_3:
            Set_Buzzer(0); // key1按键功能：关闭蜂鸣器和全部LED灯
            Set_led(1, 0);
            Set_led(2, 0);
            break;
        case GPIO_PIN_4:
            Set_led(1, 1); // key2按键功能：打开LED1
            break;
        case GPIO_PIN_5:
            Set_Buzzer(1); // key3按键功能：打开蜂鸣器并打开LED2
            Set_led(2, 1);
            break;
    }
}
//任务：循迹、避障模式处理
```

8.3.2 循迹避障

循迹的基础上进行避障。实现小车在黑色循迹赛道中没有遇到障碍物时一直前行，遇到障碍物后停止前行。

8.3.3 综合实验

利用黑线循迹、避障、视觉识别模块使小车沿着特定的轨迹从起点行进至终点（注意三种模块优先级设置）。

部分例程代码如下：现象：1.打开开关后正常巡线；2.识别到限速标志以低于此速度巡线；3.识别到解除限速标志恢复正常巡线；4.识别到左/右转，优先转弯；5.识别到鸣笛，停下鸣笛两声再恢复巡线；6.识别 1、2 号库，入库 2s 再恢复巡线。

```
#include "bsp.h"

#define speed_go 300 //出停止线速度 Stop line speed
#define de_time 200 //出停止线延迟时间 Exit stop line delay time

uint8_t stop_flag = 0;
uint32_t n_stime = 0; //恢复正常巡线的时间 Time to resume normal line patrol
uint16_t wait_k210 = 700; //等待k210启动的时间 Waiting time for k210 to start
uint8_t id_num = MAX_id;
uint8_t id_num_back = MAX_id;

void BSP_Init(void)
{
    Bsp_Tim_Init();
    PID_Param_Init(); //电机PID初始化 Motor PID initialization
    // 这个库初始化完成会有0xff的数据
    // After the initialization of this library is completed, there will be 0xff data sent
    USART2_UART_Init();

    //设置一个等待k210正常开启的时间 Set a time to wait for k210 to turn on normally
    while(wait_k210);
}

void BSP_Loop(void)
{
    id_num = get_dataid(); //获取最新的id Get the latest id

    if(id_num != id_num_back) //状态发生改变 status changes
    {
        id_num_back = id_num;

        //以下的标志都逃离黑线 The following signs are all escaping from the black line
        switch(id_num)
        {
            case green_light:
            case school:
            case walk:
            case right:
            case left:
            case freeSpeed:
            case limitSpeed:
            case horn :
                Leave_Stop();
                Motion_Set_Pwm(200,100,200,100); //处理出黑线检查不到线的状态
                break;

            case one:
                Reverse_parking_no1(); //倒1库 Back up into parking garage 1
                break;

            case two:
                Reverse_parking_no2(); //倒2库 Back up into parking garage 2
                break;

            case chuku_track_line: break; //不做任何操作 Do nothing

            default: Motion_Set_Pwm(200,100,200,100); //处理出黑线检查不到线的状态
        }
    }
}
```

```

switch(id_num_back)
{
    case red_light:
        Motion_Set_Speed(0,0,0,0); //小车停止 Car stops
        set_dataid(MAX_id); //回到正常巡线 Return to normal line patrol
        break;

    case green_light:
        set_dataid(MAX_id); //回到正常巡线 Return to normal line patrol
        break;

    case school:
    case walk:
        set_dataid(MAX_id); //回到正常巡线 Return to normal line patrol
        break;

    case right:
        Road_sign_right(); //右转优先巡线 Right turn priority line patrol
        break;

    case left:
        Road_sign_left(); //左转优先巡线 Left turn priority line patrol
        break;

    case freeSpeed:
        set_dataid(MAX_id); //回到正常巡线 Return to normal line patrol
        break;

    case limitSpeed:
        Road_sign_speedlimit(); //低速巡线 Low speed line patrol
        break;

    case chuku_track_line: //出库后的巡线 Line patrol after exiting the garage
        car_outbound_track();
        break;

    case horn:
        beep();
        break;

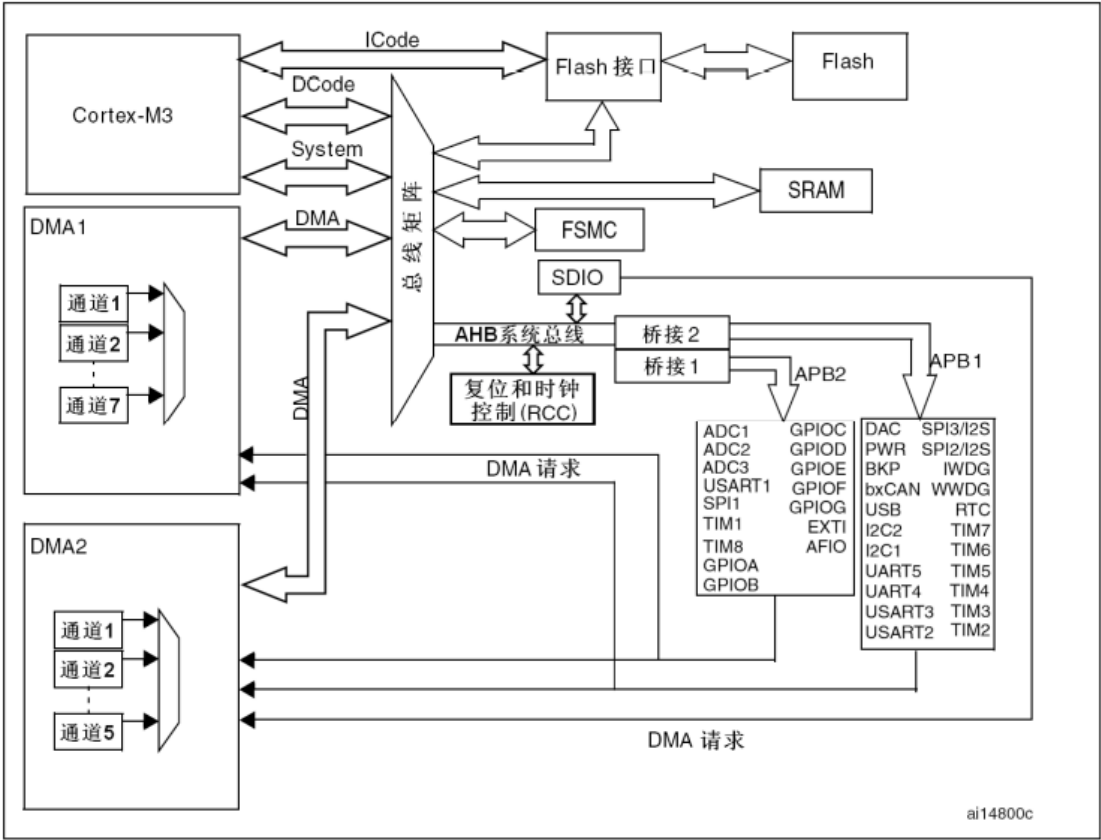
    //不做任何处理 No processing
    case one:
    case two:

    default: Track_line(); //调用正常巡线函数 Call the normal line following function
}

```

附录 A STM32 寄存器分布表

A.1 系统架构



A.2 时钟控制寄存器 (RCC_CR)

偏移地址: 0x00

复位值: 0x000 XX83, X代表未定义

访问: 无等待状态, 字, 半字 和 字节访问

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
保留						PLL RDY	PLLON	保留				CSS ON	HSE BYP	HSE RDY	HSE ON
						r	rw					rw	rw	r	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
HSICAL[7:0]								HSITRIM[4:0]				保留	HSI RDY	HSION	
r	r	r	r	r	r	r	r	rw	rw	rw	rw	rw		r	rw

位31:26	保留, 始终读为0。
位25	PLL RDY : PLL时钟就绪标志 (PLL clock ready flag) PLL锁定后由硬件置'1'。 0: PLL未锁定; 1: PLL锁定。
位24	PLLON : PLL使能 (PLL enable) 由软件置'1'或清零。 当进入待机和停止模式时, 该位由硬件清零。当PLL时钟被用作或被选择将要作为系统时钟时, 该位不能被清零。 0: PLL关闭; 1: PLL使能。
位23:20	保留, 始终读为0。
位19	CSSON : 时钟安全系统使能 (Clock security system enable) 由软件置'1'或清零以使能时钟监测器。 0: 时钟监测器关闭; 1: 如果外部4-16MHz振荡器就绪, 时钟监测器开启。
位18	HSEBYP : 外部高速时钟旁路 (External high-speed clock bypass) 在调试模式下由软件置'1'或清零来旁路外部晶体振荡器。只有在外部4-16MHz振荡器关闭的情况下, 才能写入该位。 0: 外部4-16MHz振荡器没有旁路; 1: 外部4-16MHz外部晶体振荡器被旁路。
位17	HSERDY : 外部高速时钟就绪标志 (External high-speed clock ready flag) 由硬件置'1'来指示外部4-16MHz振荡器已经稳定。在HSEON位清零后, 该位需要6个外部4-25MHz振荡器周期清零。 0: 外部4-16MHz振荡器没有就绪; 1: 外部4-16MHz振荡器就绪。

位16	HSEON: 外部高速时钟使能 (External high-speed clock enable) 由软件置'1'或清零。 当进入待机和停止模式时, 该位由硬件清零, 关闭4-16MHz外部振荡器。当外部4-16MHz振荡器被用作或被选择将要作为系统时钟时, 该位不能被清零。 0: HSE振荡器关闭; 1: HSE振荡器开启。
位15:8	HSICAL[7:0]: 内部高速时钟校准 (Internal high-speed clock calibration) 在系统启动时, 这些位被自动初始化
位7:3	HSITRIM[4:0]: 内部高速时钟调整 (Internal high-speed clock trimming) 由软件写入来调整内部高速时钟, 它们被叠加在HSICAL[5:0]数值上。 这些位在HSICAL[7:0]的基础上, 让用户可以输入一个调整数值, 根据电压和温度的变化调整内部HSI RC振荡器的频率。 默认数值为16, 可以把HSI调整到8MHz±1%; 每步HSICAL的变化调整约40kHz。
位2	保留, 始终读为0。
位1	HSIRDY: 内部高速时钟就绪标志 (Internal high-speed clock ready flag) 由硬件置'1'来指示内部8MHz振荡器已经稳定。在HSION位清零后, 该位需要6个内部8MHz振荡器周期清零。 0: 内部8MHz振荡器没有就绪; 1: 内部8MHz振荡器就绪。
位0	HSION: 内部高速时钟使能 (Internal high-speed clock enable) 由软件置'1'或清零。 当从待机和停止模式返回或用作系统时钟的外部4-16MHz振荡器发生故障时, 该位由硬件置'1'来启动内部8MHz的RC振荡器。当内部8MHz振荡器被直接或间接地用作或被选择将要作为系统时钟时, 该位不能被清零。 0: 内部8MHz振荡器关闭; 1: 内部8MHz振荡器开启。

A.3 时钟配置寄存器

偏移地址: 0x04

复位值: 0x0000 0000

访问: 0到2个等待周期, 字, 半字和字节访问

只有当访问发生在时钟切换时, 才会插入1或2个等待周期。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
保留					MCO[2:0]			保留	USB PRE	PLLMUL[3:0]			PLL XTPRE	PLL SRC	
					RW	RW	RW		RW	RW	RW	RW	RW	RW	RW
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ADCPRE[1:0]		PPRE2[2:0]			PPRE1[2:0]			HPRE[3:0]			SWS[1:0]		SW[1:0]		
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	R	R	RW	RW
位31:27		保留, 始终读为0。													

位26:24	MCO: 微控制器时钟输出 (Microcontroller clock output) 由软件置'1'或清零。 0xx: 没有时钟输出; 100: 系统时钟(SYSCLK)输出; 101: 内部RC振荡器时钟(HSI)输出; 110: 外部振荡器时钟(HSE)输出; 111: PLL时钟2分频后输出。 注意: - 该时钟输出在启动和切换MCO时钟源时可能会被截断。 - 在系统时钟作为输出至MCO引脚时, 请保证输出时钟频率不超过50MHz (I/O口最高频率)。																
位22	USBPRE: USB预分频 (USB prescaler) 由软件置'1'或清'0'来产生48MHz的USB时钟。在RCC_APB1ENR寄存器中使能USB时钟之前, 必须保证该位已经有效。如果USB时钟被使能, 该位不能被清零。 0: PLL时钟1.5倍分频作为USB时钟 1: PLL时钟直接作为USB时钟																
位21:18	PLLMUL: PLL倍频系数 (PLL multiplication factor) 由软件设置来确定PLL倍频系数。只有在PLL关闭的情况下才可被写入。 注意: PLL的输出频率不能超过72MHz <table border="0"> <tr> <td>0000: PLL 2倍频输出</td><td>1000: PLL 10倍频输出</td></tr> <tr> <td>0001: PLL 3倍频输出</td><td>1001: PLL 11倍频输出</td></tr> <tr> <td>0010: PLL 4倍频输出</td><td>1010: PLL 12倍频输出</td></tr> <tr> <td>0011: PLL 5倍频输出</td><td>1011: PLL 13倍频输出</td></tr> <tr> <td>0100: PLL 6倍频输出</td><td>1100: PLL 14倍频输出</td></tr> <tr> <td>0101: PLL 7倍频输出</td><td>1101: PLL 15倍频输出</td></tr> <tr> <td>0110: PLL 8倍频输出</td><td>1110: PLL 16倍频输出</td></tr> <tr> <td>0111: PLL 9倍频输出</td><td>1111: PLL 16倍频输出</td></tr> </table>	0000: PLL 2倍频输出	1000: PLL 10倍频输出	0001: PLL 3倍频输出	1001: PLL 11倍频输出	0010: PLL 4倍频输出	1010: PLL 12倍频输出	0011: PLL 5倍频输出	1011: PLL 13倍频输出	0100: PLL 6倍频输出	1100: PLL 14倍频输出	0101: PLL 7倍频输出	1101: PLL 15倍频输出	0110: PLL 8倍频输出	1110: PLL 16倍频输出	0111: PLL 9倍频输出	1111: PLL 16倍频输出
0000: PLL 2倍频输出	1000: PLL 10倍频输出																
0001: PLL 3倍频输出	1001: PLL 11倍频输出																
0010: PLL 4倍频输出	1010: PLL 12倍频输出																
0011: PLL 5倍频输出	1011: PLL 13倍频输出																
0100: PLL 6倍频输出	1100: PLL 14倍频输出																
0101: PLL 7倍频输出	1101: PLL 15倍频输出																
0110: PLL 8倍频输出	1110: PLL 16倍频输出																
0111: PLL 9倍频输出	1111: PLL 16倍频输出																
位17	PLLXTPRE: HSE分频器作为PLL输入 (HSE divider for PLL entry) 由软件置'1'或清'0'来分频HSE后作为PLL输入时钟。只能在关闭PLL时才能写入此位。 0: HSE不分频 1: HSE 2分频																
位16	PLLSRC: PLL输入时钟源 (PLL entry clock source) 由软件置'1'或清'0'来选择PLL输入时钟源。只能在关闭PLL时才能写入此位。 0: HSI振荡器时钟经2分频后作为PLL输入时钟 1: HSE时钟作为PLL输入时钟。																
位15:14	ADCPRE[1:0]: ADC预分频 (ADC prescaler) 由软件置'1'或清'0'来确定ADC时钟频率 00: PCLK2 2分频后作为ADC时钟 01: PCLK2 4分频后作为ADC时钟 10: PCLK2 6分频后作为ADC时钟 11: PCLK2 8分频后作为ADC时钟																
位13:11	PPRE2[2:0]: 高速APB预分频 (APB2) (APB high-speed prescaler (APB2)) 由软件置'1'或清'0'来控制高速APB2时钟(PCLK2)的预分频系数。 0xx: HCLK不分频 100: HCLK 2分频 101: HCLK 4分频 110: HCLK 8分频 111: HCLK 16分频																

位10:8	PPRE1[2:0]: 低速APB预分频(APB1) (APB low-speed prescaler (APB1)) 由软件置'1'或清'0'来控制低速APB1时钟(PCLK1)的预分频系数。 警告: 软件必须保证APB1时钟频率不超过36MHz。 0xx: HCLK不分频 100: HCLK 2分频 101: HCLK 4分频 110: HCLK 8分频 111: HCLK 16分频
位7:4	HPRE[3:0]: AHB预分频 (AHB Prescaler) 由软件置'1'或清'0'来控制AHB时钟的预分频系数。 0xxx: SYSCLK不分频 1000: SYSCLK 2分频 1100: SYSCLK 64分频 1001: SYSCLK 4分频 1101: SYSCLK 128分频 1010: SYSCLK 8分频 1110: SYSCLK 256分频 1011: SYSCLK 16分频 1111: SYSCLK 512分频 注意: 当AHB时钟的预分频系数大于1时, 必须开启预取缓冲器。详见闪存读取(第2.3.3节)。
位3:2	SWS[1:0]: 系统时钟切换状态 (System clock switch status) 由硬件置'1'或清'0'来指示哪一个时钟源被作为系统时钟。 00: HSI作为系统时钟; 01: HSE作为系统时钟; 10: PLL输出作为系统时钟; 11: 不可用。
位1:0	SW[1:0]: 系统时钟切换 (System clock switch) 由软件置'1'或清'0'来选择系统时钟源。 在从停止或待机模式中返回时或直接或间接作为系统时钟的HSE出现故障时, 由硬件强制选择HSI作为系统时钟(如果时钟安全系统已经启动) 00: HSI作为系统时钟; 01: HSE作为系统时钟; 10: PLL输出作为系统时钟; 11: 不可用。

A.4 时钟中断寄存器

偏移地址: 0x08

复位值: 0x0000 0000

访问: 无等待周期, 字, 半字 和 字节访问

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
保留								CSSC	保留		PLL RDYC	HSE RDYC	HSI RDYC	LSE RDYC	LSI RDYC
								W			W	W	W	W	W
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
保留		PLL RDYIE	HSE RDYIE	HSI RDYIE	LSE RDYIE	LSI RDYIE	CSSF	保留		PLL RDYF	HSE RDYF	HSI RDYF	LSE RDYF	LSI RDYF	
		rW	rW	rW	rW	rW	r			r	r	r	r	r	
位31:24		保留，始终读为0。													
位23		CSSC ：清除时钟安全系统中断 (Clock security system interrupt clear) 由软件置'1'来清除CSSF安全系统中断标志位CSSF。 0：无作用； 1：清除CSSF安全系统中断标志位。													

位22:21	保留，始终读为0。
位20	PLLRDYC : 清除PLL就绪中断 (PLL ready interrupt clear) 由软件置'1'来清除PLL就绪中断标志位PLLRDYF。 0: 无作用; 1: 清除PLL就绪中断标志位PLLRDYF。
位19	HSERDYC : 清除HSE就绪中断 (HSE ready interrupt clear) 由软件置'1'来清除HSE就绪中断标志位HSERDYF。 0: 无作用; 1: 清除HSE就绪中断标志位HSERDYF。
位18	HSIRDYC : 清除HSI就绪中断 (HSI ready interrupt clear) 由软件置'1'来清除HSI就绪中断标志位HSIRDYF。 0: 无作用; 1: 清除HSI就绪中断标志位HSIRDYF。
位17	LSEDYC : 清除LSE就绪中断 (LSE ready interrupt clear) 由软件置'1'来清除LSE就绪中断标志位LSERDYF。 0: 无作用; 1: 清除LSE就绪中断标志位LSERDYF。
位16	LSIRDYC : 清除LSI就绪中断 (LSI ready interrupt clear) 由软件置'1'来清除LSI就绪中断标志位LSIRDYF。 0: 无作用; 1: 清除LSI就绪中断标志位LSIRDYF。
位15:13	保留，始终读为0。
位12	PLLRDYIE : PLL就绪中断使能 (PLL ready interrupt enable) 由软件置'1'或清'0'来使能或关闭PLL就绪中断。 0: PLL就绪中断关闭; 1: PLL就绪中断使能。
位11	HSERDYIE : HSE就绪中断使能 (HSE ready interrupt enable) 由软件置'1'或清'0'来使能或关闭外部4-16MHz振荡器就绪中断。 0: HSE就绪中断关闭; 1: HSE就绪中断使能。
位10	HSIRDYIE : HSI就绪中断使能 (HSI ready interrupt enable) 由软件置'1'或清'0'来使能或关闭内部8MHz RC振荡器就绪中断。 0: HSI就绪中断关闭; 1: HSI就绪中断使能。
位9	LSEDYIE : LSE就绪中断使能 (LSE ready interrupt enable) 由软件置'1'或清'0'来使能或关闭外部32kHz RC振荡器就绪中断。 0: LSE就绪中断关闭; 1: LSE就绪中断使能。
位8	LSIRDYIE : LSI就绪中断使能 (LSI ready interrupt enable) 由软件置'1'或清'0'来使能或关闭内部40kHz RC振荡器就绪中断。 0: LSI就绪中断关闭; 1: LSI就绪中断使能。
位7	CSSF : 时钟安全系统中断标志 (Clock security system interrupt flag) 在外部4-16MHz振荡器时钟出现故障时，由硬件置'1'。 由软件通过置'1' CSSC位来清除。 0: 无HSE时钟失效产生的安全系统中断; 1: HSE时钟失效导致了时钟安全系统中断。
位6:5	保留，始终读为0。

位4	PLLRDYF : PLL就绪中断标志 (PLL ready interrupt flag) 在PLL就绪且PLLRDYIE位被置'1'时, 由硬件置'1'。 由软件通过置'1' PLLRDYC位来清除。 0: 无PLL上锁产生的时钟就绪中断; 1: PLL上锁导致时钟就绪中断。
位3	HSERDYF : HSE就绪中断标志 (HSE ready interrupt flag) 在外部低速时钟就绪且HSERDYIE位被置'1'时, 由硬件置'1'。 由软件通过置'1' HSERDYC位来清除。 0: 无外部4-16MHz振荡器产生的时钟就绪中断; 1: 外部4-16MHz振荡器导致时钟就绪中断。
位2	HSIRDYF : HSI就绪中断标志 (HSI ready interrupt flag) 在内部高速时钟就绪且HSIRDYIE位被置'1'时, 由硬件置'1'。 由软件通过置'1' HSIRDYC位来清除。 0: 无内部8MHz RC振荡器产生的时钟就绪中断; 1: 内部8MHz RC振荡器导致时钟就绪中断。
位1	LSERDYF : LSE就绪中断标志 (LSE ready interrupt flag) 在外部低速时钟就绪且LSERDYIE位被置'1'时, 由硬件置'1'。 由软件通过置'1' LSERDYC位来清除。 0: 无外部32kHz振荡器产生的时钟就绪中断; 1: 外部32kHz振荡器导致时钟就绪中断。
位0	LSIRDYF : LSI就绪中断标志 (LSI ready interrupt flag) 在内部低速时钟就绪且LSIRDYIE位被置'1'时, 由硬件置'1'。 由软件通过置'1' LSIRDYC位来清除。 0: 无内部40kHz RC振荡器产生的时钟就绪中断; 1: 内部40kHz RC振荡器导致时钟就绪中断。

A.5 APB2 外设复位寄存器

偏移地址: 0x0C

复位值: 0x0000 0000

访问: 无等待周期, 字, 半字 和字节访问

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
保留															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ADC3 RST	USART1 RST	TIM8 RST	SPI1 RST	TIM1 RST	ADC2 RST	ADC1 RST	IOPG RST	IOPF RST	IOPE RST	IOPD RST	IOPC RST	IOPB RST	IOPA RST	保留	AFIO RST
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	res	RW
位31:16		保留, 始终读为0。													
位15		ADC3RST : ADC3接口复位 (ADC3 interface reset) 由软件置'1'或清'0' 0: 无作用; 1: 复位ADC3接口。													
位14		USART1RST : USART1复位 (USART1 reset) 由软件置'1'或清'0' 0: 无作用; 1: 复位USART1。													

位13	TIM8RST: TIM8定时器复位 (TIM8 timer reset) 由软件置'1'或清'0' 0: 无作用; 1: 复位TIM8定时器。
位12	SPI1RST: SPI1复位 (SPI 1 reset) 由软件置'1'或清'0' 0: 无作用; 1: 复位SPI1。
位11	TIM1RST: TIM1定时器复位 (TIM1 timer reset) 由软件置'1'或清'0' 0: 无作用; 1: 复位TIM1定时器。
位10	ADC2RST: ADC2接口复位 (ADC 2 interface reset) 由软件置'1'或清'0' 0: 无作用; 1: 复位ADC2接口。
位9	ADC1RST: ADC1接口复位 (ADC 1 interface reset) 由软件置'1'或清'0' 0: 无作用; 1: 复位ADC1接口。
位8	IOPGRST: IO端口G复位 (IO port G reset) 由软件置'1'或清'0' 0: 无作用; 1: 复位IO端口G。
位7	IOPFRST: IO端口F复位 (IO port F reset) 由软件置'1'或清'0' 0: 无作用; 1: 复位IO端口F。
位6	IOPERST: IO端口E复位 (IO port E reset) 由软件置'1'或清'0' 0: 无作用; 1: 复位IO端口E。
位5	IOPDRST: IO端口D复位 (IO port D reset) 由软件置'1'或清'0' 0: 无作用; 1: 复位IO端口D。
位4	IOPCRST: IO端口C复位 (IO port C reset) 由软件置'1'或清'0' 0: 无作用; 1: 复位IO端口C。
位3	IOPBRST: IO端口B复位 (IO port B reset) 由软件置'1'或清'0' 0: 无作用; 1: 复位IO端口B。
位2	IOPARST: IO端口A复位 (IO port A reset) 由软件置'1'或清'0' 0: 无作用; 1: 复位IO端口A。
位1	保留, 始终读为0。

位0	AFIORST: 辅助功能IO复位 (Alternate function I/O reset) 由软件置'1'或清'0' 0: 无作用; 1: 复位辅助功能。
----	--

A.6 APB1 外设复位寄存器

偏移地址: 0x10

复位值: 0x0000 0000

访问: 无等待周期, 字, 半字和字节访问

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
保留	DACRST	PWR RST	BKP RST	保留	CAN RST	保留	USB RST	I2C2 RST	I2C1 RST	UART5R ST	UART4R ST	USART3 RST	USART2 RST	保留	
		RW	RW	RW		RW		RW	RW	RW	RW	RW	RW	RW	
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SPI3 RST	SPI2 RST	保留	WWDG RST	保留	保留	保留	保留	保留	TIM7 RST	TIM6 RST	TIM5 RST	TIM4 RST	TIM3 RST	TIM2 RST	
RW	RW		RW							RW	RW	RW	RW	RW	RW

位31:30	保留, 始终读为0。
位29	DACRST: DAC接口复位 (DAC interface reset) 由软件置'1'或清'0' 0: 无作用; 1: 复位DAC接口。
位28	PWRRST: 电源接口复位 (Power interface reset) 由软件置'1'或清'0' 0: 无作用; 1: 复位电源接口。
位27	BKPRST: 备份接口复位 (Backup interface reset) 由软件置'1'或清'0' 0: 无作用; 1: 复位备份接口。
位26	保留, 始终读为0。
位25	CANRST: CAN复位 (CAN reset) 由软件置'1'或清'0' 0: 无作用; 1: 复位CAN。
位24	保留, 始终读为0。
位23	USBRST: USB复位 (USB reset) 由软件置'1'或清'0' 0: 无作用; 1: 复位USB。
位22	I2C2RST: I2C 2复位 (I2C 2 reset) 由软件置'1'或清'0' 0: 无作用; 1: 复位I2C 2。
位21	I2C1RST: I2C 1复位 (I2C 1 reset) 由软件置'1'或清'0' 0: 无作用; 1: 复位I2C 1。

位20	UART5RST : UART5复位 (UART 5 reset) 由软件置'1'或清'0' 0: 无作用; 1: 复位UART5。
位19	UART4RST : UART4复位 (UART 4 reset) 由软件置'1'或清'0' 0: 无作用; 1: 复位UART4。
位18	USART3RST : USART3复位 (USART 3 reset) 由软件置'1'或清'0' 0: 无作用; 1: 复位USART3。
位17	USART2RST : USART2复位 (USART 2 reset) 由软件置'1'或清'0' 0: 无作用; 1: 复位USART2。
位16	保留, 始终读为0。
位15	SPI3RST SPI3 复位 (SPI 3 reset) 由软件置'1'或清'0' 0: 无作用; 1: 复位SPI3。
位14	SPI2RST : SPI2复位 (SPI 2 reset) 由软件置'1'或清'0' 0: 无作用; 1: 复位SPI2。
位13:12	保留, 始终读为0。
位11	WWDGRST : 窗口看门狗复位 (Window watchdog reset) 由软件置'1'或清'0' 0: 无作用; 1: 复位窗口看门狗。
位10:6	保留, 始终读为0。
位5	TIM7RST : 定时器7复位 (Timer 7 reset) 由软件置'1'或清'0' 0: 无作用; 1: 复位TIM7定时器。
位4	TIM6RST : 定时器6复位 (Timer 6 reset) 由软件置'1'或清'0' 0: 无作用; 1: 复位TIM6定时器。
位3	TIM5RST : 定时器5复位 (Timer 5 reset) 由软件置'1'或清'0' 0: 无作用; 1: 复位TIM5定时器。
位2	TIM4RST : 定时器4复位 (Timer 4 reset) 由软件置'1'或清'0' 0: 无作用; 1: 复位TIM4定时器。
位1	TIM3RST : 定时器3复位 (Timer 3 reset) 由软件置'1'或清'0' 0: 无作用; 1: 复位TIM3定时器。
位0	TIM2RST : 定时器2复位 (Timer 2 reset) 由软件置'1'或清'0' 0: 无作用; 1: 复位TIM2定时器。

A.7 AHB 外设时钟使能寄存器

偏移地址: 0x14

复位值: 0x0000 0014

访问: 无等待周期, 字, 半字 和 字节访问

注: 当外设时钟没有启用时, 软件不能读出外设寄存器的数值, 返回的数值始终是0x0。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
保留															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
保留					SDIOEN	保留	FSMCEN	保留	CRCEN	保留	FLITF EN	保留	SRAM EN	DMA2 EN	DMA1 EN
					I'W		I'W		I'W		I'W		I'W	I'W	I'W

位31:11	保留, 始终读为0。
位10	SDIOEN : SDIO时钟使能 (SDIO clock enable) 由软件置'1'或清'0'。 0: SDIO时钟关闭; 1: SDIO时钟开启。
位9	保留, 始终读为0。
位8	FSMCEN : FSMC时钟使能 (FSMC clock enable) 由软件置'1'或清'0'。 0: FSMC时钟关闭; 1: FSMC时钟开启。
位7	保留, 始终读为0。
位6	CRCEN : CRC时钟使能 (CRC clock enable) 由软件置'1'或清'0'。 0: CRC时钟关闭; 1: CRC时钟开启。
位5	保留, 始终读为0。
位4	FLITFEN : 闪存接口电路时钟使能 (FLITF clock enable) 由软件置'1'或清'0'来开启或关闭睡眠模式时闪存接口电路时钟。 0: 睡眠模式时闪存接口电路时钟关闭; 1: 睡眠模式时闪存接口电路时钟开启。
位3	保留, 始终读为0。
位2	SRAMEN : SRAM时钟使能 (SRAM interface clock enable) 由软件置'1'或清'0'来开启或关闭睡眠模式时SRAM时钟。 0: 睡眠模式时SRAM时钟关闭; 1: 睡眠模式时SRAM时钟开启。

位1	DMA2EN : DMA2时钟使能 (DMA2 clock enable) 由软件置'1'或清'0'。 0: DMA2时钟关闭; 1: DMA2时钟开启。
位0	DMA1EN : DMA1时钟使能 (DMA1 clock enable) 由软件置'1'或清'0'。 0: DMA1时钟关闭; 1: DMA1时钟开启。

A.8 APB2 外设时钟使能寄存器

偏移地址：0x18

复位值：0x0000 0000

访问：字，半字和字节访问

通常无访问等待周期。但在APB2总线上的外设被访问时，将插入等待状态直到APB2的外设访问结束。

注： 当外设时钟没有启用时，软件不能读出外设寄存器的数值，返回的数值始终是0x0。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
保留															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ADC3 EN	USART1 EN	TIM8 EN	SPI1 EN	TIM1 EN	ADC2 EN	ADC1 EN	IOPG EN	IOPF EN	IOPE EN	IOPD EN	IOPC EN	IOPB EN	IOPA EN	保留	AFIO EN
1'W	1'W	1'W	1'W	1'W	1'W	1'W	1'W	1'W	1'W	1'W	1'W	1'W	1'W	1'W	1'W
位31:16	保留，始终读为0。														
位15	ADC3EN : ADC3接口时钟使能 (ADC 3 interface clock enable) 由软件置'1'或清'0' 0: ADC3接口时钟关闭; 1: ADC3接口时钟开启。														
位14	USART1EN : USART1时钟使能 (USART1 clock enable) 由软件置'1'或清'0' 0: USART1时钟关闭; 1: USART1时钟开启。														
位13	TIM8EN : TIM8定时器时钟使能 (TIM8 Timer clock enable) 由软件置'1'或清'0' 0: TIM8定时器时钟关闭; 1: TIM8定时器时钟开启。														
位12	SPI1EN : SPI1时钟使能 (SPI 1 clock enable) 由软件置'1'或清'0' 0: SPI1时钟关闭; 1: SPI1时钟开启。														
位11	TIM1EN : TIM1定时器时钟使能 (TIM1 Timer clock enable) 由软件置'1'或清'0' 0: TIM1定时器时钟关闭; 1: TIM1定时器时钟开启。														
位10	ADC2EN : ADC2接口时钟使能 (ADC 2 interface clock enable) 由软件置'1'或清'0' 0: ADC2接口时钟关闭; 1: ADC2接口时钟开启。														

位9	ADC1EN: ADC1接口时钟使能 (ADC 1 interface clock enable) 由软件置'1'或清'0' 0: ADC1接口时钟关闭; 1: ADC1接口时钟开启。
位8	IOPGEN: IO端口G时钟使能 (I/O port G clock enable) 由软件置'1'或清'0' 0: IO端口G时钟关闭; 1: IO端口G时钟开启。
位7	IOPFEN: IO端口F时钟使能 (I/O port F clock enable) 由软件置'1'或清'0' 0: IO端口F时钟关闭; 1: IO端口F时钟开启。
位6	IOPEEN: IO端口E时钟使能 (I/O port E clock enable) 由软件置'1'或清'0' 0: IO端口E时钟关闭; 1: IO端口E时钟开启。
位5	IOPDEN: IO端口D时钟使能 (I/O port D clock enable) 由软件置'1'或清'0' 0: IO端口D时钟关闭; 1: IO端口D时钟开启。
位4	IOPCEN: IO端口C时钟使能 (I/O port C clock enable) 由软件置'1'或清'0' 0: IO端口C时钟关闭; 1: IO端口C时钟开启。
位3	IOPBEN: IO端口B时钟使能 (I/O port B clock enable) 由软件置'1'或清'0' 0: IO端口B时钟关闭; 1: IO端口B时钟开启。
位2	IOPAEN: IO端口A时钟使能 (I/O port A clock enable) 由软件置'1'或清'0' 0: IO端口A时钟关闭; 1: IO端口A时钟开启。
位1	保留, 始终读为0。
位0	AFIOEN: 辅助功能IO时钟使能 (Alternate function I/O clock enable) 由软件置'1'或清'0' 0: 辅助功能IO时钟关闭; 1: 辅助功能IO时钟开启。

A.9 APB1 外设时钟使能寄存器

偏移地址: 0x1C

复位值: 0x0000 0000

访问: 字、半字和字节访问

通常无访问等待周期。但在APB1总线上的外设被访问时, 将插入等待状态直到APB1外设访问结束。

注: 当外设时钟没有启用时, 软件不能读出外设寄存器的数值, 返回的数值始终是0x0。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
保留	DACEN	PWR EN	BKP EN	保留	CAN EN	保留	USB EN	I2C2 EN	I2C1 EN	UART5 EN	UART4 EN	USART3 EN	USART2 EN	保留	
	r'w	r'w	r'w		r'w		r'w	r'w	r'w	r'w	r'w	r'w	r'w		
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SPI3 EN	SPI2 EN	保留	WWDG EN	保留						TIM7 EN	TIM6 EN	TIM5 EN	TIM4 EN	TIM3 EN	TIM2 EN
r'w	r'w		r'w							r'w	r'w	r'w	r'w	r'w	r'w

位31:30	保留，始终读为0。
位29	DACEN : DAC接口时钟使能 (DAC interface clock enable) 由软件置'1'或清'0' 0: DAC接口时钟关闭; 1: DAC接口时钟开启。
位28	PWREN : 电源接口时钟使能 (Power interface clock enable) 由软件置'1'或清'0' 0: 电源接口时钟关闭; 1: 电源接口时钟开启。
位27	BKPEN : 备份接口时钟使能 (Backup interface clock enable) 由软件置'1'或清'0' 0: 备份接口时钟关闭; 1: 备份接口时钟开启。
位26	保留，始终读为0。
位25	CANEN : CAN时钟使能 (CAN clock enable) 由软件置'1'或清'0' 0: CAN时钟关闭; 1: CAN时钟开启。
位24	保留，始终读为0。
位23	USBEN : USB时钟使能 (USB clock enable) 由软件置'1'或清'0' 0: USB时钟关闭; 1: USB时钟开启。
位22	I2C2EN : I2C 2时钟使能 (I2C 2 clock enable) 由软件置'1'或清'0' 0: I2C 2时钟关闭; 1: I2C 2时钟开启。
位21	I2C1EN : I2C 1时钟使能 (I2C 1 clock enable) 由软件置'1'或清'0' 0: I2C 1时钟关闭; 1: I2C 1时钟开启。
位20	UART5EN : UART5时钟使能 (UART 5 clock enable) 由软件置'1'或清'0' 0: UART5时钟关闭; 1: UART5时钟开启。
位19	UART4EN : UART4时钟使能 (UART 4 clock enable) 由软件置'1'或清'0' 0: UART4时钟关闭; 1: UART4时钟开启。

位18	USART3EN: USART3时钟使能 (USART 3 clock enable) 由软件置'1'或清'0' 0: USART3时钟关闭; 1: USART3时钟开启。
位17	USART2EN: USART2时钟使能 (USART 2 clock enable) 由软件置'1'或清'0' 0: USART2时钟关闭; 1: USART2时钟开启。
位16	保留, 始终读为0。
位15	SPI3EN: SPI 3时钟使能 (SPI 3 clock enable) 由软件置'1'或清'0' 0: SPI 3时钟关闭; 1: SPI 3时钟开启。
位14	SPI2EN: SPI 2时钟使能 (SPI 2 clock enable) 由软件置'1'或清'0' 0: SPI 2时钟关闭; 1: SPI 2时钟开启。
位13:12	保留, 始终读为0。
位11	WWDGEN: 窗口看门狗时钟使能 (Window watchdog clock enable) 由软件置'1'或清'0' 0: 窗口看门狗时钟关闭; 1: 窗口看门狗时钟开启。
位10:6	保留, 始终读为0。
位5	TIM7EN: 定时器7时钟使能 (Timer 7 clock enable) 由软件置'1'或清'0' 0: 定时器7时钟关闭; 1: 定时器7时钟开启。
位4	TIM6EN: 定时器6时钟使能 (Timer 6 clock enable) 由软件置'1'或清'0' 0: 定时器6时钟关闭; 1: 定时器6时钟开启。
位3	TIM5EN: 定时器5时钟使能 (Timer 5 clock enable) 由软件置'1'或清'0' 0: 定时器5时钟关闭; 1: 定时器5时钟开启。
位2	TIM4EN: 定时器4时钟使能 (Timer 4 clock enable) 由软件置'1'或清'0' 0: 定时器4时钟关闭; 1: 定时器4时钟开启。
位1	TIM3EN: 定时器3时钟使能 (Timer 3 clock enable) 由软件置'1'或清'0' 0: 定时器3时钟关闭; 1: 定时器3时钟开启。
位0	TIM2EN: 定时器2时钟使能 (Timer 2 clock enable) 由软件置'1'或清'0' 0: 定时器2时钟关闭; 1: 定时器2时钟开启。

A.10 备份域控制寄存器

偏移地址：0x20

复位值：0x0000 0000，只能由备份域复位有效复位

访问：0到3等待周期，字、半字和字节访问

当连续对该寄存器进行访问时，将插入等待状态。

注意： 备份域控制寄存器中(RCC_BDCR)的LSEON、LSEBYP、RTCSEL和RTCEN位处于备份域。因此，这些位在复位后处于写保护状态，只有在电源控制寄存器(PWR_CR)中的DBP位置'1'后才能对这些位进行改动。进一步信息请参考5.1节。这些位只能由备份域复位清除(见6.1.3节)。任何内部或外部复位都不会影响这些位。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
保留															BDRST
															RW
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RTC EN	保留					RTCSEL[1:0]		保留					LSE BYP	LSE RDY	LSEON
RW						RW	RW						RW	R	RW

位31:17	保留，始终读为0。
位16	BDRST ：备份域软件复位 (Backup domain software reset) 由软件置'1'或清'0' 0：复位未激活； 1：复位整个备份域。
位15	RTCEN ：RTC时钟使能 (RTC clock enable) 由软件置'1'或清'0' 0：RTC时钟关闭； 1：RTC时钟开启。
位14:10	保留，始终读为0。
位9:8	RTCSEL[1:0] ：RTC时钟源选择 (RTC clock source selection) 由软件设置来选择RTC时钟源。一旦RTC时钟源被选定，直到下次后备域被复位，它不能被改变。可通过设置BDRST位来清除。 00：无时钟； 01：LSE振荡器作为RTC时钟； 10：LSI振荡器作为RTC时钟； 11：HSE振荡器在128分频后作为RTC时钟。
位7:3	保留，始终读为0。
位2	LSEBYP ：外部低速时钟振荡器旁路 (External low-speed oscillator bypass) 在调试模式下由软件置'1'或清'0'来旁路LSE。只有在外部32kHz振荡器关闭时，才能写入该位 0：LSE时钟未被旁路； 1：LSE时钟被旁路。
位1	LSERDY ：外部低速LSE就绪 (External low-speed oscillator ready) 由硬件置'1'或清'0'来指示是否外部32kHz振荡器就绪。在LSEON被清零后，该位需要6个外部低速振荡器的周期才被清零。 0：外部32kHz振荡器未就绪； 1：外部32kHz振荡器就绪。
位0	LSEON ：外部低速振荡器使能 (External low-speed oscillator enable) 由软件置'1'或清'0' 0：外部32kHz振荡器关闭； 1：外部32kHz振荡器开启。

A.11 控制/状态寄存器

当连续对该寄存器进行访问时，将插入等待状态。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
LPWR RSTF	WWDG RSTF	IWDG RSTF	SFT RSTF	POR RSTF	PIN RSTF	保留	RMVF	保留							
RW	RW	RW	RW	RW	RW		RW								
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
保留													LSI RDY	LSION	
													r	RW	

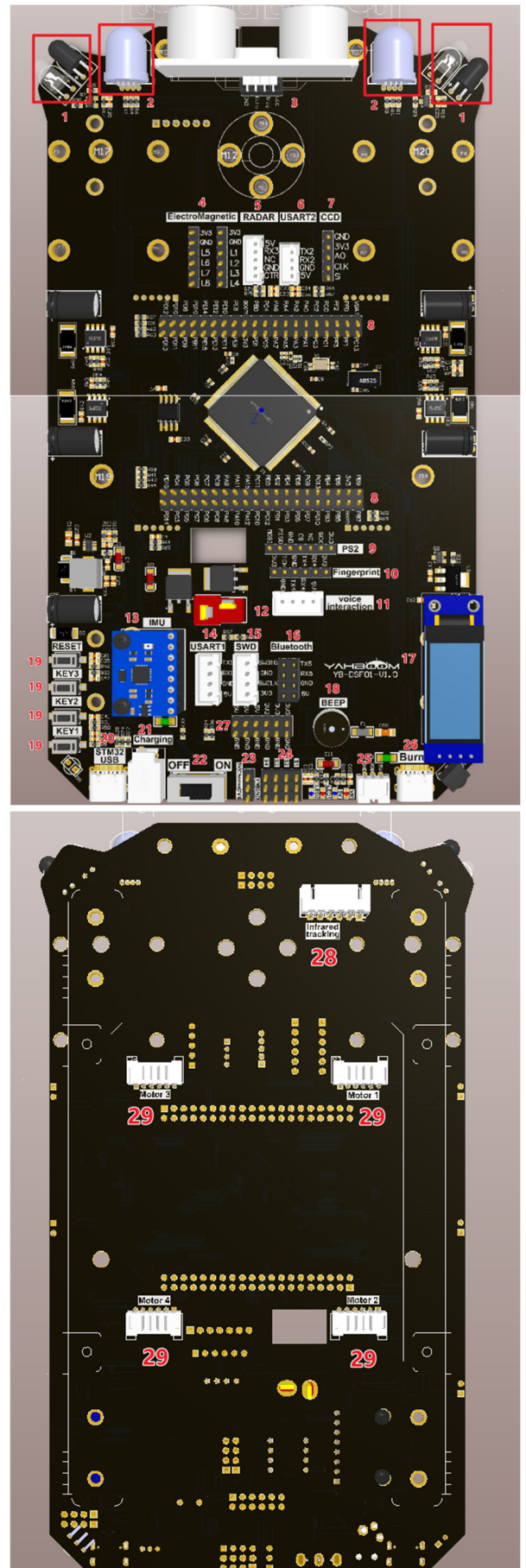
位31	LPWRRSTF : 低功耗复位标志 (Low-power reset flag) 在低功耗管理复位发生时由硬件置'1'; 由软件通过写RMVF位清除。 0: 无低功耗管理复位发生; 1: 发生低功耗管理复位。 关于低功耗管理复位的详细信息, 请参考6.1.1节的“低功耗管理复位”。
位30	WWDGRSTF : 窗口看门狗复位标志 (Window watchdog reset flag) 在窗口看门狗复位发生时由硬件置'1'; 由软件通过写RMVF位清除。 0: 无窗口看门狗复位发生; 1: 发生窗口看门狗复位。
位29	IWDGRSTF : 独立看门狗复位标志 (Independent watchdog reset flag) 在独立看门狗复位发生在V _{DD} 区域时由硬件置'1'; 由软件通过写RMVF位清除。 0: 无独立看门狗复位发生; 1: 发生独立看门狗复位。
位28	SFTRSTF : 软件复位标志 (Software reset flag) 在软件复位发生时由硬件置'1'; 由软件通过写RMVF位清除。 0: 无软件复位发生; 1: 发生软件复位。
位27	PORRSTF : 上电/掉电复位标志 (POR/PDR reset flag) 在上电/掉电复位发生时由硬件置'1'; 由软件通过写RMVF位清除。 0: 无上电/掉电复位发生; 1: 发生上电/掉电复位。
位26	PINRSTF : NRST引脚复位标志 (PIN reset flag) 在NRST引脚复位发生时由硬件置'1'; 由软件通过写RMVF位清除。 0: 无NRST引脚复位发生; 1: 发生NRST引脚复位。
位25	保留, 读操作返回0
位24	RMVF : 清除复位标志 (Remove reset flag) 由软件置'1'来清除复位标志。 0: 无作用; 1: 清除复位标志。
位23:2	保留, 读操作返回0
位1	LSIRDY : 内部低速振荡器就绪 (Internal low-speed oscillator ready) 由硬件置'1'或清'0'来指示内部40kHz RC振荡器是否就绪。在LSION清零后, 3个内部40kHz RC振荡器的周期后LSIRDY被清零。 0: 内部40kHz RC振荡器时钟未就绪; 1: 内部40kHz RC振荡器时钟就绪。

位0	LSION : 内部低速振荡器使能 (Internal low-speed oscillator enable) 由软件置'1'或清'0'。 0: 内部40kHz RC振荡器关闭; 1: 内部40kHz RC振荡器开启。
----	--

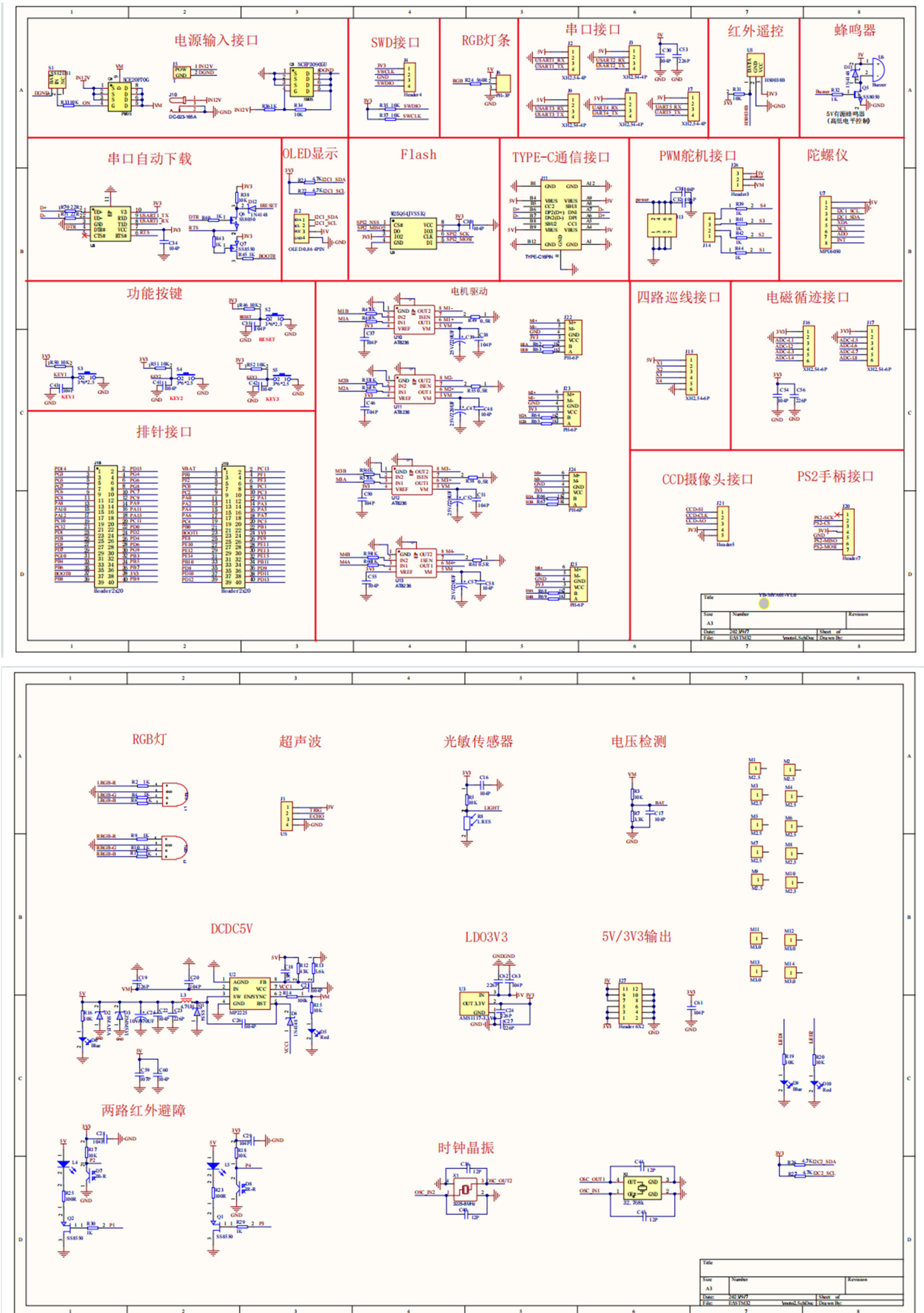
附录 B 单片机系统原理图

B.1 开发板实物引脚分配图

1. 红外避障模块
2. RGB 探照灯
3. 超声波模块接口
4. 电磁循迹接口
5. X3 雷达接口
6. 串口 2 接口：用于外接 k210 视觉模块
7. 线性 CCD 摄像头接口
8. STM32f103ZET6 引出的引脚
9. PS2 手柄接收器接口
10. 指纹模块接口
11. 语音交互模块接口
12. T 型 DC 7.4V 电源输入接口：主电源输入，连接到 DC 7.4V 电源或者 7.4V 电池
13. MPU6050 接口
14. 串口 1 接口：用于调试串口
15. SWD 接口：用于单片机调试
16. 蓝牙模块接口
17. 0.91 寸 OLED 屏接口
18. 蜂鸣器：板载有源蜂鸣器
19. 按键 KEY1、KEY2、KEY3：用户功能按键，可通过编程实现自定义功能。按键 RESET：板载单片机复位按键
20. STM32-USB 设备接口
21. 充电接口：可给外接的电池充电
22. 开关键：扩展板总的开关按键
23. PWM 舵机电压切换：改变跳线帽的位置可选择 7.4V 或者 5V 电压为 PWM 舵机供电
24. PWM 舵机接口：可连接 7.4V 或者 5V 电压 PWM 舵机，需根据舵机电压在 23 选择对应的电压
25. RGB 灯条接口
26. Type-C 数据接口：给单片机烧录程序
27. (3.3/5V) 输出接口：向外输出 3.3V/5V 电压
28. 四路循迹模块接口
29. 电机接口：用于外接 310 电机



B.2 开发板原理图



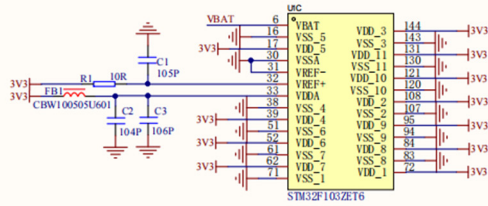
下图中的"X"表示不向外扩展该 IO 引脚接口，只供内部使用

X	RRGB-R	PE2	1	PE2	142	PE1	ADO	X
X	RRGB-G	PE3	2	PE3	141	PE0	INT	X
X	RRGB-B	PE4	3	PE4	138	BOOT0		
X	P1	PE5	4	PE5	132	PG15	LED2	X
X	P3	PE6	5	PE6	129	PG14	LED1	X
		PE0	10	PE0	128	PG13	RGB	X
		PF1	11	PF1	127	PG12	Buzz	X
		PF2	12	PF2	126	PG11	H80038B	X
		PF3	13	PF3	125	PG10		
X	CCDSI	PE4	14	PF4	124	PG9		
X	CCDCLK	PE5	15	PF5	106			
X	CCDAO	PE6	18	PF6	NC			
X	BAT	PE7	19	PF7	92	PG7		
X	LIGHT	PE8	20	PF8	91	PG6		
X	P2	PE9	21	PF9	90	PG5	KEY3	
X	P4	PE10	22	PF10	89	PG4	KEY2	
	OSC_IN2	PE3	23	PF3	88	PG3	KEY1	
	OSC_OUT2	PE4	24	PF4	87	PG2	LRGB-B	X
	RISE	PE5	25	PF5	86	PG1		
X	TRIG	PE11	49	PF11	67	PE14	M4B	
X	EXIO	PE12	50	PF12	66	PE13	M4A	
X	X1	PE13	53	PF13	65	PE12		
X	X2	PE14	54	PF14	64	PE11	M3B	
X	X3	PE15	55	PF15	63	PE10		
X	X4	PE0	56	PF0	60	PE9	M3A	
X	LRGB-R	PE1	57	PF1	59	PE8		
X	LRGB-G	PE2	58	PF2				

STM32F103ZE16

		7		5A		
OSC_IN1	PC14	8	PC13-TAMPER-RTC	PB9	140	PB8
OSC_OUT1	PC15	9	PC14-OSC32_IN	PB8	139	PB7
ADC-1.1	PC0	26	PC15-OSC32_OUT	PB7	137	PB6
ADC-1.2	PC1	27	PC0	PB6	136	PB5
ADC-1.3	PC2	28	PC1	PB5	135	PB4
ADC-1.4	PC3	29	PC2	PB4	134	PB3
IHA	PA0	34	PC3	PB3	133	PB2
IHB	PA1	35	PA0-KUP	PD7	123	PD7
ADC-1.5	PA2	36	PA1	PD6	122	PD6
ADC-1.6	PA3	37	PA2	PD5	119	PD5
PS2-CS	PA4	40	PA3	PD4	118	PD4
PS2-SCK	PA5	41	PA4	PD3	117	PD3
PS2-MISO	PA6	42	PA5	PD2	116	PD2
PS2-MOSI	PA7	43	PA6	PD1	115	PD1
ADC-1.7	PC4	44	PA7	PD0	114	PD0
ADC-1.8	PC5	45	PC4	PC12	113	PC12
S1	PB0	46	PC5	PC11	112	PC11
S2	PB1	47	PB0	PC10	111	PC10
BOOT1	PB2	48	PB1	PC9	110	PC9
IC2 SCL	PB10	69	PB2	PA15	109	PA15
IC2 SDA	PB11	70	PB10	PA14	108	PA14
SPD_NSS	PB12	73	PB11	PA13	107	PA13
SPD_SCK	PB13	74	PB12	PA12	106	PA12
SPD_MISO	PB14	75	PB13	PA11	105	PA11
SPD_MOSI	PB15	76	PB14	PA10	104	PA10
USART1_TX	PD8	77	PB15	PA9	103	PA9
USART1_RX	PD9	78	PD8	PC9	99	PC9
	PD10	79	PD9	PC8	98	PC8
	PD11	80	PD10	PC7	97	PC7
	PD12	81	PD11	PC6	96	PC6
	PD13	82	PD12	PC5	85	PC5
	PD13	82	PD13	PC4	84	PC4

STM32F103ZE16



STM32F103ZE16

